SHIVA ZEN

Sistema de Gestão para Clínica de Estética

Jaqueline Aranha Estética

Documento Técnico
Arquitetura, dados, segurança, qualidade, operação e custo

Versão 1.1 | 30/10/2026

Sumário

1. Introdução 4
2. Stack 5
3. Arquitetura 6
3.1 Estrutura de pastas 6
3.2 Camadas e fluxo de uma requisição 7
4. Decisões de arquitetura 14
5. Banco de dados 18
5.1 Resumo 18
5.2 Tabelas 18
5.3 Invariantes garantidas pelo banco 21
5.4 Lógica no banco (0056–0060, só Postgres) 23
5.5 Padrão de nomes e metadados 25
6. Segurança 29
7. LGPD — implementação 30
7.1 Bases legais (política de privacidade e termo LGPD v1.0) 30
7.2 Consentimentos e provas 30
7.3 Direitos do titular 31
7.4 Retenção 32
7.5 Incidente (art. 48) 33
8. Integrações externas 34
9. Rotinas automáticas (jobs) 36
10. API (OpenAPI 3.1) 38
10.1 Contrato das respostas 38
10.2 Autenticação 38
10.3 Operações 38
11. Rotas 40
11.1 Site público 40
11.2 Agendamento e auto-atendimento da cliente 40
11.3 Equipe 41
11.4 Webhooks e infraestrutura 43
12. Testes e qualidade 44

13. Implantação e configuração 47
13.1 Ambiente dev publicado (testes, sai da branch dev) 47
13.2 Serviço Cron (jobs) 48
13.3 Variáveis de ambiente 49
14. Convenções de código 52
15. Pendências técnicas e de operação 54
16. Custo operacional 55
16.1 Premissas 55
16.2 Itens de custo 55
16.3 Estimativa da hospedagem (Railway) 56
16.4 Custo mensal por cenário 56
16.5 E-mail: qual plano 57
16.6 Alternativas para gastar menos 57
16.7 Custos que não são mensais 58
16.8 Travas de custo no sistema 58
16.9 Esforço de operação (tempo, não dinheiro) 59
16.10 Fontes (consultadas em 30/10/2026) 59
17. Identidade visual, design e marketing 61
17.1 Posicionamento 61
17.2 Paleta e psicologia das cores 61
17.3 Tipografia 63
17.4 Tom de voz e conteúdo 63
17.5 Experiência e layout 64
17.6 Acessibilidade como decisão de design 64
17.7 Recursos de marketing do sistema 65
17.8 Métricas para acompanhar 65
17.9 Referências 66

1. Introdução

Este documento reúne as questões técnicas do Shiva Zen — como o sistema é construído, onde vive cada responsabilidade, o que o banco garante, como ele é testado, publicado e operado, o que ainda está pendente, quanto custa por mês mantê-lo no ar e as escolhas de design e marketing. O conteúdo é extraído dos documentos vivos do repositório (PROJECT.md, ARCHITECTURE.md, padrão SQL, contrato OpenAPI, CUSTOS-OPERACIONAIS.md e DESIGN-E-MARKETING.md), atualizados a cada mudança de código.

Item	Quantidade
Tabelas do domínio	38
Migrations	79
Testes automatizados	2090
Rotas HTTP do sistema	34
Caminhos documentados em OpenAPI	27
Decisões de arquitetura registradas	51

Sistema single-tenant da clínica de estética Jaqueline Aranha (São José do Rio Preto/SP). Um único projeto Django, renderizado no servidor, com duas superfícies:

Superfície	Quem usa	Base de template
Site público — vitrine, agendamento online, "Meus agendamentos", lista de espera, NPS, termos, LGPD	Clientes (sem conta/senha; identidade = celular + OTP por SMS)	`estrutura/base_v2.html`
Painel da equipe — agenda, clientes, ficha/prontuário, pacotes, promoções, comissões, NPS, termos, auditoria, Branding	ADMIN (painel completo, 2FA obrigatório)	`painel/base_v2.html` (app-shell PWA)
Portal do profissional — agenda própria, aprovar/rejeitar pedidos, "Realizado", anotações, ficha das próprias clientes, "Minhas comissões"	PROFISSIONAL	`profissional/base.html` (casca própria: sem rodapé, CTA, WhatsApp, SEO nem aviso de cookies do site; mesmos tokens e bundle)

Só dois papéis de equipe: **ADMIN** e **PROFISSIONAL** (a dona faz a recepção pelo painel; o papel RECEPCAO saiu na 0072 — ver `AUTENTICACAO-E-ACESSO.md`). Todo atendimento é presencial.

2. Stack

Camada	Tecnologia (versões em `requirements.txt` / `package.json`)
Linguagem	Python 3.12 (`.python-version`), Node 22 no build (`.nvmrc`; Tailwind v4 exige ≥ 20)
Web	Django 5.2 (MVT, server-render), gunicorn 23, WhiteNoise 6 (estáticos `CompressedManifest` em prod)
Banco	PostgreSQL 18 em produção e CI; SQLite (`db_dev.sqlite3`; testes em `.cache/db_test.sqlite3`) em dev e testes rápidos
Front-end	Vite 6 + Tailwind CSS v4 (`@tailwindcss/vite`) + Alpine.js via `@alpinejs/csp` + django-vite 3 + django-cotton 2. **Sem HTMX, Bootstrap ou jQuery**
Libs de terceiros no front	FullCalendar 6.1.21 (`painel/calendar.html`), Chart.js 4.5.1 (`painel/overview.html`) e swagger-ui-dist 5.34.0 (`painel/api.html`): dependências npm com versão exata, empacotadas pelo Vite (entradas `painel-calendario`, `painel-visao-geral` e `painel-api`) — **nenhum CDN**. A telemetria de instalação do Swagger (`@scarf/scarf`) está desligada em `package.json` (`scarfSettings`)
Fontes	Playfair Display + Lato servidas pelo próprio site (`@fontsource`, subconjunto latin, no build do Vite; sem Google Fonts) — tokens em `static/src/css/tokens.css`
Jobs / fila	Sem Celery: jobs são funções (`jobs.py`) com horário na `JOB_AGENDA`, chamadas pelo serviço Cron do Railway (`manage.py rodar_jobs_devidos`, a cada 15 min), pelo cron HTTP `/cron/run/<job>/` ou por `manage.py rodar_job`; e-mail transacional com fila no banco (`Notificacao` EMAIL, reenvio pelo job `emails`)
Cache / sessão	LocMemCache + sessão no banco (padrão, 1 worker). `REDIS_URL` troca para django-redis + sessão em cache
Auth	Usuário próprio (`AbstractBaseUser`, e-mail), django-axes 8, django-otp + django-two-factor-auth (só models/TOTP; rotas `/account/*` **não** publicadas), django-ratelimit 4
API	Sem API REST pública (a DRF saiu: não tinha consumidor). As rotas de máquina — JSON do wizard, do portal e do painel, webhooks, cron, feed ICS, saúde — são views Django no contrato `utils/respostas`, descritas em **OpenAPI 3.1** (`aranha_estetica/api/openapi.yaml`, escrito à mão e conferido por `tests/test_openapi.py`) e navegáveis no **Swagger UI** em `/painel/api/` (só ADMIN)
Integrações	Zenvia SMS (OTP), WhatsApp Cloud API (Meta), e-mail (backend Django), pywebpush (VAPID), Cloudflare Turnstile, Sentry. A agenda da profissional sai só pelo feed ICS assinado (sem Google Calendar)
Outros	bleach (HTML de promoção), openpyxl (Excel), python-dateutil (RRULE), python-json-logger (logs JSON em prod), qrcode (QR SVG do 2FA, sem Pillow)
Deploy	Railway (Dockerfile multi-stage + `railway.json` do serviço web e `railway.cron.json` do serviço Cron)
CI	GitHub Actions `.github/workflows/ci.yml`

`requirements.lock` fixa as dependências transitivas (usado com `-c` no CI e no Docker).

3. Arquitetura

3.1 Estrutura de pastas

```
aranha_estetica/
├── models/          acesso, clientes, profissionais, agenda (AgendaHorario/AgendaExcecao), procedimentos,
│                    agendamentos (Atendimento/FSM), notificacoes, links (LinkAcesso), consentimentos,
│                    prontuario, anamnese, termos, nps, pacotes, sistema, push, carteira, comissao,
│                    historico (trilha historico_alteracao), execucoes (ExecucaoJob), configuracao
│                    (Configuracao + o registro tipado das regras operacionais), mixins
├── db/              peças de banco usadas por migrations (Indice, PadronizarNomes) — congelado
├── migrations/sql/  texto SQL das rotinas por versão (v0056 triggers, v0057 leitura/views, v0058
│                    procedures, v0059 trilhas append-only, v0065 faturamento por competência, v0066
│                    comissão, v0072 links, v0074 anonimização, v0077 retenção de saúde/pacote e
│                    pseudonimização, v0079 metadados em inglês) — congelado, importado só pela migration da versão
├── urls/            uma lista de rotas por área (publico, conta, painel, profissional, infra), todas no
│                    namespace `aranha` — espelha views/
├── views/publico/   site (páginas institucionais, `/duvidas/`), agendamento + agendamento_api (wizard e horários),
│                    verificacao (OTP), portal (meus agendamentos: reagendar/cancelar pela sessão do OTP,
│                    preferências, revogação de saúde, pedidos LGPD, baixar meus dados), confirmacao
│                    (presença), agenda_profissional (agendar por profissional/embed), formularios
│                    (anamnese/pesquisa), nps, termos (aceite), lgpd (política, "Meus dados",
│                    descadastro), lista_espera, contato
├── views/conta/     autenticacao (login, logout, recuperação de senha), dois_fatores (2FA TOTP)
├── views/painel/    um módulo por tela da equipe: visao_geral, agendamentos, aprovacao, calendario,
│                    retornos, clientes, prontuario, anamneses, procedimentos, profissionais, excecoes,
│                    bloqueios, pacotes, promocoes, lista_espera, financeiro, comissoes, nps, termos,
│                    conformidade_termos, notificacoes, usuarios, configuracoes, marca, previa_email,
│                    auditoria, api (Swagger UI + o openapi.yaml)
├── views/profissional/  portal (agenda do profissional, aprovar, realizar, anotar, link do ICS, minhas comissões)
├── views/infra/     healthcheck, cron (HTTP), webhooks (WhatsApp/Zenvia), webpush, feed_ics
├── views/comum.py   helpers de view usados por mais de uma área (link do feed ICS do painel e do portal);
│                    as áreas não se importam (contrato "Áreas de views independentes" do import-linter)
├── services/        agenda (reservar/reagendar/mover), atendimento (efeitos de criar/mudar status),
│                    agendamento_online (o pedido pelo site), comunicacao (canais, fila de e-mail),
│                    agendamento (avisos), disponibilidade (SlotService, horários agrupados, dias com
│                    expediente), precos (vitrine), catalogo (preço base e habilitações), otp, lgpd,
│                    termos, alertas (alertas e marcadores das listas em lote, leitura auditada),
│                    prontuario (vínculo clínico), comissao, fidelidade, retorno, lista_espera
│                    (inscrição e aviso de vaga), portal_cliente (sessão e pedidos LGPD da área da
│                    cliente), financeiro (competência), relatorios (Excel), indicadores, pacotes,
│                    links, consentimentos (inclusive o de saúde), anamnese, push, lembretes (D-1 e
│                    sem desfecho), nps (cálculo, envio, detrator), marketing (promoção e aniversário),
│                    feriados, prova_social (números do site público só com dado real, cache de 10 min)
├── utils/           branding, email, sms, whatsapp, telefone (forma canônica, E.164, máscara), precos,
│                    saude (alertas em lote e o marcador de consentimento revogado), fichas, seguranca,
│                    dois_fatores, pii, captcha (Turnstile e `captcha_invalido`), datas (inclusive
│                    `ler_instante_iso` do FullCalendar), parse, busca (trigram no PG), auditoria,
│                    recorrencia (RRULE do subconjunto), respostas (contrato JSON), banco (GUCs app.* e
│                    CALL das procedures), cifragem, formatos
├── conteudo/        textos que também são regra, gerados de constantes (sem Django): lgpd.py (Política
│                    de Privacidade v1.0 — texto, TRATAMENTOS, OPERADORES, PRAZOS_RETENCAO: fonte do
│                    termo LGPD, da página e do DSAR) e faq.py (perguntas frequentes de `/duvidas/`;
│                    `em_destaque` = as 5 da home)
├── forms/           agendamento.py (AgendamentoPublicoForm: o POST do wizard → PedidoAgendamento),
│                    agenda.py (AgendamentoPainelForm do novo agendamento pela recepção,
│                    ValorAtendimentoForm, AgendaExcecaoForm e TIPOS_PAINEL), cadastros.py
│                    (Usuario/UsuarioEdicao, Profissional, GradeSemanal, Procedimento, Configuracao,
│                    Marca), cliente.py (ClientePainelForm, ClienteRapidoForm), painel.py (BloqueioForm,
│                    TermoForm, CampoPreco/parse_preco, EscolhaPorId, ProfissionalOuTodos,
│                    avisar_erros), publico.py (ListaEsperaForm, ContatoForm: honeypot), financeiro.py
│                    (pacote, cancelar/prorrogar compra, regra de comissão, repasse), promocoes.py, nps.py
├── management/commands/  seed, bootstrap_admin, migrate_atomico, setup_2fa, carregar_feriados, rodar_job,
│                    rodar_jobs_devidos, gerar_chave_cifragem, rotacionar_chave_cifragem,
│                    reaplicar_esquecimentos, consultas_lentas
├── templatetags/    painel_tags (`MENU_PAINEL`: menu do painel; filtro `iniciais` do avatar), site_tags
│                    (inclusive o filtro `texto_termo` da política, a tag `{% foto %}` e o monograma da
│                    profissional sem foto), formatos, dict_extras
├── templates/       estrutura/ (bases), cotton/ (componentes do site e do design system da equipe),
│                    publico/ (site; servicos/), agendamento/ (wizard, portal da cliente, links), conta/
│                    (login, senha), painel/ (`base_v2.html` = casca), profissional/ (`base.html` = casca
│                    do portal), partials/, email/, pwa/, erros (400/403/403_csrf/404/429/500),
│                    axes_bloqueio.html, robots.txt
├── static/src/      fonte do Vite — css/tokens.css, css/app.css, css/componentes.css (design system das
│                    telas da equipe) e css/site.css (páginas do site público), e uma entrada JS por tela
│                    (app.js, wizard.js, painel.js — abas, menu "⋯", modais, Ctrl+K —, painel-busca.js,
│                    painel-status.js — seletor de status e lote da agenda —, painel-calendario.js
│                    (FullCalendar), painel-visao-geral.js (Chart.js), painel-api.js (Swagger UI) e
```

```
│                   webpush.js; nome com hash no build)
│                   + lib/requisicao.js (POST JSON com CSRF no contrato utils/respostas) — fora do
│                   collectstatic
├── static/assets/clinica/variantes/  fotos do site em WebP + JPEG (480/768/1024 px) e o manifest.json
│                   lido pelo `{% foto %}` — geradas por `scripts/imagens.py` e versionadas
├── static/dist/    build do Vite (gitignored; gerado no Docker/CI)
├── api/openapi.yaml contrato HTTP das rotas de máquina (OpenAPI 3.1): parâmetros, corpos, códigos de erro,
│                   autenticação e limites; `x-url-name` liga cada caminho à rota do Django
├── signals.py      só os caches que dependem de Configuracao (post_save/post_delete): Branding e números
│                   do site (`services/prova_social`)
├── jobs.py / jobs_manutencao.py     jobs periódicos (`JOB_MAP`, `JOB_AGENDA` = horários, `executar`)
├── middleware.py   Enforce2FAMiddleware, SecurityHeadersMiddleware, ContentSecurityPolicyMiddleware,
│                   ContextoBancoMiddleware (abre o Ator do request)
├── checks.py       system checks de produção aranha.W001–W016 e erros E001–E004 (§14 f)
├── excecoes.py     RegraDeNegocio (mensagem + codigo), HorarioIndisponivel, TransicaoInvalida, TermoPendente
├── ator.py         Ator (usuário, IP, origem) da borda em ContextVar → LogAuditoria + GUCs app.*
├── inventario_lgpd.py  tabela → classe (titular/equipe/sistema); coluna pessoal → anonimiza/apaga/purga/retém
├── context_processors.py · decorators.py · validators.py · constants.py · sitemaps.py · admin.py
clinica/            settings/{base,dev,prod,staging,test}.py (leem e validam a env; §15), urls.py,
                    wsgi.py, apps.py, gunicorn_logger.py, logging_filtros.py (redação de token/PII no log)
scripts/            backup/ (dump cifrado e teste de restauração, §14 h) e imagens.py (variantes
                    responsivas das fotos do site; Pillow do requirements-dev) — fora da imagem Docker
docs/               os Markdown vivos (um por assunto; índice no README), specs/ (banco, front, registry),
                    uml/ (PlantUML gerado do código + Lucidchart) e tcc/ (geradores e os DOCX/PDF do TCC)
```

3.2 Camadas e fluxo de uma requisição

```
Navegador ──HTTPS──▶ gunicorn (1 worker × 4 threads)
  └─ middleware: Security → GZip → WhiteNoise → Session → CSRF → Auth → OTP → Messages
                 → XFrame → CSP (nonce) → SecurityHeaders (no-store/same-origin) → Enforce2FA
                 → Axes → Ratelimit
     └─ view (validação + permissão) ──▶ services/ e utils/ (regra de negócio)
          └─ models (FSM do Atendimento, managers, constraints) ──▶ PostgreSQL
               └─ services/atendimento (na mesma transação da FSM): faltas, débito de
                  pacote, comissão, cashback, retorno, lista de espera; push do novo
                  atendimento no on_commit
```

- **FSM do atendimento** (`models/agendamentos.py`): toda mudança de status passa por `aprovar/confirmar/cancelar/marcar_realizado/marcar_falta/marcar_reagendado`, que travam a linha (`select_for_update`), validam `TRANSICOES`, gravam `LogAuditoria` e aplicam os efeitos (`services/atendimento.ao_mudar_status`). `status = X; save()` pula tudo isso e é proibido no código.
- **Efeitos do atendimento** (`services/atendimento.py`, chamados explicitamente — sem post_save nem EventBus): `ao_mudar_status` roda dentro da transação da FSM, em ordem — REALIZADO zera as faltas, debita a sessão do pacote (item congelado na venda; nunca de retorno; validade pela data da sessão), lança a comissão, libera o cashback de indicação e sugere o retorno obrigatório; CANCELADO estorna comissão e cashback; FALTOU conta a falta (3 seguidas suspendem o agendamento online); CANCELADO/REAGENDADO de um ativo avisa a lista de espera (savepoint, best-effort). Efeito que falha desfaz a transição inteira (status + livro-razão gravam juntos ou nada grava). `ao_criar` (chamado por `services.agenda.reservar` e pelo "adicionar" do Django admin) manda o push à profissional no `on_commit` e tira da lista de espera as inscrições da cliente para aquele procedimento (savepoint, best-effort). `status = X; save()` não aplica regra nenhuma. `cancelar(..., pela_cliente=True)` só nos fluxos da própria cliente (portal e link de confirmação, por `agenda.cancelar_pela_cliente`) — fica na trilha (`[pela cliente]`). No mesmo módulo, os casos de uso da equipe: `marcar_realizado` (painel e portal da profissional: termo de procedimento não aceito levanta `TermoPendente`; com `aceitar_sem_termo` marca e grava "Realizado sem termo aceito") e `registrar_valor_cobrado` (valor combinado, crédito de indicação abatido; em REALIZADO recalcula a comissão pendente e libera o cashback; status final e retorno recusam).

- **Aviso de cancelamento** (`services/agendamento.avisar_cancelamento`): e-mail à cliente quando a clínica cancela — painel, rejeição, Django admin e pedido PENDENTE que venceu sem aprovação. Cancelamento pela própria cliente não avisa.
- **Disponibilidade**: fonte única `services/disponibilidade.py::SlotService` (slots de 30 min; grade semanal `AgendaHorario` e exceções `AgendaExcecao` — folga/feriado/recesso, horário diferente, bloqueios pontuais/recorrentes do profissional ou globais —, buffers, `min_notice_horas`, `max_advance_dias`). Usada pelo wizard, pelo reagendamento, pelo agendamento interno e pela validação do servidor. Em volta dele, no mesmo módulo: `horarios_agrupados` (horários do dia com as profissionais livres e o preço na data — o reagendamento omite o preço) e `dias_com_expediente` (o calendário do mês do wizard em 2 consultas: fora feriado/recesso, folga, `max_advance` e dia todo bloqueado); `procedimentos_agendaveis` = ativos com ao menos 1 profissional ativa habilitada.
- **Agenda** (`services/agenda.py`): as 5 portas que gravam horário (wizard, agendamento pelo painel, reagendamento pelo portal, arrasto do calendário, retorno marcado pela recepção) passam por `validar_horario` (habilitação, horário futuro, feriado/recesso, SlotService; `detalhado=True` dá o motivo à equipe — conflito, bloqueio, folga, expediente — e o site vê a mensagem genérica), `reservar` (conflito com `select_for_update`, preço da data, INSERT em savepoint; corrida perdida para o EXCLUDE vira `HorarioIndisponivel`), `reagendar` (antigo REAGENDADO + novo com preço/promoção e ficha) e `mover`. Moram aqui também `agendar_pela_recepcao` (painel: cliente nova gravada na mesma transação, preço do dia ou o combinado — `preco_da_recepcao` —, AGENDADO, e-mail CONFIRMACAO à cliente e a trilha; outro cadastro com o mesmo telefone/e-mail/CPF no meio vira `cliente_duplicada`) e `cancelar_pela_cliente` (portal e link do lembrete, linha travada: com menos de `JANELA_MINIMA_ALTERACAO` de um horário AGENDADO/CONFIRMADO é **cancelamento tardio** — `Cliente.registrar_cancelamento_tardio` soma 1 falta sem virar FALTOU; recusas `nao_encontrado`, `passado`, `nao_cancelavel`). O que é só de uma porta fica nela (o pedido do wizard, a validação do formulário do painel em `forms/agenda`, e-mails do portal).
- **Pedido online** (caso de uso, `services/agendamento_online.solicitar(PedidoAgendamento, request=)`): o `forms/agendamento.AgendamentoPublicoForm` lê o POST do wizard, valida formato, idade e o gate do OTP (o celular tem de ser o verificado **nesta** sessão) e entrega um `PedidoAgendamento`; o service tem a regra inteira, testável sem HTTP — termo LGPD vigente → `validar_horario` → fichas (validade de 12 meses; consentimento de saúde se houver resposta) → termos do procedimento → e-mail de outro cadastro → trava do horário no cache → numa transação: cadastro da **titular** do celular, bloqueio online, teto de 3 horários ativos, opt-ins, `reservar`, e-mail (APROVACAO ou CONFIRMACAO), aceites com prova e fichas com a trilha do art. 11. Toda recusa é `RegraDeNegocio` com `codigo` (`sem_termo_lgpd`, `ficha_invalida`, `consent_saude_ausente`, `termo_pendente`, `email_de_outro_cadastro`, `horario_em_confirmacao`, `bloqueado_online`, `limite_online`, `horario_ocupado`...); a view (`views/publico/agendamento`) só faz honeypot, form, service e a página de sucesso.
- **Catálogo** (`services/catalogo`): o preço base muda abrindo uma vigência nova a partir de hoje (só quando o valor muda; o histórico fica) e tirar a habilitação de quem ainda tem atendimento futuro do procedimento é recusado (`habilitacao_em_uso`).
- **Recusas** (`excecoes.py`): service diz "não" levantando `RegraDeNegocio(mensagem, codigo=)` (ou `HorarioIndisponivel`/`TransicaoInvalida`/`TermoPendente`, que leva os termos pendentes); a view só mostra `exc.mensagem` (ou `json_recusa(exc)`). `Atendimento.TransicaoInvalida` é a mesma classe; a FSM recusa REALIZADO/FALTOU antes do início (menos `TOLERANCIA_DESFECHO`, 30 min) com `desfecho_antes_do_inicio`.
- **Agenda do painel** (`views/painel/agendamentos`, `aprovacao`, `calendario`, `visao_geral`): views finas sobre os casos de uso acima e o `AgendamentoService` (aprovar/rejeitar e `decidir_em_lote`, item a item, cada um na sua transação). Os marcadores de cada linha (alerta de saúde, termo e ficha pendentes, pacote

e crédito) saem em lote de `services/alertas.marcar_atendimentos` e a tela grava "Leu alertas de saúde" (`alertas.registrar_leitura_saude`, tabela `prontuario`, `{clientes, tela}`) quando mostra algum alerta; o botão "Agendar retorno (até dd/mm)" vem de `retorno.devidos_na_lista` (1 consulta). Exceções de agenda pelo `forms/agenda.AgendaExcecaoForm` (reenviar o mesmo dia/período atualiza a vigente — `AgendaExcecao.objects.vigente_para`) e `AgendaExcecao.clean` com uma regra por método.

- **Área da cliente** (`views/publico/portal` + `services/portal_cliente`): sessão do OTP (do portal ou aberta no fim do wizard sem novo SMS) presa ao telefone do cadastro, 1 h, com a hora da verificação; cada atendimento só pelo id dentro dela (anti-IDOR). Preferências e revogação de saúde em `services/consentimentos` (origem PORTAL); pedido de correção/exclusão = `registrar_pedido` (trilha com o prazo de 15 dias + e-mail ao encarregado > `email_admin` > `ADMIN_EMAIL` > e-mail da marca); "baixar meus dados" = o export sem saúde de `services/lgpd.exportar_dados`, só com verificação de até 10 min.
- **Respostas JSON** (`utils/respostas.py`, contrato único das APIs do site — inclusive as do wizard, `/ajax/dias-disponiveis/`, `/ajax/horarios-disponiveis/` e o OTP —, do portal da cliente (login, horários do reagendamento, cancelar), do painel (status, calendário, anotação clínica), da infra e do 403 `2fa_required` do middleware): sucesso `{"ok": true, ...}`, erro `{"ok": false, "codigo": "<estável>", "mensagem": "<texto>"}` (`json_ok`, `json_erro`, `json_recusa`) — o front decide pelo `codigo`, nunca pelo texto. Fora do envelope, de propósito: webhooks (formato do provedor), healthchecks, eventos do FullCalendar, feed ICS e manifests. Cada rota está em `api/openapi.yaml` (Swagger UI em `/painel/api/`); `tests/test_openapi.py` reprova rota JSON/ICS nova sem entrada, caminho que não resolve para o `x-url-name`, `$ref` quebrado e jobs do cron diferentes do `JOB_MAP`.
- **Telefone** (`utils/telefone.py`, fonte única): forma canônica gravada em `Cliente.telefone` e usada no OTP, nas buscas e no webhook do WhatsApp (só dígitos, DDD + número, sem DDI/zero de tronco), E.164 só para telefone BR válido (destino de SMS/WhatsApp), a exibição (`formatar`, filtro builtin `{{ tel|telefone }}` → "(17) 99999-0001") e a máscara de log.
- **Preço**: `utils/precos.py` — preço vigente por data (profissional > base) e promoção vigente na data; `preco_com_promocao()` é o valor gravado no atendimento. A vitrine (cards do wizard, widget embutido, `/especialidades/`, página do serviço, `/promocoes/`) sai de uma função só, `services/precos.precos_de_vitrine` (2 consultas para o catálogo inteiro): preço base do dia (sem base, o menor entre os profissionais) com a promoção que o agendamento aplicaria — e só se ela reduz o valor (a vitrine nunca anuncia o que o atendimento não grava).
- **Branding**: `utils/branding.get_branding()` — precedência **tela Identidade visual ("Branding"; tabela configuracao, forms/cadastros.MarcaForm) > env > padrão**; contatos vazios escondem o item no site; campo igual ao valor em uso não conta como alteração e apagar o ajuste volta ao padrão. As regras operacionais (`email_admin`, `MAX_FALTAS_BLOQUEIO`, `anos_experiencia`) são outra tela, com o registro tipado de `models/configuracao.py`.
- **Números do site** (`services/prova_social.numeros_do_site`, home, "Sobre" e páginas de serviço): só o que o banco ou a dona sustentam — nota média do NPS em 5 estrelas (÷ 2) só com 10+ respostas, clientes distintas com atendimento REALIZADO só a partir de 50 e arredondadas para baixo ("120+"), tratamentos = procedimentos agendáveis online, anos de experiência = regra `anos_experiencia` (em branco, some). Uma conta a cada 10 min (cache), zerada quando `Configuracao` muda (`signals.py`); o template esconde o número que vier `None`. Regras em `REGRAS-DE-NEGOCIO.md` §18.
- **Comunicação** (`services/comunicacao.py`): toda mensagem ligada a um atendimento é uma `Notificacao` (tipo × canal × status) — o histórico do que a cliente e a equipe receberam. Os e-mails de confirmação, cancelamento, lembrete D-1 (quem não autorizou o WhatsApp) e o pedido de aprovação à profissional nascem `Notificacao` EMAIL PENDENTE **na transação de quem pediu** (outbox) e saem no `on_commit`; se falhar, o job `emails` reenvia com espera crescente (5/10/20/40 min) até 5 tentativas ou 48 h → FALHOU (lembrete de horário que já passou ou foi cancelado vira FALHOU sem envio). A fila não guarda cópia de

PII: destino e texto saem do atendimento na hora do envio. Marketing (`services/marketing`) e avisos em lote saem direto (disparo da promoção no request, em lotes; aniversário e pacote pelos jobs). Sem Celery (nada roda em segundo plano).

- **Front**: `static/src/js/app.js` registra componentes Alpine CSP-safe (`temaToggle`, `modal`, `navMenu`, `cookieConsent`, `adminShell`, `alertaDismiss`, `anotacaoModal`, `pacoteCriar`) + `data-confirm` global. As telas da equipe (painel e portal da profissional) carregam também `painel.js` — **vanilla, por delegação no document, sem Alpine**: abas no padrão WAI-ARIA (setas, Home/End, aba lembrada por página, aba com erro do servidor abre sozinha), menu "⋯" (`<details>` nativo: fecha os outros, no clique fora e no Esc devolvendo o foco), modais (`data-abrir-dialogo`/`data-fechar-dialogo`), Ctrl+K na busca do topo e foco/nome na tabela que rola na horizontal. A agenda tem módulos próprios: `painel-status.js` (seletor de status de `partials/_select_status.html` — só AGENDADO/CONFIRMADO, Realizado/Faltou só a partir do início — e a barra de aprovar/recusar em lote; `termo_pendente` pede a confirmação e reenvia com `sem_termo`) e `painel-calendario.js` (FullCalendar do npm: recusa do arrasto desfaz o movimento e mostra a `mensagem`; o ARIA de grade que o FullCalendar não fecha vira `presentation`), ambos com `lib/requisicao.js` (POST JSON com o CSRF da `<meta>`; resposta não-JSON vira `falha_http`); os gráficos da visão geral são `painel-visao-geral.js` (Chart.js do npm, cores dos tokens). Libs de terceiros entram pelo `package.json` (as do painel com versão exata) e saem no bundle — nenhum `<script src>` externo (`test_csp` reprova). **CSS em duas camadas sobre os tokens**: `componentes.css` (design system das telas da equipe: `.btn`, `.campo`, `.cartao`, `.selo`, `.nota`, `.kpi`, `.tabela`, `.aba`, `.menu-acoes`, `.dialogo-*`, `.vazio`... em `@layer components`, então o utilitário do template sempre vence sem `!important`; texto base 15px, rótulo/meta 13px, nada abaixo de 12px) e `site.css` (o CSS das páginas do site público que antes vinha em `<style>` por template — wizard, "Meus agendamentos", reagendar, aviso de cookies, arte da marca, números —, só `var(--cor-*)`, sem hex). Componentes cotton do site: `<c-cta-agendar>` (o "Agendar" do site: wizard ou, sem SMS, o WhatsApp — tag `{% agendar_link %}`, builtin, usada em todo link p/ o wizard), `<c-servico.passo>`, `<c-servico.cartao>`/`<c-servico.item>` (páginas de serviço) e o chrome (`cabecalho` com o menu de `{% menu_publico %}` — uma lista p/ desktop e celular, `aria-current` —, `rodape`, `mensagens`, `cookie_consent` — faixa fina no pé, informativa: o site só usa cookies essenciais; "Entendi" grava `cookie_consent=essenciais` no próprio navegador e nada vai ao servidor). Componentes do design system da equipe: `<c-botao>` (`variante` primario/secundario/perigo/perigo-contorno/sucesso/fantasma/link, `tamanho` sm/lg, `href` vira `<a>`, `icone`, `so_icone`, `loading`), `<c-campo>`, `<c-card>`, `<c-selo>`, `<c-nota>`, `<c-kpi>` (variação com seta e texto), `<c-vazio>`, `<c-icone>`, `<c-menu-acoes>` ("⋯"), `<c-dialogo>`, `<c-tabela>` (`responsiva`: tabela no desktop, cartões empilhados < 768px com `data-rotulo`) e `<c-abas>` + `<c-abas.aba>`/`<c-abas.painel>` (sem JS, links de âncora para seções visíveis). Fotos do site pela tag `{% foto 'nome' alt=... sizes=... prioridade=... %}` (`site_tags`): `<picture>` com WebP no `srcset` e JPEG de reserva das variantes de `scripts/imagens.py`, `width`/`height` reais (sem salto de layout), `loading="lazy"` — ou `fetchpriority="high"` na foto do topo; moldura sem foto real usa a arte da marca (`partials/_arte_marca.html`), nunca foto inventada. Casca do painel `painel/base_v2.html` (menu lateral no cromo escuro quente da marca — café/ouro, tokens `--cor-cromo-*` —, barra do topo com a busca global, cabeçalho de página igual em toda tela) e do portal `profissional/base.html` (mesmo design system, abas de navegação na barra escura). Telas-chave: a agenda é tabela no desktop e cartões no celular, com as ações secundárias no "⋯" e valor/desfazer realização em modal; a ficha da cliente em abas (Dados · Agendamentos · Pacotes e crédito · Saúde · Consentimentos); a visão geral abre com os atalhos "sem desfecho" e "retornos a marcar", depois Hoje/Amanhã e pendentes de aprovação, depois os KPIs comparados com o período anterior. Painel, portal e páginas do site sem `style="..."` — sobram só o widget `agendamento/embed.html` (fora do bundle), as páginas de erro autossuficientes (400/403_csrf/500), a página offline do `sw.js` e os e-mails; no site, `test_a11y_front.SitePublicoSemEstiloInlineTests` barra atributo, `<style>` por página e cor crua em classe. Títulos, botões e rótulos em caixa de frase. Menu do painel numa lista só,

templatetags/painel_tags.MENU_PAINEL (Operação: visão geral, agenda, calendário, retornos, clientes, prontuário, notificações; Cadastros; Relatórios: financeiro, comissões, NPS; Sistema). Tema: sem escolha segue o sistema (prefers-color-scheme); o botão grava o cookie tema (data-theme="claro|escuro") sem FOUC. Tokens em tokens.css (contraste AA conferido nos 2 temas): texto, --cor-borda-campo (3:1 nos inputs), --cor-foco (anel :focus-visible global), erro/sucesso/aviso/info/destaque com -fundo, o status do atendimento (--st-<status>-fundo/-texto, classes .status-<STATUS>) e os do design system da equipe (cromo, --cor-fundo-realce, --cor-borda-forte, --cor-marca-suave, --cor-perigo-forte, sombras); o verde do WhatsApp é um token do site.css (--site-whatsapp = o verde de ação, 5,3:1). Nada de cor crua do Tailwind (bg-red-50…) fora de preenchimento sólido. prefers-reduced-motion zera animações. Modal = <dialog class="dialogo"> nativo (showModal: foco preso, Esc fecha, fundo inerte): <c-dialogo> aberto pelo painel.js ou, no Alpine, componente modal (ou comDialogo({...}) em app.js) com x-ref="dialogo". Formulário: todo campo tem nome acessível — <label for> (o app.js associa o rótulo solto ao campo seguinte e a dica no meio vira aria-describedby) ou aria-label nos filtros. Toast do painel: erro = role="alert", o resto role="status"; nenhum some sozinho.

- **PWA**: /manifest.json + /sw.js (service worker v10: cacheia só páginas públicas — lista fixa, inclusive /duvidas/, e /servicos/detalhe/ — e estáticos; nunca HTML privado; sem rede, página offline com a marca), /painel/manifest.json (app "Painel": abre a visão geral, atalhos Agenda/Novo agendamento/Clientes/Calendário) e /profissional/manifest.json (app "Minha agenda" da profissional) — cada um no próprio escopo, sem trava de orientação, lang pt-BR e splash no cromo da marca; no iPhone, apple-touch-icon e barra de status default (o conteúdo não fica sob o relógio). Registrado no site público, no painel e no portal. Ícones em static/assets/pwa/ (192 e 512 "any" e 512 "maskable" com margem de segurança — só o monograma "JA"; o texto da logo some em ícone pequeno). Subir VERSION no sw.js a cada mudança dele.

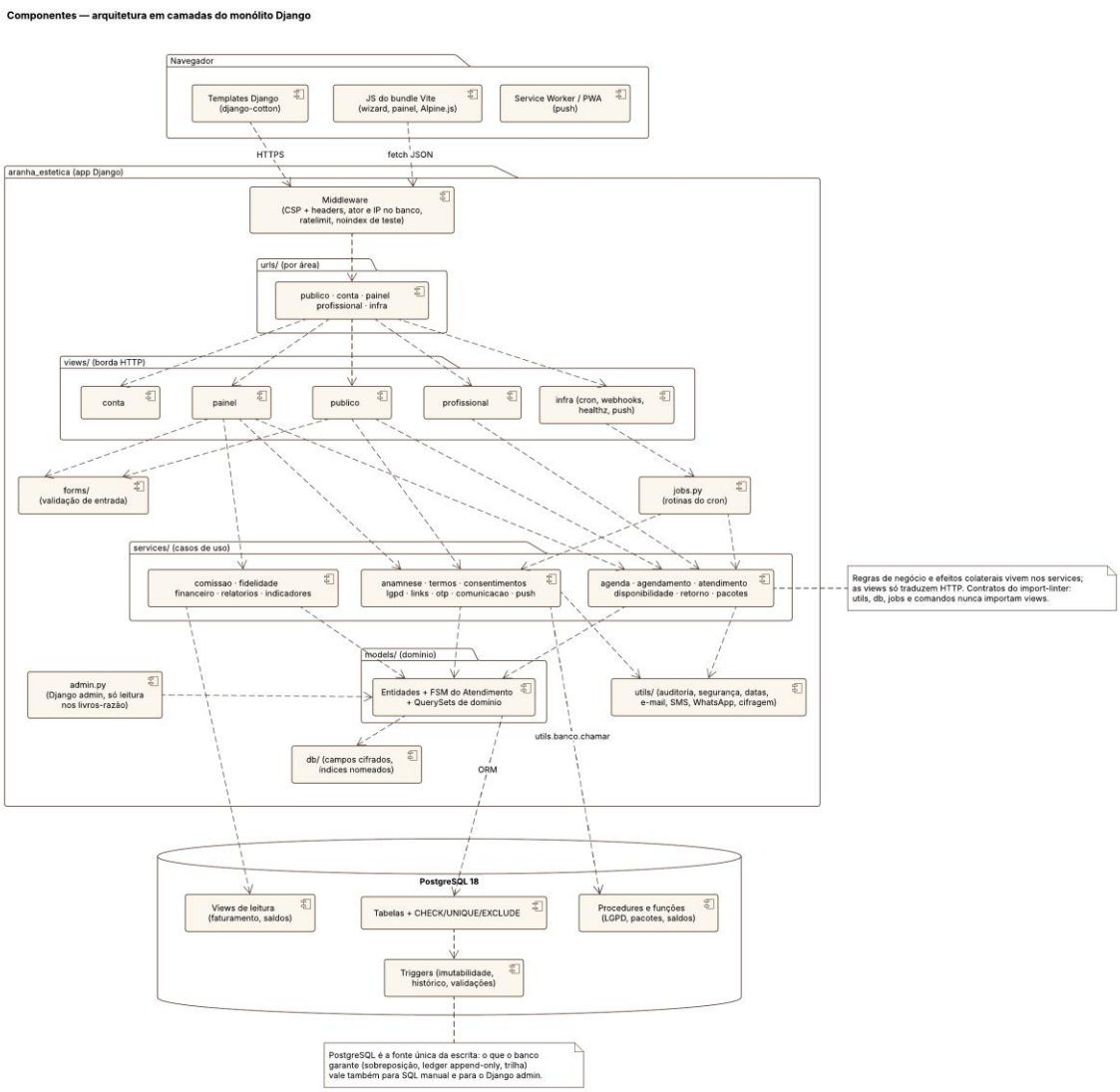


Figura 1 — Componentes — arquitetura em camadas do monólito.

Componentes — integrações externas

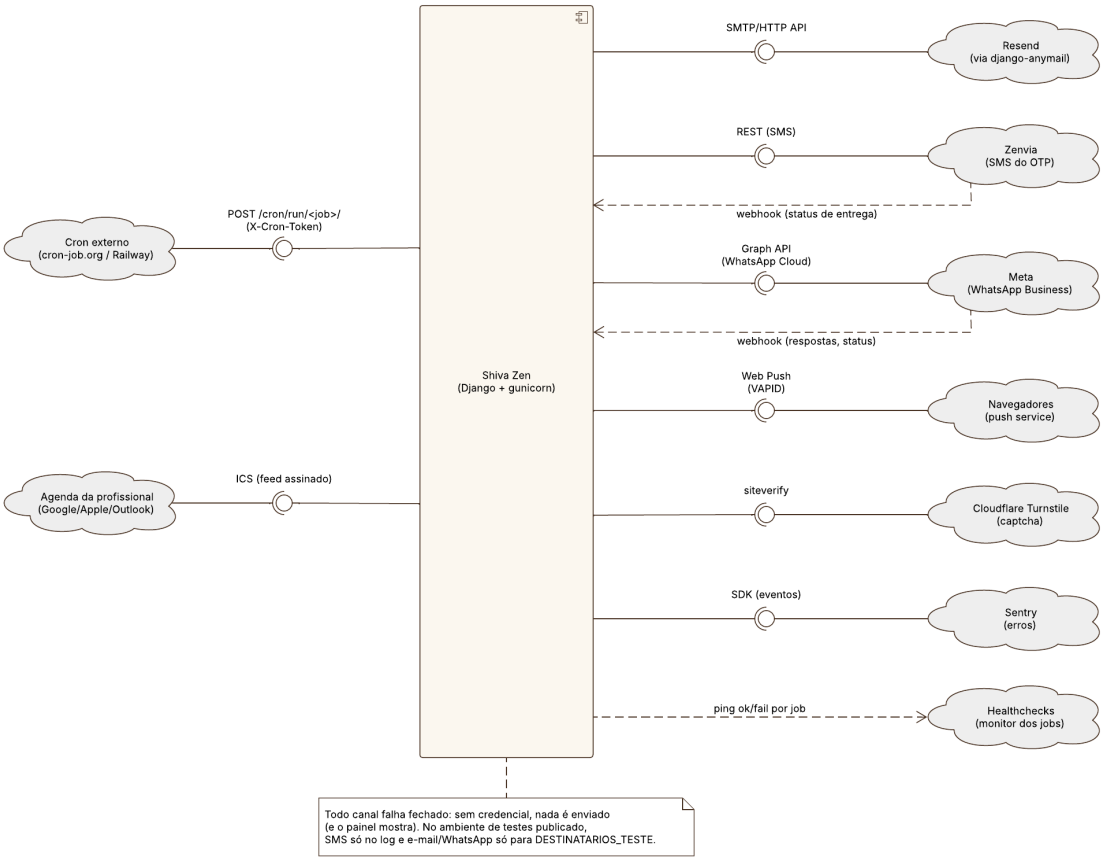


Figura 2 — Componentes — integrações externas.

4. Decisões de arquitetura

Decisões fixadas ao longo do projeto (ADR resumido), com o motivo de cada uma.

ID	Decisão	Motivo
D1	Backend Django server-render mantido	Sólido, SEO grátis, 1 mantenedor
D2	Não vira SPA; **Alpine CSP, sem HTMX**	Nenhuma tela precisava de parcial por HTMX; CSP estrita
D3	Tailwind v4 (não Bootstrap)	1 config de tokens para site + painel
D4	Direção visual "Clínico-premium" (dourado #C9A84C, serif + sans)	Confiança; afunila para o agendamento. Porquê das cores, fontes e tom: `DESIGN-E-MARKETING.md`
D5	Painel como PWA app-shell	Uso como app instalado
D6	Fase 7 do banco em PR dedicada	Mexe no cálculo de slots
D7	**Todo agendamento público exige OTP do celular**; a identidade é o telefone, e-mail nunca é identidade	Anti-sequestro de cadastro; sem enumeração
D8	Agendamento público nasce **PENDENTE** (clínica aprova), exceto cliente de confiança (já atendida, sem falta, < 2 horários futuros) que nasce AGENDADO; interno nasce AGENDADO. Pelo site, no máximo 3 horários futuros ativos por cliente (`MAX_ATIVOS_ONLINE`; a recepção marca além disso). Toda decisão sobre o pedido passa pelo `AgendamentoService` (botão ou seletor)	Tira o gargalo da aprovação de quem já é cliente sem abrir a agenda a desconhecidos (o PENDENTE trava o horário: sem teto, um celular verificado reservaria a semana); a cliente sempre recebe a resposta
D9	**2FA obrigatório para ADMIN e PROFISSIONAL** em prod (válvulas `ADMIN_2FA_OBRIGATORIO=false` / `PROFISSIONAL_2FA_OBRIGATORIO=false`, avisos W008/W012); códigos de backup gerados no painel	Painel e portal leem dado de saúde (LGPD art. 11/46)
D10	Rotas do django-two-factor (`/account/*`) e ReDoc não publicadas	2ª tela de login burlava o 2FA; ReDoc exige `unsafe-inline`
D11	~~RECEPCAO sem login até existirem telas próprias~~ — substituída pela D27 (o papel saiu)	Papel sem superfície própria = acesso indevido ao painel
D12	Prova de aceite imutável (SHA-256 do texto + trigger) e termo aceito imutável	Evidência legal (LGPD art. 8)
D13	Histórico append-only do prontuário (`prontuario_versao`)	CFM 1.638/2002 — edição não apaga o anterior
D14	O sistema nunca marca FALTOU sozinho: atendimento sem desfecho vira um push diário à profissional (job `sem_desfecho`, 08:00), não falta	Falta é terminal e bloqueia online; só a equipe decide — e a pendência precisa chegar a quem decide, não só ao log
D15	Aniversário = felicitação sem desconto; promoção sem cupom, anunciada pela **data do atendimento** (o período em que o preço vale), disparo único em lotes com o cursor na própria promoção (reenvio só confirmado)	Nada é prometido sem mecanismo que aplique; clique duplo ou queda no meio nunca repetem e-mail
D16	Deploy por Dockerfile + `migrate_atomico` no pre-deploy; **rollback = restaurar dump**	Upgrade 0026→0046 tudo-ou-nada; reverses não recuperam dados
D17	**Sem Celery** (saiu em 2026-10-27): jobs são funções com horário na `jobs.JOB_AGENDA` (fonte única), rodadas por um serviço Cron do Railway (`rodar_jobs_devidos` a cada 15 min), pelo cron HTTP (`X-Cron-Token`) ou `rodar_job`; job de mensagem sem marca por destinatário roda 1×/dia (`execucao_job`, 409 na 2ª); e-mail transacional é fila na própria `notificacao` (0061, reenvio pelo job `emails`)	Custo; um serviço de jobs no lugar de um por horário; nada roda "em segundo plano" sem rastro nem manda duas vezes
D18	`/anamnese/<token>/` e `/pesquisa/<token>/` **com produtor**: "Link da ficha" na agenda do painel (cliente agendada pela recepção) e convite da pesquisa no fim do NPS	Ficha obrigatória chega a quem não passou pelo wizard; a pesquisa reaproveita o link do NPS (sem segunda mensagem)
D19	Site só pt-BR (sem LocaleMiddleware); sessão da equipe 8h deslizantes	Sem traduções; 30 min fixos deslogavam no meio do atendimento
D20	Faturamento = avulsos + venda de pacote; sessão de pacote não soma; comissão da sessão de pacote = `valor_pago / total de sessões` (congelado na venda, `compra_pacote_item.valor_unitario`); ticket médio e LTV da	Evita contar a receita duas vezes e inflar comissão

	ficha pela mesma regra (`services/financeiro`)	
D21	Dado de saúde **cifrado na aplicação** (Fernet, `CHAVES_CIFRAGEM`, subchave por finalidade; 0060), sem busca no banco por conteúdo (o token OAuth do Google, também cifrado, saiu com a D26)	Dump/backup vazado não expõe a ficha (LGPD art. 46/48 §3); pgcrypto deixaria a chave no SQL de toda consulta
D22	Trilhas append-only no banco: `log_auditoria` imutável e expurgo só com 180+ dias (0059)	Mesmo usuário do banco p/ app e migrations: SQL injetado não apaga a prova (art. 37)
D23	**Postgres é a fonte única da escrita** de esquecimento, retenção LGPD, limpeza e expiração de pacotes (procedures 0058, sem fallback Python); funções de leitura seguem como espelho testado do Python	Decisão do dono (2026-10-27): uma regra, um lugar; custa exigir Postgres (Docker) para rodar esses testes e rotinas localmente
D24	Ambiente de testes publicado = **perfil staging** (`prod.py` + travas: SMS só em log com a prévia, e-mail/WhatsApp só p/ `DESTINATARIOS_TESTE`, noindex, faixa, `seed --demo`); `DJANGO_ENV=dev` no Railway derruba o boot e produção sem Postgres é Error (`aranha.E002`)	Testar o fluxo inteiro (wizard + OTP) na internet sem mandar mensagem a cliente real nem expor DEBUG
D25	Financeiro por **competência mensal** (`services/financeiro.competencia`): pacote = vendas pela data da venda − reembolsos pela data do cancelamento (`compra_pacote.cancelado_em`); comissão pela data do atendimento; rateio da venda **ponderado pelo preço de tabela**; CRÉDITO de comissão congela base e percentual; regra usada é imutável (trigger)	Mês fechado não muda; o valor de cada sessão reflete o procedimento; comissão explicável depois
D26	**Google Agenda removido** (0071): a agenda da profissional sai só pelo feed ICS assinado (`link_acesso` FEED_ICS, nome público da cliente); sem OAuth, sem importação de eventos como bloqueio	O feed cobre Google, Apple e Outlook sem guardar token de terceiro nem manter sincronização em dois sentidos; um operador e um segredo a menos
D27	Papéis da equipe só **ADMIN e PROFISSIONAL** (0072; substitui a D11): a dona faz a recepção pelo painel	YAGNI: o papel RECEPCAO não tinha tela nem login e só existia como risco de acesso indevido
D28	**Todo atendimento é presencial** (0073): sem modalidade nem link de videoconferência; a ficha vale por 3 escopos (global, categoria, procedimento)	Nenhum serviço online no catálogo; o escopo MODALIDADE só repetia o GLOBAL
D29	A cliente tem **um acesso só**: o portal "Meus agendamentos" por OTP (reagendar/cancelar por id + sessão). O link legado de reagendar (GERENCIAR_AGENDAMENTO) e o alvo atendimento do `link_acesso` saíram (0072)	Os tokens legados vinham do banco da 0026, descartado no go-live; dois caminhos eram duas superfícies de ataque para a mesma ação
D30	**Minimização do cadastro** (0074): RG, profissão e foto da cliente não são coletados; CPF, CEP e endereço ficam pela finalidade (nota fiscal, contrato de pacote)	LGPD art. 6º III (necessidade): dado sem finalidade é risco sem benefício e entra no esquecimento à toa
D31	**Cookies só essenciais**: o aviso é informativo e "Entendi" grava só no navegador; nada é registrado no servidor (saiu `/lgpd/aceitar-cookies/`)	Sem cookie de análise ou de terceiro não há consentimento a provar; guardar a resposta era dado pessoal sem finalidade
D32	**Runner único** `manage.py test` (pytest, pytest-django e conftest saíram; teste em função solta reprova)	Dois runners coletavam conjuntos diferentes: teste só do pytest ficava fora do CI
D33	**Áreas de views independentes** (publico, painel, profissional, conta, infra não se importam — import-linter); o que mais de uma área renderiza igual mora em `views/comum.py`, a regra em `services/`	Mudar uma tela não quebra outra área; o acoplamento entre áreas fica explícito num lugar só
D34	**Configuração só nos settings**: a env é lida e validada em `clinica/settings` e o código lê `settings.X` na chamada (ruff `TID251`); o `.env.example` é conferido contra o código; segredo fraco num perfil publicado é erro (E003)	Valor inválido derruba o boot com o nome da variável em vez de virar 500 no meio de um fluxo; `override_settings` vale nos testes; o exemplo não sobe segredo para o Railway
D35	**utils/telefone é a fonte única do telefone**: forma canônica sem DDI (a do cadastro, do OTP e das buscas), E.164 só para telefone BR válido, máscara de log	O telefone é a identidade da cliente: cópias da normalização em OTP, lista de espera, formulário e webhook divergiam, e número estrangeiro virava destino de SMS
D36	**Contrato JSON único** (`utils/respostas`): `{"ok": true, ...}` ou `{"ok": false, "codigo", "mensagem"}`; o front decide pelo `codigo`	Cada view respondia num formato; o texto da mensagem não é contrato

D37	**Logs redigidos e imagem endurecida**: filtro `RedigirLog` (token de link, DETAIL do Postgres, e-mail, CPF, telefone) no log da aplicação; imagem sem `curl`, usuário sem shell, código do root só leitura, testes e scripts fora	O `django.request` levava `/confirmar/<token>/` para o log do Railway em todo 4xx/5xx; um RCE não reescreve código nem baixa ferramenta
D38	**Pedido online como caso de uso**: `services/agendamento_online.solicitar(PedidoAgendamento)` tem a regra inteira (termo LGPD, horário, fichas válidas por 12 meses, termos, cadastro da titular do celular, teto, reserva, e-mails e provas); `forms/agendamento.AgendamentoPublicoForm` lê o POST e confere o OTP; a view só liga os dois. Recusa = `RegraDeNegocio` com código (`sem_termo_lgpd`, `ficha_invalida`, `consent_saude_ausente`, `termo_pendente`, `email_de_outro_cadastro`, `horario_em_confirmacao`, `bloqueado_online`, `limite_online`, `horario_ocupado`)	A view tinha complexidade ciclomática 71 (hoje 9) e a regra só era testável por HTTP; outra porta (painel, API) reusa o service
D39	**Regras operacionais em registro tipado** (`models/configuracao.CHAVES`: `email_admin`, `MAX_FALTAS_BLOQUEIO` 1–10 e, com a D51, `anos_experiencia` 1–60 opcional), lidas por `Configuracao.ler`; a tela grava só essas chaves (sem chave livre) e a marca tem a tela Identidade visual	A chave-valor livre listava estado interno, aceitava chave inventada e um valor inválido caía no padrão em silêncio
D40	**Desligamento revoga o feed ICS**: desativar a profissional, o login dela ou trocar o vínculo do login revoga o link na hora (auditado); reativar exige link novo	O calendário de quem saiu continuaria recebendo nome de cliente (o feed é um operador fora da clínica)
D41	**Política de Privacidade como fonte única** (`conteudo/lgpd.py`): o mesmo texto é o termo LGPD aceito no wizard (0045/0078 e `seed`), a página `/politica-de-privacidade/` (renderiza a versão LGPD **vigente**) e a declaração do "baixar meus dados" (finalidades, bases, operadores, prazos). Os prazos LGPD ficam fixos em `settings.RETENCAO_*`, sem env, e o check `aranha.W016` compara com o texto da versão vigente. Revisão jurídica da dona antes do go-live	A página e o termo eram dois textos que divergiam, e o prazo que a procedure aplicava podia não ser o que a cliente leu; a prova do aceite (SHA-256) agora cobre exatamente o que a página mostra
D42	**Retenção por janela e pseudonimização completa** (0077): o registro de saúde é expurgado 20 anos depois do último evento de saúde (etapa `saude` do `sp_purgar_dados_expirados`), o pacote retém a cliente 5 anos depois de expirar/cancelar, e a pseudonimização tira também o código de indicação, quem indicou, IP/user-agent do aceite LGPD e a inscrição anônima da lista de espera. Os triggers só aceitam o expurgo com a flag local **e** 20 anos de idade	Quem tinha ficha ou pacote ficava identificada para sempre (contra os arts. 15/16 e a política); o mínimo legal do registro de saúde fica garantido pelo banco mesmo com um prazo errado no settings. É pseudonimização (art. 13 §4), não anonimização: a saúde e os contratos seguem ligados ao id
D43	**Consentimento de saúde revogável** (art. 11 I, art. 8 §5): a concessão é o aceite SAÚDE e a revogação é `cliente.saude_revogada_em` (portal ou Django admin com motivo), fora da tabela `consentimento`; os alertas mostram a revogação e um aceite SAÚDE novo reativa	Com base só no consentimento, revogar tem de ser possível e tão fácil quanto dar; a ficha já feita fica pelo prazo de guarda (exercício de direitos)
D44	**Direitos pela área da cliente, sem a ficha de saúde no site**: "Meus agendamentos" concentra preferências (origem PORTAL), revogação de saúde, pedido de correção/exclusão (trilha com prazo de 15 dias + e-mail ao encarregado) e "baixar meus dados"; o export self-service (portal e `/lgpd/meus-dados/`) não leva saúde e, no portal, exige código de até 10 min; a sessão do portal abre no fim do wizard sem novo SMS	O celular é um fator só (SIM-swap): o dado sensível não sai pelo site. Um lugar para tudo o que a titular pede, sem e-mail solto nem segundo login
D45	**Cancelamento tardio conta falta**: a cliente sempre pode cancelar online antes do início (a vaga volta para a lista), mas com menos de 24 h de um horário aprovado soma 1 falta para o bloqueio online, sem virar FALTOU (`agenda.cancelar_pela_cliente`, portal e link); pedido PENDENTE não conta	Cancelar em cima da hora tem o mesmo efeito de faltar para a agenda; o aviso antes de confirmar e a política (art. 20) deixam a regra explícita
D46	**Lista de espera sem cadastro e formulários públicos com anti-abuso**: a inscrição é anônima (nome + celular) e só se liga à cliente — e só aceita e-mail — com o celular verificado por OTP; o aviso não tira da fila e agendar encerra a inscrição. Lista e contato com captcha, honeypot e limite por IP (contato: teto global diário); o contato não tem caixa de marketing e o mapa não carrega nada	Um formulário sem login não pode criar nem alterar cadastro, nem prender um e-mail ao telefone de outra pessoa; a cota de e-mail é compartilhada com o reset de senha; opt-in de marketing só onde há prova

	do Google	
D47	**Agenda do painel por casos de uso**: `agenda.agendar_pela_recepcao` (cliente nova na mesma transação, preço do dia ou combinado, AGENDADO, e-mail de confirmação), `atendimento.marcar_realizado` (a regra do termo igual no painel e no portal, `TermoPendente`), `atendimento.registrar_valor_cobrado` e `AgendamentoService.decidir_em_lote` (item a item, cada um na sua transação); a entrada é Django Form (`forms/agenda`) e a view só traduz HTTP	A view do novo agendamento tinha 191 linhas (hoje 43; complexidade do mccabe 27 → 7) e o lote numa transação só desfazia tudo por um item decidido em outra tela
D48	**Seletor de status só com o que há para decidir**: sem PENDENTE (decide nos botões, pelo `AgendamentoService`) nem status final; Realizado/Faltou só a partir do início (a FSM recusa com `desfecho_antes_do_inicio`, 409). O JS da agenda e do calendário são entradas do Vite (`painel-status`, `painel-calendario`, `lib/requisicao.js`) no contrato `utils/respostas`; as ações da agenda (status, valor, desfazer realização, aprovar/recusar/lote, mover, exceções e bloqueios) usam `@escrita_painel` (POST + 30/min por usuário)	Um clique na agenda da semana seguinte gerava comissão, debitava pacote e contava falta; o mesmo JS existia em duas cópias inline
D49	**Leitura de alerta de saúde auditada**: toda tela que mostra alerta (agenda e visão geral do painel, ficha da cliente, lista de prontuários, agenda do portal da profissional) grava "Leu alertas de saúde" (tabela `prontuario`, `{clientes, tela}`, sem nome)	O RIPD promete que cada leitura de dado de saúde vai para a trilha (art. 37); o alerta na lista também é leitura
D50	**Design system das telas da equipe** (painel e portal da profissional): classes de componente em `static/src/css/componentes.css` na camada `@layer components` (o utilitário do template sempre vence, sem `!important`), emitidas por componentes cotton (`<c-botao>`, `<c-campo>`, `<c-card>`, `<c-selo>`, `<c-nota>`, `<c-kpi>`, `<c-vazio>`, `<c-icone>`, `<c-menu-acoes>`, `<c-dialogo>`, `<c-tabela>`, `<c-abas>`); abas WAI-ARIA, menu "⋯", modais, Ctrl+K e foco da tabela que rola num `painel.js` **vanilla** por delegação, não em Alpine; lib de terceiro só pelo npm no bundle do Vite (FullCalendar 6.1.21, Chart.js 4.5.1 — nenhum CDN na CSP); o ARIA de grade do FullCalendar achatado para `presentation`; caixa de frase, texto base 15px, rótulo 13px e nada abaixo de 12px, sem `style=""`; toda rota pública, da equipe e da profissional varrida no CI (1366px claro e 390px escuro) com axe-core serious/critical	As telas repetiam classe utilitária longa e ~160 `style=""`, com cor fixa de tema claro que o escuro não alcançava, e cada uma resolvia modal e menu do seu jeito: o componente é um ponto de mudança só, com o padrão acessível embutido. Sem JS as abas são âncoras para seções visíveis e o "⋯" é `<details>` nativo; com Alpine cada instância pediria `x-data` e nada funcionaria antes do JS. O jsDelivr serviria qualquer pacote npm com o nonce da página. O FullCalendar 6 desenha um `role="grid"` de layout sem linhas (axe `aria-required-children`/`aria-required-parent`): como tabela de layout, a informação segue no texto. A varredura pega erro de JS/CSP, rolagem lateral no celular e regressão de acessibilidade que só apareceriam olhando tela por tela
D51	**Prova social só com dado real** (`services/prova_social`): nota = média do NPS ÷ 2 (escala de 5), só com **10+ respostas**, e as estrelas desenhadas em meias, sem arredondar 4,7 para 5; clientes atendidas = clientes distintas com atendimento REALIZADO, só a partir de **50** e arredondadas **para baixo** ("127" vira "120+", "1.234" vira "1.200+"); tratamentos = procedimentos agendáveis online (no total e por categoria); anos de experiência = regra operacional `anos_experiencia` (1–60, opcional; em branco não aparece). O que não atinge o mínimo some do template. Uma conta a cada 10 min (cache), zerada quando `Configuracao` muda. A "decisão do dono: manter" o selo fixo "5.0/5 · 32 avaliações" (e os "5+ / 500+ / 20+") foi revertida a pedido do dono ao aplicar a revisão do front	Número sem lastro no site de uma clínica de estética é promessa que a cliente não tem como conferir (e o selo fixo nunca mudava); com 3 notas 10 a média "5,0" diz mais sobre o acaso que sobre a clínica, e o "+" nunca promete mais do que houve. O cache evita 4 agregações a cada visita da home

5. Banco de dados

5.1 Resumo

- **37 tabelas de domínio** (+ tabelas do Django, django-otp, django-axes). PK `BigAutoField`.
- Migrations `0001`–`0079`. Ainda não há produção: o primeiro deploy cria o banco do zero (§14). O Postgres do ambiente de desenvolvimento no Railway (na `0026`) está em stand-by. Histórico em `specs/remodelagem-banco-v2.md` e, da `0048` em diante, o padrão SQL em `specs/padrao-sql.md`.
- **Lógica no banco** (0056–0059, revista na 0065–0066, 0072, 0074, 0077 e 0079; só Postgres — §5.4): 49 triggers (`updated_at`, imutabilidade, validação, trilha `historico_alteracao`), 25 funções `fn_*`, 9 views `vw_*` de relatório e 4 procedures `sp_*` que são a **fonte única** da escrita de retenção/LGPD. Exige as extensões `btree_gist`, `pg_trgm` e `unaccent` (o PG 18 do Railway tem).
- **Padrão SQL em todas as tabelas** (0054/0055): nomes ASCII snake_case PT-BR no singular; colunas `eh_`/`tem_` (booleano), `data_`/`hora_`/`_em`, `valor_`, `qtd_`, `url_`, `_hash` (`char(64)`), `_ref` (id externo), FK `<tabela>[_papel]_id`; **metadados de linha em inglês e iguais em toda tabela** (0079): `created_at` (todas), `updated_at` (toda tabela com UPDATE de negócio), `deleted_at` (soft delete), autoria `created_by_id` + snapshot `created_by_name` — o atributo Python segue em português (`criado_em`, `atualizado_em`...) via `db_column`; objetos `pk_`/`fk_` (com `ON DELETE` real, `DEFERRABLE INITIALLY DEFERRED`)/`uq_`/`ck_`/`ix_`/ `ex_`/`tg_`/`fn_`; sem índice `_like`; `COMMENT` em toda tabela; todo default também no banco (`db_default`); textos `NOT NULL DEFAULT ''` (exceto e-mail/CPF/telefone sob UNIQUE parcial e dado pessoal opcional: ausente = `NULL`, nunca `''`). Conformidade testada no Postgres por `tests/test_nomes_padrao.py`.
- **Nome só no banco**: atributos e classes Python não mudaram — o model aponta a coluna/tabela por `db_column`/`db_table` (ex.: `Cliente.ativo` → `eh_ativo`, `Procedimento.buffer_minutos` → `intervalo_minutos`, `ItemPacote` → `pacote_item`, `FormularioAnamnese` → `formulario`). SQL cru usa o nome do banco; mapa completo no §3.2 da spec.
- Nenhum token de link público fica em texto puro: `link_acesso` guarda só o `sha256` (0049).

5.2 Tabelas

Domínio	Tabela (model)	Para que serve
Acesso	`usuario` (Usuario)	Equipe. Login por e-mail; `papel` ADMIN/PROFISSIONAL (CHECK; RECEPCAO saiu na 0072); `is_staff` = ADMIN; 1:1 opcional com Profissional; `eh_ativo` (desativar, nunca excluir)
	`assinatura_push` (AssinaturaPush)	Inscrições Web Push (VAPID) por usuário (`usuario_id`, `chave_p256dh`, `chave_auth`)
	`log_auditoria` (LogAuditoria)	Trilha LGPD art. 37: autor (`created_by_id`) + snapshot `created_by_name` ("Nome <email>"), ação, `tabela` + `registro_pk` (bigint), detalhes JSON (PII mascarada), IP. Append-only no PG (0059): só a purga com 180+ dias, a pseudonimização do esquecimento e o `SET NULL` do autor
	`historico_alteracao` (HistoricoAlteracao)	**Trilha de dados** gravada por trigger (0056, só PG) em prontuario, atendimento, compra_pacote, carteira, regra_comissao, preco, promocao, pacote, versao_termo, usuario e configuracao: operação, só as `colunas_alteradas` com antes/depois (`senha_hash` = `"[redigido]"`), autor/IP/origem da sessão (`web`, `cron:<job>`, `comando:<job>`, `migracao`), `transacao_ref`. Append-only; Django admin só leitura (leitura auditada). `cliente` fica fora de propósito (PII; o esquecimento obrigaria a expurgar a trilha)
Pessoas	`cliente` (Cliente)	Cliente da clínica. Telefone canônico (`utils/telefone`, 10–11 dígitos) é a chave natural; e-mail, CPF, nascimento, CEP e endereço opcionais (ausente = `NULL`) — CPF/CEP/endereço só pela finalidade (nota fiscal e contrato de pacote); RG, profissão e foto não são coletados (saíram na 0074); consentimentos de comunicação na tabela `consentimento`

		(histórico, 0050); saude_revogada_em (revogação do consentimento de dado de saúde; NULL = vigente ou nunca dado — a concessão é o aceite SAÚDE, e um aceite novo limpa a data; 0075); qtd_faltas_consecutivas (FALTOU e cancelamento tardio somam)/eh_bloqueado_online; soft delete (deleted_at); cliente_indicador_id (cashback); nome com collation pt_br
	profissional (Profissional)	Quem atende. antecedencia_minima_horas (0–720, padrão 2), antecedencia_maxima_dias (1–365, padrão 60), vitrine (especialidade, foto, biografia). A agenda vai para o calendário dela só pelo feed ICS (link_acesso FEED_ICS; o Google Agenda saiu na 0071)
Catálogo	procedimento (Procedimento)	Serviço: duração, intervalo_minutos (folga após, 0–120), categoria, retorno obrigatório (tem_retorno_obrigatorio + janela em dias + duração do retorno). Todo atendimento é presencial (modalidade e link de videoconferência saíram na 0073)
	habilitacao (Habilitacao)	Profissional × procedimento (M2M through)
	preco (Preco)	Preço versionado por data_inicio_vigencia, base (sem profissional) ou por profissional
	promocao (Promocao)	Desconto 0–100% **XOR** preço fixo, período (vale pela data do atendimento), procedimento (nulo = geral, só percentual vale); disparo por e-mail com o estado na própria linha (envio_cursor = último cliente já reservado num lote em andamento, disparada_em = início do último disparo; 0068)
Agenda	agenda_horario (AgendaHorario)	Grade semanal: várias janelas por dia, sem sobreposição; dia_semana **ISO** (1=seg … 7=dom = isoweekday())
	agenda_excecao (AgendaExcecao)	Tudo que muda a grade numa data: **FOLGA** (intervalo data_inicio..data_fim; sem profissional = clínica inteira: feriado nacional/local, recesso), **HORARIO_DIFERENTE** (substitui a grade num dia) e **BLOQUEIO** (data_hora_*, pontual ou recorrente por RRULE no subconjunto DAILY/WEEKLY/MONTHLY + data_fim_recorrencia; sem profissional = "Todos"). origem MANUAL/FERIADO_NACIONAL/FERIADO_LOCAL/RECESSO (GCAL e origem_ref saíram na 0071; as importadas viraram MANUAL); eh_ativo=false = registrada e ignorada (ex.: feriado em que a clínica abre). 0048 unificou as 4 tabelas antigas
	atendimento (Atendimento)	Agendamento/sessão. FSM de 7 estados, preço gravado (valor_cobrado, valor_original, promocao, descricao_preco), retorno (eh_retorno + atendimento_origem_id), atendimento_reagendado_id (o antigo que este substitui)
	notificacao (Notificacao)	Mensagem por atendimento: tipo (LEMBRETE — D-1 por WhatsApp ou e-mail —, CONFIRMACAO, CANCELAMENTO, NPS, APROVACAO, TERMO; LEMBRETE_2H e PESQUISA, sem produtor, saíram na 0073), canal, status PENDENTE/ENVIADO/FALHOU; também a fila do e-mail (tentativas, proxima_tentativa_em, índice parcial ix_notificacao_fila_email, 0061); o link público (/confirmar/, /nps/, /termo/) vive em link_acesso
	lista_espera (ListaEspera)	Pedido de aviso de vaga (procedimento, data, turno_desejado MANHA/TARDE/NOITE ou NULL = qualquer, profissional_desejado_id opcional). **Anônima** (nome_contato + telefone_contato, sem cadastro) ou ligada à cliente (cliente_id, nulável) quando o celular foi verificado por OTP — só então aceita email_contato (CHECK); ultimo_aviso_em = último aviso entregue (a inscrição continua na fila; era eh_notificado até a 0076). Sai quando a data passa ou a cliente agenda
Clínico	prontuario (Prontuario)	Ficha base 1:1 com a cliente: alergias, contraindicações, histórico, medicamentos, observações + respostas (JSONB {key: valor} do questionário do prontuário = formulário PRONTUARIO ativo, 0051; atributo Python respostas_extras)
	prontuario_versao (ProntuarioVersao)	**Histórico append-only** da ficha: foto do estado anterior a cada edição, com autor (created_by_id + created_by_name, 0046/0079); DELETE só no expurgo de saúde aos 20 anos (0077)
	anotacao_sessao (AnotacaoSessao)	Nota clínica por atendimento; autor PROTECT (created_by_id) + snapshot created_by_name; append-only, DELETE só no expurgo de

		saúde aos 20 anos (0077)
	formulario (FormularioAnamnese)	Questionário único em perguntas ([{key, tipo, label, obrigatorio, opcoes?, alerta_saude?}]): ANAMNESE e PESQUISA (respondidas pela cliente) e PRONTUARIO (perguntas extras da ficha, preenchidas pela equipe — no máximo 1 ativo, sempre GLOBAL, nunca obrigatório; 0051). Escopo GLOBAL/CATEGORIA/PROCEDIMENTO (o MODALIDADE saiu na 0073) + eh_obrigatorio. Era formulario_anamnese até a 0054
	resposta_formulario (RespostaAnamnese)	Ficha respondida (respostas, respondido_em) por cliente/atendimento; link público do convite em link_acesso (FORMULARIO, 60 dias). Era resposta_anamnese até a 0054
LGPD	versao_termo (VersaoTermo)	Termo versionado (LGPD, SAUDE ou PROCEDIMENTO); 1 ativo por escopo; imutável depois do 1º aceite. A v1.0 LGPD é a Política de Privacidade completa (conteudo/lgpd.TERMO_LGPD_CONTEUDO; 0045 e seed, reescrita sem aceite pela 0078) e a página /politica-de-privacidade/ mostra a versão vigente (VersaoTermo.lgpd_vigente)
	aceite_termo (AceiteTermo)	Prova do aceite: cliente, versão, atendimento, ip_origem, user-agent, conteudo_hash (SHA-256 do texto, char(64); NULL = aceite anterior à 0044), data. Imutável (trigger), salvo a minimização LGPD (IP → NULL, user-agent → ''; 0077). Aceite SAÚDE novo reativa o consentimento de saúde revogado
	codigo_otp (CodigoOtp)	Desafio OTP (codigo_hash HMAC-SHA256 com a SECRET_KEY, char(64)) por identificador (e-mail ou sms+<dígitos>@shivazen.local), propósito AGENDAMENTO/LOGIN_CLIENTE/DSAR, TTL, qtd_tentativas
	consentimento (Consentimento)	**Histórico append-only** de concessão/revogação por finalidade × canal (MARKETING/EMAIL, PESQUISA_SATISFACAO/WHATSAPP, LEMBRETE_AGENDAMENTO/WHATSAPP): origem (AGENDAMENTO_ONLINE, DESCADASTRO, PAINEL, ESQUECIMENTO, MIGRACAO, ADMIN, WHATSAPP — o "PARAR" respondido no WhatsApp, 0069 —, PORTAL — "Minhas preferências" da área da cliente, 0075), texto exibido, IP, user-agent, autor, registrado_em. Estado atual = linha mais recente do par; sem linha = não concedido (0050). O consentimento de dado de saúde fica fora daqui (aceite SAÚDE + cliente.saude_revogada_em)
	link_acesso (LinkAcesso)	Link público com token ("link mágico"): finalidade (CONFIRMAR_PRESENCA, NPS, TERMO, FORMULARIO, DESCADASTRO, FEED_ICS) + exatamente 1 alvo (notificação, ficha, cliente ou profissional; o legado GERENCIAR_AGENDAMENTO e o alvo atendimento saíram na 0072); só o **SHA-256** do token (token_hash char(64), sem pepper: rotacionar a SECRET_KEY não derruba link enviado), expira_em, usado_em, revogado_em. O token em claro só existe na URL entregue (0049)
NPS	avaliacao_nps (AvaliacaoNPS)	Nota 0–10 + comentário, eh_publicacao_autorizada (opt-in da cliente) e eh_publicacao_aprovada (moderação; CHECK: só com o opt-in), eh_alerta_enviado
Pacotes	pacote / pacote_item	**Catálogo** (editável a qualquer momento): pacote (valor_total, validade em meses) e seus itens (procedimento × qtd_sessoes) — só vale p/ vendas futuras. pacote_item era item_pacote até a 0054
	compra_pacote (CompraPacote)	Venda a uma cliente (CompraPacote.vender): pacote_nome congelado, valor_pago, valor_reembolsado, status ATIVO/FINALIZADO/CANCELADO/EXPIRADO, data_expiracao (prorrogável pelo ADMIN), cancelado_em (instante do cancelamento: o reembolso deduz a receita do mês dele — competência, 0065)
	compra_pacote_item (CompraPacoteItem)	**O que foi vendido** (snapshot imutável, 0052): procedimento, procedimento_nome, qtd_sessoes, valor_unitario (valor pago rateado pelo preço de tabela de cada procedimento no dia da venda; uniforme se falta preço — base da comissão da sessão). Saldo, débito, finalização, e-mail de expiração e DSAR leem daqui
	consumo_sessao (ConsumoSessao)	1 sessão consumida por atendimento (OneToOne) de 1 item da compra (compra_pacote_item)

Financeiro	carteira / movimento_carteira	Saldo da cliente (valor_saldo, CHECK ≥ 0) e ledger append-only (trigger; valor > 0, valor_saldo_resultante ≥ 0, autor created_by_id) — programa de indicação: origem CASHBACK_INDICACAO/CASHBACK_ESTORNO, USO_AGENDAMENTO (crédito usado no valor do atendimento), CANCELAMENTO (devolução) e AJUSTE_MANUAL, sempre explícita (sem default; VALE_PRESENTE e OUTRO saíram na 0067). Reembolso de pacote é em dinheiro, nunca crédito
	regra_comissao / movimento_comissao	Regra (percentual 0–100 XOR valor_fixo; usada não muda nem se exclui — só desativa: tg_regra_comissao_validar, 0066) e **livro-razão** de comissão (0053): CREDITO (no máximo 1 ativo — não estornado — por atendimento, 0063; regra_comissao_id + snapshot valor_base_calculo/percentual_aplicado, 0066), PAGAMENTO (baixa ou repasse em lote) e ESTORNO (referenciam o crédito, created_by_id). Status PENDENTE/PAGA/ESTORNADA e saldo (pode ficar negativo) são derivados (creditos_com_status, ComissaoService.saldos_por_profissional)
Infra	configuracao (Configuracao, models/configuracao.py)	Classe EQUIPE no inventário LGPD (guarda o contato do encarregado e o email_admin). Chave-valor com dois donos: a marca e os contatos (utils/branding.BRANDING_FIELDS, tela Identidade visual, validação própria) e as **regras operacionais** — registro tipado CHAVES (email_admin: e-mail válido ou vazio; MAX_FALTAS_BLOQUEIO: inteiro de 1 a 10, padrão 3; anos_experiencia: inteiro **opcional** de 1 a 60, padrão em branco — vazio esconde o número no site), lido por Configuracao.ler (valor em branco ou recusado pelo registro → padrão, com aviso no log). A tela Regras operacionais lista e grava só o registro, sem chave livre; "Restaurar padrão" apaga a linha. As perguntas do prontuário saíram daqui na 0051 (formulário PRONTUARIO) e o cursor do disparo de promoção na 0068 (vai na promocao)
	execucao_job (ExecucaoJob)	Uma linha por job de mensagem sem marca por destinatário (aniversario, pacote_expirando, nps_24h) e dia local: UNIQUE job+dia faz a 2ª chamada do dia receber 409 em vez de reenviar tudo (jobs.executar); concluido_em NULL = em andamento ou falhou, resultado = resumo ou tipo do erro. Sem dado pessoal (0070)

5.3 Invariantes garantidas pelo banco

Invariante	Objeto	Migration
Profissional não atende 2 clientes ao mesmo tempo (status PENDENTE/AGENDADO/CONFIRMADO)	ex_atendimento_sobreposicao — EXCLUDE gist com btree_gist, DEFERRABLE (**só Postgres**)	0035 / 0055
Telefone/e-mail/CPF únicos entre clientes ativos (sem caixa no e-mail); vazio não existe (ausente = NULL)	uq_cliente_telefone_ativo, uq_cliente_email_ativo (UPPER(email): também é o índice do email__iexact), uq_cliente_cpf_ativo, ck_cliente_email_nao_vazio	0034 / 0055
Formato de telefone (10–11 dígitos) e CPF (11)	ck_cliente_telefone_formato, ck_cliente_cpf_formato (**só Postgres**)	0034 / 0055
1 termo ativo por escopo; 1 preço por vigência	uq_versao_termo_ativa_por_procedimento/_global, uq_preco_vigencia/uq_preco_base_vigencia	0034 / 0055
Lista de espera: 1 inscrição por pessoa (cliente ou celular de contato) + procedimento + dia; anônima tem nome e celular (10–11 dígitos); e-mail de contato só com cliente verificada	uq_lista_espera_cliente, uq_lista_espera_telefone_contato, ck_lista_espera_cliente_ou_contato, ck_lista_espera_telefone_contato_formato, ck_lista_espera_email_contato_so_verificado	0076
Promoção: desconto 0–100, preço ≥ 0, desconto XOR preço, geral só percentual	ck_promocao_*	0034 / 0047 / 0055
Carteira nunca negativa; ledger imutável com valor > 0, tipo/origem no domínio e saldo resultante ≥ 0	ck_carteira_valor_saldo_nao_negativo, ck_movimento_carteira_*; trigger tg_movimento_carteira_imutavel	0034 / 0038 / 0054 /

	(fn_bloquear_mutacao, **só PG**)	0055
Valor pago de pacote ≥ 0; nota NPS 0–10 (aprovar depoimento exige o opt-in); comissão 0–100%, valor fixo ≥ 0	ck_compra_pacote_valor_pago_nao_negativo, ck_avaliacao_nps_nota_0_10, ck_avaliacao_nps_aprovacao_exige_autorizacao, ck_regra_comissao_*	0039 / 0044 / 0055
No máximo 1 retorno vivo por atendimento de origem; valores do atendimento ≥ 0	uq_atendimento_retorno_por_origem, ck_atendimento_valores_nao_negativos	0043 / 0055
Notificação e OTP com tipo/canal/status/propósito no domínio; tentativas 0–10; hashes sha256 hex	ck_notificacao_{tipo,canal,status,resposta}, ck_codigo_otp_{canal,proposito,qtd_tentativas_0_10}; ck_codigo_otp_codigo_hash_formato, ck_aceite_termo_conteudo_hash_formato (**só PG**)	0055
JSON no formato certo (objeto/lista)	ck_prontuario_respostas_objeto, ck_resposta_formulario_respostas_objeto, ck_formulario_perguntas_lista (validados na 0055), ck_log_auditoria_detalhes_objeto, ck_prontuario_versao_dados_objeto (**só PG**)	0043 / 0055
Prova de aceite imutável (só a minimização LGPD de IP/user-agent passa); termo aceito imutável; histórico da ficha imutável (DELETE só com a flag local app.expurgo_autorizado e 20 anos de idade)	triggers tg_aceite_termo_imutavel, tg_versao_termo_imutavel, tg_prontuario_versao_imutavel (**só PG**; SQLSTATE 23000 → IntegrityError)	0046 / 0054 / 0077
Ordenação pt-BR de nomes	collation ICU pt_br em cliente/profissional/procedimento.nome, no estado do Django (db_collation; o SQLite dos testes tem um equivalente em db/collation.py)	0038 / 0054
Um cashback por atendimento	uq_movimento_carteira_cashback_por_atendimento	0025/0032 / 0055
Padrão de nomes: PK pk_<t>, FK fk_<t>_<coluna> com ON DELETE igual ao on_delete do model, UNIQUE de OneToOne uq_, sequência <t>_id_seq, nenhum índice _like, COMMENT em toda tabela	PadronizarNomes() + tests/test_nomes_padrao.py (**só PG**)	0055
Grade sem janelas sobrepostas; folgas ativas do mesmo profissional sem sobreposição	ex_agenda_horario_sobreposicao, ex_agenda_excecao_folga_sobreposta — EXCLUDE gist (**só PG**)	0048
Exceção coerente com o tipo (FOLGA = datas, HORARIO_DIFERENTE = 1 dia + horas, BLOQUEIO = instantes); feriado/recesso só como FOLGA da clínica; origem MANUAL/FERIADO_NACIONAL/FERIADO_LOCAL/RECESSO	ck_agenda_excecao_* (tipo, origem, folga, horario_diferente, bloqueio, recorrencia, feriado_global)	0048 / 0071
RRULE só no subconjunto que o motor expande	ck_agenda_excecao_regra_recorrencia_suportada (regex, **só PG**; clean() normaliza)	0048
1 horário diferente ativo por profissional/dia; 1 feriado por origem/data	uq_agenda_excecao_horario_diferente_dia, uq_agenda_excecao_feriado	0048
Link público: alvo coerente com a finalidade (1 FK: notificação, ficha, cliente ou profissional); 1 link por hash; hash = sha256 hex; depois de criado só usado_em/revogado_em mudam (uma vez cada)	ck_link_acesso_finalidade, ck_link_acesso_alvo_por_finalidade, uq_link_acesso_token_hash; ck_link_acesso_token_hash_formato e trigger tg_link_acesso_imutavel (**só PG**)	0049 / 0072
Consentimento: só os 3 pares finalidade × canal que existem, origem no domínio (inclui WHATSAPP e PORTAL); histórico append-only (só a pseudonimização pode apagar IP/user-agent)	ck_consentimento_finalidade, _canal, _finalidade_canal, _origem; trigger tg_consentimento_imutavel (**só PG**)	0050 / 0069 / 0075
Questionário: tipo/escopo no domínio e coerentes (escopo PROCEDIMENTO/CATEGORIA com o alvo); PRONTUARIO sempre GLOBAL e não obrigatório; no máximo 1 PRONTUARIO ativo	ck_formulario_tipo, ck_formulario_escopo, ck_formulario_escopo_coerente, ck_formulario_prontuario_global, uq_formulario_prontuario_ativo	0051 / 0073
Papel da equipe só ADMIN/PROFISSIONAL; tipo de notificação só os 6 com produtor	ck_usuario_papel, ck_notificacao_tipo	0072 / 0073
Pacote vendido: 1 item por procedimento na compra, sessões > 0, valor ≥ 0; item imutável; o consumo é de um item da mesma compra, do	uq_compra_pacote_item_compra_procedimento, ck_compra_pacote_item_*; FK composta fk_consumo_sessao_compra_pacote_item_compra,	0052

mesmo procedimento, nunca de retorno e só com saldo	triggers `tg_consumo_sessao_validar` e `tg_compra_pacote_item_imutavel` (**só PG**; o Python espelha no `save()`)	
Comissão: no máximo 1 CREDITO ativo (não estornado) por atendimento — estornado, pode nascer outro (valor corrigido, realização refeita) —, ≤ 1 PAGAMENTO e ≤ 1 ESTORNO por crédito, valor > 0, origem só fora do CREDITO, regra e snapshot (`valor_base_calculo`, `percentual_aplicado` 0–100) só no CREDITO, comissão nunca acima da base; PAGAMENTO/ESTORNO repetem profissional/atendimento/valor do crédito; estornado não se paga; ledger append-only	`…_estorno_por_origem`, `…_pagamento_por_origem`, `ck_movimento_comissao_*` (inclui `_snapshot_so_credito`, `_valor_ate_base_calculo`, `_percentual_aplicado_0_100`); triggers `tg_movimento_comissao_validar` (o crédito ativo único, 0063) e `tg_movimento_comissao_imutavel` (**só PG**; o Python espelha no `save()`)	0053 / 0063 / 0066
Regra de comissão já usada não muda profissional, procedimento, percentual nem valor fixo (só desativa)	trigger `tg_regra_comissao_validar` (**só PG**; `RegraComissao.save()` espelha)	0066
Compra de pacote: `cancelado_em` preenchido se e só se CANCELADO; só a cancelada tem reembolso	`ck_compra_pacote_cancelado_em_coerente`, `ck_compra_pacote_reembolso_so_cancelado`	0065
Movimento da carteira só com origem que tem fluxo, sempre explícita	`ck_movimento_carteira_origem` (sem VALE_PRESENTE/OUTRO, sem default)	0067
Job de mensagem roda no máximo 1 vez por dia (local)	`uq_execucao_job_job_dia`	0070
`updated_at` sempre atualizado, inclusive em `QuerySet.update()`, procedure e SQL manual (o valor que o ORM manda é respeitado)	`fn_set_updated_at` + `tg_<t>_updated_at` nas 24 tabelas com a coluna (**só PG**; o teste de conformidade reprova tabela com a coluna e sem o trigger)	0056 / 0079
Nota de sessão é append-only (correção = nova nota); DELETE só no expurgo de saúde (flag local e 20 anos)	`tg_anotacao_sessao_imutavel` → `fn_anotacao_sessao_imutavel` (**só PG**; o Python recusa save/delete/update)	0056 / 0077
Toda escrita nas 11 tabelas sensíveis fica na trilha `historico_alteracao` (autor, IP, origem); a trilha é append-only — só o `SET NULL` do autor e a purga de retenção (flag local `app.expurgo_autorizado` e 180+ dias; a do prontuário sai com o expurgo de saúde aos 20 anos, e o prontuário expurgado não deixa cópia na trilha) passam	`fn_registrar_historico` + `tg_<t>_historico`; `fn_historico_alteracao_imutavel` + `tg_historico_alteracao_imutavel`; `ck_historico_alteracao_*` (**só PG**)	0056 / 0059 / 0077
`log_auditoria` append-only: UPDATE só da pseudonimização (`acao` com `[ANONIMIZADO-<id>]`, `detalhes` NULL) e do `SET NULL` do autor; DELETE só com 180+ dias	`fn_log_auditoria_imutavel` + `tg_log_auditoria_imutavel` (**só PG**)	0059

Além disso: FKs `PROTECT` em tudo que é histórico clínico/financeiro (prontuário, anotação, ficha, compra de pacote, carteira, aceite), índices só onde há query real ou FK (spec §2.11; o GIN de `prontuario` saiu na 0055 — nenhuma query de contenção), `COMMENT ON` em todas as tabelas e nas colunas não óbvias. Detalhe por migration: `specs/remodelagem-banco-v2.md` (0027–0047) e `specs/padrao-sql.md` (0048+).

5.4 Lógica no banco (0056–0060, só Postgres)

O SQLite dos testes não tem nada disto: o Python cobre (auto_now, `save()`/`delete()` que recusam, fallback das procedures). Texto SQL versionado em `aranha_estetica/migrations/sql/vNNNN_*.py` (mudar = módulo + migration nova com `CREATE OR REPLACE`). Detalhe e desvios: spec §5 e §5.9.

Objeto	O que é	Quem usa
`tg_<t>_updated_at` (24)	`fn_set_updated_at`: preenche `updated_at` quando o UPDATE não mexeu nela	todo UPDATE
`tg_<t>_historico` (11)	`fn_registrar_historico` (AFTER, por linha): grava `historico_alteracao` — só colunas alteradas; UPDATE só de `updated_at` não gera	prontuario, atendimento, compra_pacote, carteira, regra_comissao, preco,

	linha	promocao, pacote, versao_termo, usuario (senha_hash redigida), configuracao
tg_*_imutavel / tg_*_validar (13)	append-only (movimento_*, anotacao_sessao e prontuario_versao — DELETE só no expurgo de saúde aos 20 anos, 0077 —, compra_pacote_item, aceite_termo — só a minimização de IP/user-agent —, versao_termo aceita, consentimento, link_acesso, historico_alteracao) e validação (consumo_sessao, movimento_comissao — crédito ativo único, 0063 —, regra_comissao usada, 0066) — SQLSTATE 23 → IntegrityError	—
fn_fuso_clinica(), fn_data_local(ts), fn_sem_acento(txt)	fuso explícito (America/Sao_Paulo = TIME_ZONE), data local, lower(unaccent()) IMMUTABLE	views/funções; ix_cliente_nome_trgm (GIN trigram) e a busca de cliente (utils/busca.filtrar_clientes: "joao" acha "João")
fn_horarios_livres, fn_ocorrencias_recorrencia, fn_recorrencia_casa_dia, fn_saldo_pacote, fn_nps_periodo, fn_saldo_comissao, fn_tem_consentimento, fn_clientes_para_anonimizar	**espelho de leitura** do Python (SlotService, AgendaExcecao.ocorrencias, saldo_por_procedimento, utils/nps, ComissaoService, services/consentimentos); fn_clientes_para_anonimizar é a regra da retenção (sem espelho: LgpdService.candidatos_retencao lê dela; desde a 0077 retém só pela janela de saúde de 20 anos e até 5 anos depois do fim do pacote), travado por tests/test_pg_paridade.py	relatório/integração e as procedures — **o app continua no Python** (slots nunca vêm do SQL)
vw_agenda_dia, vw_faturamento_mensal, vw_nps_mensal, vw_comissao, vw_comissao_saldo, vw_cliente_resumo, vw_consentimento_atual, vw_pacote_saldo, vw_atendimento_sem_desfecho	views de relatório (mesmas regras de services/financeiro, NPS e comissão). vw_faturamento_mensal é por **competência** (0065): avulsos no mês do atendimento, pacotes = vendas no mês da venda − reembolsos no mês do cancelamento (pode ser negativo; colunas valor_vendas_pacotes/valor_reembolsos_pacotes no fim); vw_comissao traz o snapshot do crédito (valor_base_calculo, percentual_aplicado; 0066)	BI/SQL ad hoc (railway connect → SELECT * FROM vw_faturamento_mensal); o app não lê views
sp_anonimizar_cliente(cliente, autor, ip)	direito ao esquecimento = **pseudonimização** (efeito completo testado em test_pg_paridade/test_lgpd_esquecimento); versão vigente da 0077: também zera código de indicação e quem indicou, tira IP/user-agent do aceite LGPD (e dos de SAÚDE/PROCEDIMENTO quando não há mais registro de saúde), apaga a inscrição anônima da lista de espera pelo celular e os detalhes das trilhas LGPD: ... da titular	LgpdService.esquecer_cliente (ação "Pseudonimizar" do admin)
sp_aplicar_retencao_lgpd(...) (9 argumentos)	pseudonimiza os candidatos (fn_clientes_para_anonimizar; COMMIT por cliente), apaga fichas de pedido não realizado e espera vencida (inclusive anônima)	job_lgpd_purgar_inativos (LgpdService.aplicar_retencao)
sp_purgar_dados_expirados(...) (12 argumentos)	OTP, texto de notificação, auditoria, link_acesso vencido/revogado, trilha antiga e, desde a 0077, a etapa **saude**: expurgo do registro de saúde (prontuário, versões, anotações, fichas e a trilha do prontuário) de quem teve o último evento de saúde há mais de RETENCAO_ATENDIMENTO_SAUDE_DIAS, com a minimização dos aceites SAÚDE/PROCEDIMENTO e o registro na log_auditoria (cada etapa num sub-bloco)	job_housekeeping (etapa banco)

`sp_expirar_pacotes(hoje)`	compras ATIVO vencidas → EXPIRADO	`job_expirar_pacotes`

Contexto da sessão (GUCs app.*) lido pela trilha: `app.usuario_id`, `app.ip_origem`, `app.origem`. Quem define (`utils/banco.definir_contexto_banco`, `set_config(..., false)` = nível de sessão) e **sempre limpa**: o `Ator` (`ator.atuando`) aberto pelo `ContextoBancoMiddleware` (`web`

- usuário + IP; o `/healthz/` não toca o banco) e por `views/infra/cron.py` (`cron:<job>`; aninhado, volta

ao do request na saída) — o mesmo autor/IP vai à `LogAuditoria` —, `rodar_job` e `rodar_jobs_devidos` (`comando:<job>`), `migrate_atomico` (`SET LOCAL app.origem = 'migracao'`); conexão nova nasce limpa (`connection_created`). SQL manual sai com autor `NULL` e origem `''`. **PgBouncer em modo transação quebraria isso** (o Railway não usa): nesse dia, trocar para `ATOMIC_REQUESTS` + `SET LOCAL`.

Fonte única no Postgres (decisão de 2026-10-27, ARCHITECTURE D23): `utils/banco.chamar('sp_…', ...)` (allowlist com a aridade) é o único caminho; não há fallback Python. Fora do Postgres ele recusa com `ImproperlyConfigured` (dev: `docker compose up -d db`), e os testes dessas rotinas são só-Postgres.

5.5 Padrão de nomes e metadados

Elemento	Regra	Exemplos
Tabela (conceito autônomo)	substantivo singular na ordem natural `substantivo[_complemento]`; sem prefixo de app	`atendimento`, `cliente`, `aceite_termo`, `regra_comissao`, `movimento_carteira`, `versao_termo`
Tabela (parte de um agregado)	prefixo do agregado: `<pai>_<parte>` (agrupa no `\dt pai*`)	`agenda_horario`, `agenda_excecao`, `pacote_item`, `compra_pacote_item`, `prontuario_versao`
Associativa N:N	substantivo semântico quando existe; senão `<tabela_a>_<tabela_b>` na ordem de dependência	`habilitacao` (profissional × procedimento)
PK	`id bigint GENERATED BY DEFAULT AS IDENTITY` (`BigAutoField`)	—
FK	`<tabela_referenciada>_id`	`cliente_id`, `procedimento_id`
FK com papel / 2+ FKs p/ a mesma tabela	`<tabela_referenciada>_<papel>_id` (papel = o que a linha referenciada é para esta)	`atendimento_origem_id`, `atendimento_reagendado_id`, `cliente_indicador_id`, `profissional_desejado_id`, `movimento_comissao_origem_id`
FK de autoria (quem registrou)	sempre `created_by_id` (metadado, §1.1 item 6); nunca `user_id`/`usuario_id` p/ autoria	`anotacao_sessao.created_by_id`
Snapshot de atributo de FK	`<fk sem _id>_<atributo>` — congela o valor no momento do registro	`created_by_name`, `pacote_nome`, `procedimento_nome`
`_id`	**reservado a FK real**. Id de sistema externo: `_ref`. Chave polimórfica: `registro_pk`	`gcal_calendario_ref`, `agenda_excecao.origem_ref`, `log_auditoria.registro_pk`
Data (`date`)	`data_<qualificador>`	`data_nascimento`, `data_expiracao`, `data_inicio_vigencia`, `data_fim_recorrencia`
Instante (`timestamptz`)	`<particípio>_em`, **masculino** (uniforme); metadado de linha em inglês (`created_at`, `updated_at`, `deleted_at`)	`enviado_em`, `respondido_em`, `expira_em`, `usado_em`, `revogado_em`, `registrado_em`, `pago_em`
Exceção aprovada	`data_hora_inicio` / `data_hora_fim` (intervalo de atendimento e de bloqueio)	`atendimento`, `agenda_excecao`
Hora do dia (`time`, hora local)	`hora_<qualificador>`	`hora_inicio`, `hora_fim`
Booleano	`eh_<adjetivo/particípio>` ou	`eh_ativo`, `eh_retorno`,

	`tem_<substantivo>`; `NOT NULL` com default; forma masculina em qualquer tabela	`eh_publicacao_aprovada`, `tem_retorno_obrigatorio`
Dinheiro	`valor_<qualificador> numeric(10,2)` (agregado de view: `numeric(12,2)`). `valor` puro só quando a linha **é** o valor (`preco`, `movimento_*`)	`valor_cobrado`, `valor_saldo`, `valor_total`, `valor_fixo`
Contagem	`qtd_<coisa>` (`smallint`/`integer`)	`qtd_sessoes`, `qtd_tentativas`, `qtd_faltas_consecutivas`
Percentual	`percentual_<qualificador> numeric(5,2)` + CHECK 0–100; `percentual` puro na `regra_comissao` (é o assunto da linha)	`percentual_desconto`
Medida com unidade	`<grandeza>_<unidade>`	`duracao_minutos`, `intervalo_minutos`, `antecedencia_minima_horas`, `validade_meses`
URL	`url_<qualificador>`	`url_foto`, `url_imagem_destaque`, `url_videoconferencia`
Status / tipo / canal (enum)	`varchar(n)` + `ck_<tabela>_<coluna>` com `IN (...)`; valores MAIÚSCULOS ASCII. **Sem ENUM nativo** (trocar valor de ENUM exige `ALTER TYPE` e acopla o tipo ao Django; CHECK troca em migration comum). Enum opcional: `NULL` = não se aplica	`status`, `tipo`, `canal`, `origem`
JSON	`jsonb` + `ck_<tabela>_<coluna>_objeto` / `_lista` (`jsonb_typeof`); **sem** sufixo `_json`	`respostas`, `perguntas`, `detalhes`, `dados`
Hash de segredo	`<x>_hash char(64)` (sha256/HMAC hex) + `ck_<tabela>_<x>_hash_formato` (`^[0-9a-f]{64}$`). Exceção: `usuario.senha_hash varchar(255)` (formato `pbkdf2_sha256$…` do Django)	`token_hash`, `codigo_hash`, `conteudo_hash`
IP de origem	`ip_origem inet` (inválido vira `NULL`, nunca inventado)	—
Texto	`NOT NULL DEFAULT ''`. Exceções: (i) colunas sob UNIQUE parcial (`email`, `cpf`, `telefone`) — ausente = `NULL` + CHECK `<> ''`; (ii) dado pessoal opcional do titular (`rg`, `cep`, `endereco`, `profissao`) — `NULL` = não informado/apagado pela anonimização	—
Termo técnico de protocolo	mantém o nome do protocolo	`user_agent`, `endpoint`, `chave_p256dh`, `slug`
Auditoria de linha (metadados)	`created_at NOT NULL DEFAULT now()` em **toda** tabela. `updated_at NOT NULL DEFAULT now()` + `tg_<t>_updated_at` (`fn_set_updated_at`) em toda tabela com UPDATE de negócio. Sem `updated_at`: append-only/imutável por trigger, só INSERT/DELETE, e `link_acesso` (o único UPDATE aceito é carimbar `usado_em`/`revogado_em` uma vez). Autoria: `created_by_id` + `created_by_name`. Instante de negócio diferente da inserção ganha coluna própria (`registrado_em`, `respondido_em`)	sem `updated_at`: `log_auditoria`, `historico_alteracao`, `movimento_carteira`, `movimento_comissao`, `prontuario_versao`, `anotacao_sessao`, `aceite_termo`, `compra_pacote_item`, `consentimento`, `habilitacao`, `pacote_item`, `consumo_sessao`, `link_acesso`
Soft delete	só `cliente.deleted_at`	—
Defaults	todo default literal ou `now()` vira também `db_default` (Django ≥ 5.0) — procedures/triggers inserem por SQL	—

Objeto	Padrão	Exemplo
PK	`pk_<tabela>`	`pk_atendimento`
FK	`fk_<tabela>_<coluna sem _id>`; composta: `fk_<tabela>_<1ª coluna sem _id>_<qualificador>`	`fk_atendimento_atendimento_origem`, `fk_consumo_sessao_compra_pacote_item_compra`
Ação da FK	espelha o `on_delete`: `CASCADE→ON DELETE`	—

	`CASCADE`, `SET_NULL`→`ON DELETE SET NULL`, `PROTECT/RESTRICT`→`NO ACTION`; sempre `DEFERRABLE INITIALLY DEFERRED` (como o Django cria)	
UNIQUE (constraint ou índice único parcial/expressão)	`uq_<tabela>_<colunas ou regra>`	`uq_cliente_telefone_ativo`, `uq_movimento_comissao_credito_por_atendimento`
CHECK	`ck_<tabela>_<regra>`	`ck_atendimento_fim_apos_inicio`, `ck_promocao_percentual_desconto_0_100`
Índice	`ix_<tabela>_<colunas>`; parcial `…_parcial`; GIN trigram `…_trgm`; de expressão: nome da expressão	`ix_atendimento_profissional_data_hora_inicio`, `ix_cliente_deleted_at_parcial`, `ix_cliente_nome_trgm`
EXCLUDE	`ex_<tabela>_<regra>`	`ex_atendimento_sobreposicao`
Trigger	`tg_<tabela>_<ação>` — ações padronizadas: `updated_at`, `imutavel`, `historico`, `validar`	`tg_prontuario_historico`
Função	`fn_<nome>`; função de trigger genérica `fn_<ação>`, específica `fn_<tabela>_<ação>`	`fn_set_updated_at`, `fn_aceite_termo_imutavel`
Procedure	`sp_<verbo>_<objeto>`	`sp_anonimizar_cliente`
View	`vw_<assunto>`	`vw_agenda_dia`
Sequência	`<tabela>_id_seq` (padrão do PG; renomeada junto com a tabela)	`carteira_id_seq` (hoje `credito_cliente_id_seq`)
Collation	`pt_br` (ICU determinística, 0038)	—
Configuração de sessão (GUC)	`app.<nome>`	`app.usuario_id`, `app.ip_origem`, `app.origem`

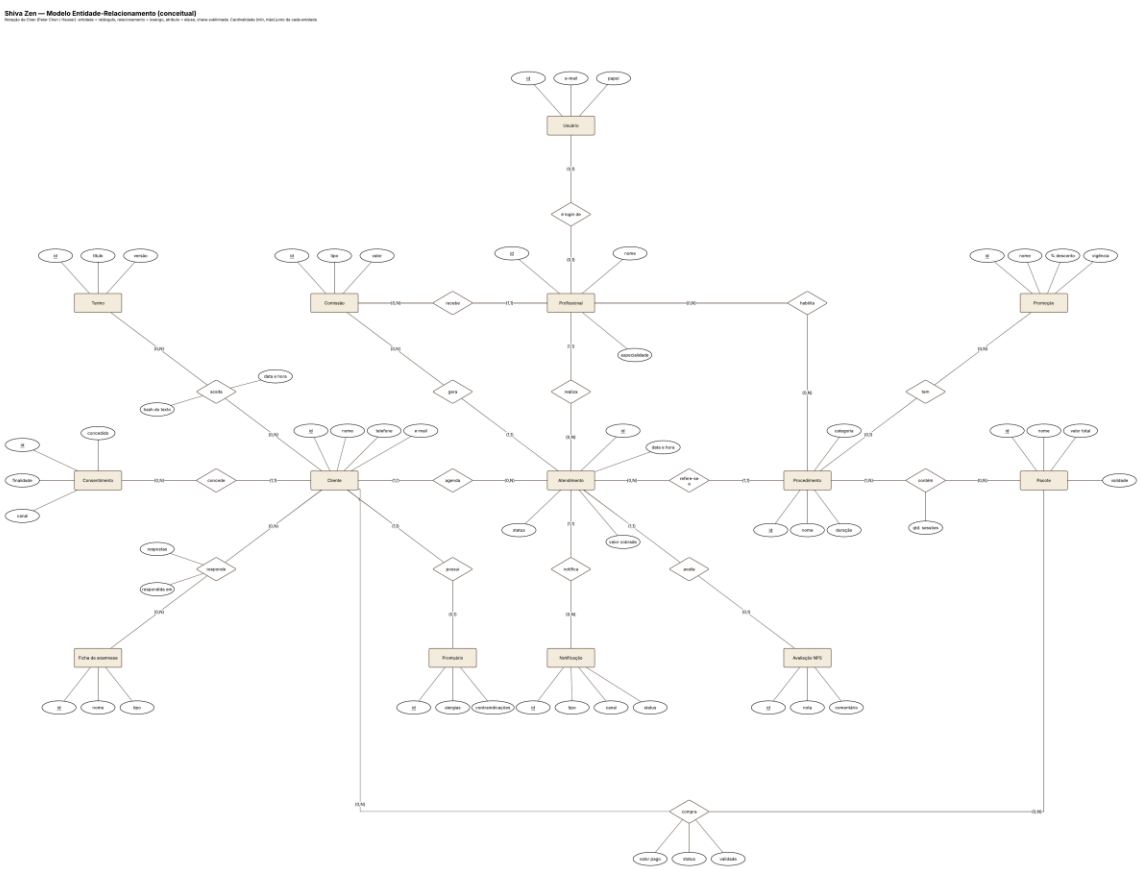


Figura 3 — MER conceitual — notação de Chen.



Figura 4 — DER — visão geral das 37 tabelas por área.

6. Segurança

Detalhe de login, 2FA, OTP e tokens em `AUTENTICACAO-E-ACESSO.md`.

- **Equipe:** e-mail + senha (mín. 10, validadores do Django), django-axes (5 falhas → bloqueio de 1h por IP+usuário e por IP — nunca por usuário sozinho, que trancaria a dona fora com o e-mail público; página pt-BR 429), rate-limit no login (10/min por IP, 5/min por e-mail), **2FA TOTP obrigatório para ADMIN e PROFISSIONAL em prod**, conferido a cada request e falhando fechado (erro de banco na consulta do 2FA leva ao desafio, nunca deixa passar só com a senha).
- **Cliente:** sem senha; OTP de 6 dígitos por SMS preso ao telefone (HMAC-SHA256, 10 min, 5 tentativas, reenvio ≥ 60 s), propósitos isolados. Anti-enumeração: o pedido de código do portal e do "Meus dados" responde igual exista ou não o cadastro, em no mínimo `OTP_LATENCIA_MINIMA_SEGUNDOS` (0,8 s, fixo no settings). Sessão do portal de 1 h presa ao telefone; baixar os dados exige verificação de até 10 min; sair só por POST.
- **Formulários públicos sem login** (contato, lista de espera): captcha quando configurado, honeypot `website` (robô descartado em silêncio) e limite por IP; o contato tem ainda o teto diário global (protege a cota de e-mail do reset de senha e dos avisos).
- **Headers:** CSP com nonce por request (sem `unsafe-inline` em `script-src`/`style-src`, sem handlers inline; `style-src-attr 'unsafe-inline'` ainda permitido), nenhum CDN (o único host externo de script é o Turnstile; FullCalendar e Chart.js vêm do npm no bundle do Vite), `frame-ancestors 'none'` (exceto o widget), HSTS 1 ano com preload, X-Frame-Options DENY, nosniff, COOP/CORP, Permissions-Policy.
- **Privacidade no navegador:** `/painel/`, `/profissional/`, `/meus-agendamentos/`, `/lgpd/`, `/api/`, `/django-admin-sv/` saem com `Cache-Control: no-store` (e views sensíveis com `@never_cache`); rotas com token no path usam `Referrer-Policy: same-origin` (o Referer não sai para outro domínio; `no-referrer` fazia o navegador mandar `Origin: null` no POST da própria página e o CSRF recusava); privadas e com token saem com `X-Robots-Tag: noindex, nofollow, noarchive` (o robots.txt só pede para não buscar); logout envia `Clear-Site-Data: "cache"`; o access log do gunicorn redige tokens (`LoggerSemTokens`) e o log da aplicação passa pelo filtro `clinica/logging_filtros.RedigirLog` (token de link, DETAIL do Postgres, e-mail, CPF e telefone na mensagem, no traceback e nos extras — a mesma redação do Sentry, `utils/pii.redigir_texto`).
- **Cron e webhooks:** segredo só em header, comparado em tempo constante (`utils/seguranca.segredo_confere`); `CRON_TOKEN` e `WHATSAPP_VERIFY_TOKEN` com 32+ caracteres (E003); cron com 30 chamadas/h por IP e corpo de erro só com o tipo da exceção e uma referência do log.
- **Imagem:** usuário `app` sem shell, código e estáticos do root (só leitura para o processo — nada em runtime escreve em `/app`), sem `curl` (o `HEALTHCHECK` usa o Python da imagem); o `.dockerignore` deixa fora testes, scripts de backup, docs, bancos e dumps.
- **Cookies:** HttpOnly, SameSite=Lax, Secure em prod; sessão da equipe 8h deslizantes que expira ao fechar o navegador.
- **IP do cliente:** `utils/seguranca.client_ip` lê `CLIENT_IP_HEADER` (prod: `HTTP_X_REAL_IP` do Railway; nunca X-Forwarded-For). Rate-limit, axes e auditoria usam a mesma função.
- **Entradas:** uploads ≤ 5 MB, ≤ 200 campos, IDs só com dígitos ASCII (`utils/parse.id_int`), células de Excel com texto forçado (anti-fórmula), JSON injetado via `json_script`.
- **Segredos e configuração:** só em env, lida e validada só em `clinica/settings` (o código lê `settings.X` na hora da chamada; ruff `TID251` barra `os.environ` fora dali); `DJANGO_SECRET_KEY` obrigatória fora de DEBUG/no Railway e, publicada, com 50+ caracteres aleatórios (E003); `DJANGO_ENV` inválido derruba o boot. Commit com segredo: gancho e job do CI com gitleaks (§12).

7. LGPD — implementação

7.1 Bases legais (política de privacidade e termo LGPD v1.0)

Fonte única: `conteudo/lgpd.TRATAMENTOS` (a seção 3 da política e a declaração do "baixar meus dados" saem dali). Resumo:

Tratamento	Base legal
Agendar, confirmar, remarcar, cancelar e realizar o atendimento, com os e-mails de confirmação, cancelamento e lembrete; nascimento (18 anos ou mais e segurança do procedimento); código por SMS; lista de espera e contato; programa de indicação	Execução de contrato e procedimentos preliminares (art. 7º, V); o SMS também prevenção a fraudes (art. 7º, IX)
CPF, CEP e endereço só para nota fiscal e contrato de pacote	Obrigação legal e execução de contrato (art. 7º, II e V)
Ficha de avaliação, prontuário e anotações (dado de saúde)	**Só consentimento específico e destacado (art. 11, I), revogável**; o registro feito fica pelo prazo de guarda (art. 7º VI, art. 11 II "d", art. 16)
Lembrete e pesquisa de satisfação por WhatsApp; promoções, novidades e aniversário por e-mail; depoimento no site	Consentimento (art. 7º, I), revogável
Nota e comentário da pesquisa (responder é opcional); segurança do site e prevenção de abusos	Legítimo interesse (art. 7º, IX)
Registros dos atendimentos, dos termos aceitos e dos consentimentos	Obrigação legal e exercício regular de direitos (art. 7º, II e VI; art. 11, II, "d")

Revisão jurídica antes do go-live: o texto da política, as bases, os operadores e os prazos (`conteudo/lgpd.py`) foram escritos a partir do que o sistema faz; a dona (de preferência com um advogado) revisa antes do primeiro deploy de produção (§14 a, `LGPD.md`).

7.2 Consentimentos e provas

- **Política de Privacidade = termo LGPD:** o texto de `conteudo/lgpd.TERMO_LGPD_CONTEUDO` é a v1.0 (gravada pela `0045` e pelo `seed`; a `0078` reescreveu a v1.0 sem aceite) e a página `/politica-de-privacidade/` mostra o texto da versão LGPD **vigente** (`VersaoTermo.lgpd_vigente`, filtro `texto_termo`) — o mesmo cujo SHA-256 vai na prova. Aceite obrigatório no wizard → `AceiteTermo` da versão vigente com IP, user-agent e SHA-256 do texto. Sem termo LGPD ativo o pedido online é recusado (`sem_termo_lgpd`; garanta um — §14.e). Os prazos da tabela da política têm de bater com `settings.RETENCAO_*` (`aranha.W016`).
- **Dado de saúde (art. 11 I):** ficha do wizard e ficha pública (`/anamnese/<token>/`) só são gravadas com a caixa de consentimento destacada marcada; o aceite do termo SAÚDE é a prova e o texto (`TEXTO_CONSENTIMENTO_SAUDE`) vai para a `LogAuditoria` com IP. **Revogável** (`services/consentimentos.revogar_saude`): pelo portal (origem PORTAL) ou pela ação do Django admin "Registrar revogação do consentimento de saúde" (motivo obrigatório, origem ADMIN) — grava `cliente.saude_revogada_em` e a trilha `LGPD: consentimento de saude revogado`; os alertas passam a mostrar a revogação; o registro feito fica pelo prazo (§9.4). Aceite SAÚDE novo reativa (`AceiteTermo.registrar` limpa a data). A ficha ainda em dia (12 meses) continua valendo no wizard depois da revogação (pendência no `ARCHITECTURE.md` §6.2).
- **Comunicação:** três opt-ins no wizard — promoções por e-mail (MARKETING/EMAIL), lembrete D-1 e aviso de vaga por WhatsApp (LEMBRETE_AGENDAMENTO/WHATSAPP) e pesquisa de satisfação por WhatsApp (PESQUISA_SATISFACAO/WHATSAPP) — nunca pré-marcados. Cada concessão/revogação é uma linha nova em `consentimento` (append-only) com o texto exibido, IP, user-agent e data; o estado vale pela linha

mais recente (`services/consentimentos`). Desmarcar (com o estado do cadastro exibido) revoga; marcar de novo depois de um descadastro **volta a valer** (reconsentimento — o antigo `aceita_comunicacao` vetava para sempre e saiu na 0050). O WhatsApp de lembrete é opt-in (política da Meta): sem ele o lembrete D-1 vai por e-mail.

- **Área da cliente** ("Minhas preferências", `consentimentos.registrar_do_portal`): os mesmos três pares, com o estado vigente no checkbox; marcar concede (texto do agendamento), desmarcar revoga; origem PORTAL, com IP e user-agent; par sem mudança não grava.
- **Descadastro** (`/lgpd/unsubscribe/<token>/`): revoga marketing e pesquisa (origem DESCADASTRO, com IP/UA de quem confirmou), **mantém o lembrete D-1** (transacional). Cada e-mail de marketing leva o seu link (`link_acesso` DESCADASTRO; descartado se o envio falha).
- **"PARAR" no WhatsApp** (mensagem inteira "parar", "sair", "stop", "cancelar mensagens" ou "descadastrar", sem acento/caixa): revoga lembrete e pesquisa por WhatsApp do titular daquele telefone (origem WHATSAPP; revogar tão fácil quanto consentir — art. 8º §5), auditado, e a cliente recebe a confirmação com o caminho de volta (texto livre, dentro da janela de 24 h).
- **Contato e lista de espera** não pedem opt-in de marketing (o do contato saiu): o consentimento de promoções só existe no agendamento, onde há prova.
- **Cookies:** só essenciais (sessão, CSRF, tema). O aviso é informativo: "Entendi" grava `cookie_consent=essenciais` no próprio navegador; nada é registrado no servidor.
- **Painel** (ficha do cliente, "Comunicação autorizada"): a equipe vê o estado por canal e **revoga** a pedido da cliente; registrar uma autorização exige a justificativa (o que foi autorizado e como), gravada como `texto` com o autor (origem PAINEL). Django admin: histórico só leitura + ações "Revogar consentimentos de comunicação" (origem ADMIN) e "Registrar revogação do consentimento de saúde".
- **Depoimento:** só aparece no site com `autoriza_publicacao` (cliente) **e** `aprovado_publicacao` (equipe), nota ≥ 9 e comentário.
- **Imutabilidade:** aceite e termo já aceito não mudam (model + trigger 0046); editar o texto = publicar nova versão. O aceite só perde IP/user-agent na minimização LGPD (0077). Aceites anteriores à 0044 ficam com `conteudo_sha256` vazio.

7.3 Direitos do titular

Direito	Como
Confirmação, acesso, portabilidade	**"Baixar meus dados"** no portal (código de até 10 min) ou `/lgpd/meus-dados/` (telefone → OTP DSAR): JSON por seção (`services/lgpd.SECOES_DSAR`) — cadastro, preferências (bloqueio, faltas, revogação de saúde), consentimentos (estado e histórico), atendimentos, NPS, aceites, pacotes, carteira, lista de espera (inclusive a anônima pelo mesmo celular), notificações — e `sobre_o_tratamento` (art. 19 II: finalidades e bases, operadores e prazos efetivos, da mesma fonte da política). **Sem a ficha de saúde** (prontuário, versões, anotações, fichas): o arquivo diz que ela sai por e-mail ou na clínica, a pedido; o export completo é `LgpdService.exportar_dados_cliente` (sem tela no painel). Resposta idêntica exista ou não o cadastro; auditado (`DSAR: exportacao de dados`)
Correção / exclusão	Pedido no portal (`POST /meus-agendamentos/pedido/`): trilha `LGPD: pedido de correcao` ou `de exclusao` com o prazo (15 dias) + e-mail à clínica (encarregado > `email_admin` > `ADMIN_EMAIL` > e-mail da marca) com a descrição. A equipe corrige na ficha (`/painel/clientes/<pk>/`) ou pseudonimiza no Django admin
Revogação de consentimento	Portal (preferências e saúde); link de descadastro; "PARAR" no WhatsApp; desmarcar no wizard; equipe na ficha do cliente (comunicação) ou no Django admin (comunicação e saúde)
Pseudonimização ("esquecimento")	Ação "Pseudonimizar" no Django admin (`LgpdService.esquecer_cliente`, lote tudo ou nada) e job semanal (§9.4)
Revisão de decisão automatizada (art. 20)	Pausa do online por faltas (o cancelamento tardio conta): o wizard dá o motivo e o caminho (WhatsApp); a recepção revisa

Pseudonimização (`esquecer_cliente` → `sp_anonimizar_cliente`, art. 13 §4): troca o nome por `[ANONIMIZADO-<pk>]` e zera CPF, e-mail, telefone, CEP, endereço, nascimento, **código de indicação e quem indicou**; revoga os consentimentos vigentes (origem ESQUECIMENTO — o histórico de datas/finalidades fica como prova de base legal, sem IP e user-agent); tira IP e user-agent do aceite LGPD (e dos de SAÚDE/PROCEDIMENTO quando não há mais registro de saúde); soft delete; apaga OTPs, a lista de espera (inclusive a inscrição anônima com o mesmo celular) e o texto das notificações; apaga fichas que não são de atendimento REALIZADO; tira o depoimento do site e apaga o comentário do NPS; **revoga todos os links públicos** do titular (descadastro, confirmação, NPS, termo, ficha); pseudonimiza o nome nos textos da `LogAuditoria` e do histórico do Django admin e apaga os detalhes das trilhas `LGPD: ...` da titular. Atendimentos, aceites, pacotes e o registro de saúde (até o expurgo aos 20 anos) ficam ligados a esse id — pseudonimização, não anonimização.

7.4 Retenção

Inventário coluna a coluna em `inventario_lgpd.py`, conferido por `tests/test_inventario_lgpd.py` contra os models, as procedures (`sp_anonimizar_cliente`, `sp_aplicar_retencao_lgpd`, `sp_purgar_dados_expirados`) e o registro de operações do `docs/LGPD.md`: tabela ou coluna pessoal nova sem tratamento declarado reprova o CI.

Os prazos **LGPD** (`RETENCAO_CLIENTE_INATIVO_DIAS`, `RETENCAO_CARENCIA_SOFT_DELETE_DIAS`, `RETENCAO_ATENDIMENTO_SAUDE_DIAS`, `RETENCAO_FICHA_SEM_ATENDIMENTO_DIAS`, `RETENCAO_PACOTE_APOS_EXPIRAR_DIAS`) são **fixos** em `settings/base.py`, sem variável de ambiente: são os publicados na política (`conteudo/lgpd.PRAZOS_RETENCAO`), e mudar exige versão nova do termo LGPD — o check `aranha.W016` acusa o settings diferente do texto da versão vigente. `services/lgpd.prazos_retencao` os passa às procedures. OTP, texto das notificações e auditoria também estão publicados na política, mas seguem por env (`RETENCAO_OTP_HORAS`, `RETENCAO_NOTIFICACAO_DIAS`, `RETENCAO_LOG_AUDITORIA_DIAS` — mudar sem versão nova do termo gera o W016); links e axes não aparecem no texto.

Dado	Prazo	Mecanismo
Código OTP	24 h (`RETENCAO_OTP_HORAS`)	`housekeeping` apaga
Sessões expiradas	—	`housekeeping` (`clearsessions`)
Texto das notificações	12 meses (`RETENCAO_NOTIFICACAO_DIAS`)	`housekeeping` limpa `mensagem`
Log de auditoria (com o IP)	5 anos (`RETENCAO_LOG_AUDITORIA_DIAS`, mínimo 180 dias — W013)	`housekeeping` apaga (o trigger só aceita apagar linha com 180+ dias)
Trilha de dados (`historico_alteracao`)	5 anos (`RETENCAO_LOG_AUDITORIA_DIAS`); a do **prontuário** sai com o expurgo de saúde	`housekeeping` apaga (flag local `app.expurgo_autorizado` + 180 dias; fora disso o trigger barra DELETE)
Link público vencido ou revogado	90 dias depois de vencer/ser revogado (`RETENCAO_LINK_ACESSO_DIAS`); feed ICS sem validade fica	`housekeeping` apaga
Logs do django-axes	90 dias (`RETENCAO_AXES_LOG_DIAS`)	`housekeeping` apaga
Registro de saúde (prontuário, versões, anotações, fichas e a trilha do prontuário)	**20 anos depois do último evento de saúde** da cliente: último REALIZADO, edição/versão do prontuário, anotação ou ficha (`RETENCAO_ATENDIMENTO_SAUDE_DIAS`)	`housekeeping`, etapa **saude** do `sp_purgar_dados_expirados` (0077): apaga e minimiza IP/user-agent dos aceites SAÚDE/PROCEDIMENTO, registra a quantidade na `log_auditoria`; os triggers só aceitam com a flag **e** 20 anos (prazo menor no settings = etapa falha, nada sai antes)

Cliente sem atendimento	5 anos (`RETENCAO_CLIENTE_INATIVO_DIAS`)	`lgpd_purgar` **pseudonimiza — exceto** enquanto houver registro de saúde dentro da janela de 20 anos (REALIZADO, versão do prontuário ou prontuário com conteúdo editado no período) ou pacote a menos de 5 anos do fim (`RETENCAO_PACOTE_APOS_EXPIRAR_DIAS`: cancelamento, validade ou, sem validade, a compra). Aceite de termo não retém
Cliente excluído (soft delete)	30 dias (`RETENCAO_CARENCIA_SOFT_DELETE_DIAS`)	`lgpd_purgar` pseudonimiza (mesmas exceções)
Ficha de pedido não realizado (CANCELADO, PENDENTE vencido, convite não respondido ou vazio de horário passado)	90 dias (`RETENCAO_FICHA_SEM_ATENDIMENTO_DIAS`)	`lgpd_purgar` apaga (a respondida de FALTOU/REAGENDADO fica)
Lista de espera	até a data desejada passar ou a cliente agendar o procedimento	`lgpd_purgar` apaga a vencida (inclusive anônima); `services/atendimento.ao_criar` apaga ao agendar
Aceites de termo	nunca apagados (prova)	IP/user-agent minimizados na pseudonimização (LGPD) e no expurgo de saúde (SAÚDE/PROCEDIMENTO)

Cifragem em repouso (0060, `utils/cifragem` + `db/campos.TextoCifrado`/`JsonCifrado`): os 5 campos clínicos e as respostas do prontuário, as versões do prontuário, as anotações de sessão e as fichas de anamnese ficam gravados cifrados (Fernet, chave de `CHAVES_CIFRAGEM` com subchave `saude`; a subchave `segredos` fica para segredo de integração e hoje não tem campo — era o token do Google Agenda, que saiu na 0071); a trilha `historico_alteracao` do prontuário guarda o mesmo texto cifrado. A aplicação lê em claro; vazio continua vazio no banco. Um dump/backup sem a chave não mostra a ficha (e um incidente com dado ininteligível pesa menos — art. 48 §3). Consequência: o banco não busca nem filtra pelo conteúdo desses campos. Não protege contra quem tem a chave (ADMIN no painel, acesso ao Railway).

No Postgres a escrita da retenção é das procedures (`sp_purgar_dados_expirados`, `sp_aplicar_retencao_lgpd`, `sp_anonimizar_cliente` — §5.4), **fonte única**: não há equivalente em Python, então no SQLite (dev sem Postgres) esquecimento, retenção e limpeza não rodam (`docker compose up -d db` — §12).

7.5 Incidente (art. 48)

Passo a passo, prazos (3 dias úteis; registro por 5 anos), registro das operações (art. 37), operadores e o relatório de impacto do dado de saúde: `LGPD.md`.

Conter (revogar tokens/sessões, `setup_2fa --force`, rotacionar `DJANGO_SECRET_KEY` — invalida sessões —, `OTP_HMAC_KEY` e, se a chave de cifragem vazou, `CHAVES_CIFRAGEM` com `rotacionar_chave_cifragem`), avaliar pela `LogAuditoria`, comunicar a ANPD e os titulares afetados quando houver risco relevante.

8. Integrações externas

Preço de cada serviço, estimativa mensal por volume e as travas de custo: `CUSTOS-OPERACIONAIS.md`.

Serviço	Uso	Configuração	Sem configuração
Zenvia SMS	OTP do agendamento, do "Meus agendamentos" e do DSAR. Só para **celular**. Quotas: 3/h por telefone, 10/h por IP, 60/h global (alerta no log a 80%)	`ZENVIA_API_TOKEN`, `ZENVIA_FROM` (+ `SMS_MAX_*`, `ZENVIA_API_URL`, webhook)	Falha fechada: sem agendamento online; CTAs "Agendar" viram WhatsApp (`AGENDAR_WHATSAPP`)
WhatsApp Cloud API (Meta)	Templates aprovados: `confirmacao_d1` (7 parâmetros: nome, data, hora, procedimento, profissional, link confirmar, link cancelar), `nps_pos_atendimento` (nome, procedimento, link), `lista_espera_vaga` (nome, procedimento, data/hora, link). Categoria UTILITY; links no corpo. Recebe a nota do NPS e o pedido de parar (revoga lembrete e pesquisa por WhatsApp, origem WHATSAPP); a confirmação do pedido é texto livre, dentro da janela de 24 h que a resposta da cliente abriu	`WHATSAPP_TOKEN`, `WHATSAPP_PHONE_ID`, `WHATSAPP_VERIFY_TOKEN` (32+ caracteres, E003), `WHATSAPP_APP_SECRET`, `WHATSAPP_TEMPLATE_*`, `WHATSAPP_API_VERSION` (padrão v23.0)	Nada é enviado nem marcado como enviado; o lembrete D-1 vai por e-mail
E-mail (backend Django)	Transacional (aprovação/rejeição, cancelamento, lembrete D-1 de quem não autorizou o WhatsApp, termos, fila de espera, pacote expirando, aprovação ao profissional, reset/convite de senha, contato) e marketing (aniversário, promoções) com `List-Unsubscribe` (RFC 8058) e HTML de promoção sanitizado (bleach)	`EMAIL_BACKEND` + credenciais, `DEFAULT_FROM_EMAIL`	Prod usa backend *dummy*: nada sai; reset de senha/convites recusam (aviso W003)
Botão WhatsApp (`wa.me`)	Contato, fallback do agendamento, link do termo pela recepção	`WHATSAPP_NUMERO` (tela Branding > env)	Botões escondidos (aviso W004)
Web Push (VAPID)	À profissional: novo agendamento, cliente cancelou/remarcou e o resumo diário de atendimentos sem desfecho	`WEBPUSH_VAPID_*` (contato do claim: `WEBPUSH_VAPID_CLAIMS_EMAIL` > `ADMIN_EMAIL` > e-mail da marca)	Botão de ativar escondido
Feed ICS (sem integração)	A profissional assina a própria agenda (Google, Apple, Outlook) pelo link `link_acesso` FEED_ICS — único caminho da agenda para fora (o Google Agenda por OAuth saiu na 0071)	—	—
Google Maps (só link)	"Abrir no Google Maps" na página de contato, com o endereço da Identidade visual; a página mostra uma ilustração local e nada do Google carrega até a pessoa clicar (a CSP não tem `frame-src` do Google)	`CLINIC_ADDRESS` (Identidade visual > env)	Sem endereço, o bloco do mapa some
Cloudflare Turnstile	Captcha no pedido de OTP (wizard, portal e "Meus dados"), no contato e na lista de espera (`utils/captcha.captcha_invalido`)	`TURNSTILE_SITE_KEY`, `TURNSTILE_SECRET_KEY`	Sem captcha (aviso W009 se o SMS for real); os formulários públicos ficam com honeypot e limites
Sentry	Erros e transações amostradas; `send_default_pii=False`, sem	`SENTRY_DSN`, `SENTRY_TRACES_SAMPLE_RATE`	Desligado

	variáveis locais nem corpo do request, `EventScrubber` recursivo (contato e dado de saúde) e filtros em erro, transação e breadcrumb que redigem tokens de link, e-mail/CPF/telefone e o DETAIL do Postgres (`utils/pii.opcoes_sentry`)	(ambiente = `SENTRY_ENVIRONMENT` > `RAILWAY_ENVIRONMENT_NAME` > `DJANGO_ENV`)	

Mídia: o sistema não recebe upload (nenhum `FileField`/`ImageField`; sem S3/`AWS_*`). Upload futuro = decisão antes (bucket privado, URL assinada).

9. Rotinas automáticas (jobs)

Os jobs são funções (`jobs.py`, `jobs_manutencao.py`; sem Celery). **Fonte única dos horários: jobs.JOB_AGENDA** (horário de Brasília). Três portas, todas por `jobs.executar` (o job roda uma vez, sem retry na hora, dentro do Ator dele):

- **Serviço Cron do Railway** (o caminho de produção, §13.2): `railway.cron.json` agenda `*/15 * * * *` e roda `python manage.py rodar_jobs_devidos`, que executa os jobs cujo horário da `JOB_AGENDA` caiu nos últimos 15 min — um serviço só, no lugar de um por horário. Um job que falha não impede os outros; no fim, exit 1 se algum falhou (o Railway marca a execução como falha).
- **Cron HTTP**: `POST /cron/run/<job>/` com o header `X-Cron-Token: $CRON_TOKEN` (nunca `?token=`), comparado em tempo constante, 30 chamadas/h por IP. Respostas no contrato `utils/respostas`: `200 {"ok": true, "job", "resultado"}`; `403` token; `404` job desconhecido (lista os disponíveis); `409 job_ja_executado` (job de mensagem que já rodou hoje); `429` acima do limite; `500 job_falhou` com **só o tipo da exceção e um ref** (o mesmo do log — o agendador guarda o corpo, e a mensagem poderia ter dado pessoal). O job roda **dentro do request** e o gunicorn corta em 120 s.
- **Linha de comando**: `python manage.py rodar_job <job>` (`--lista`; erro = exit 1; `--forcar` roda de novo no mesmo dia, depois de uma falha — pode reenviar).

Nunca mandar duas vezes: lembrete e NPS têm marca por destinatário (`Notificacao`); os jobs de mensagem sem essa marca (`aniversario`, `pacote_expirando`, `nps_24h` — `jobs.JOBS_UMA_VEZ_POR_DIA`) rodam no máximo uma vez por dia local: a linha `execucao_job` (UNIQUE job + dia) nasce antes do job, e a 2ª chamada recebe 409 (no comando: aviso, sem rodar). Cada execução avisa o monitor (`CRON_MONITOR_URL`: `<url>/<job>` ou `<url>/<job>/fail`).

```
curl -fsS -X POST -H "X-Cron-Token: $CRON_TOKEN" https://<dominio>/cron/run/<job>/
```

Job (`<job>`)	Horário BRT (`JOB_AGENDA`)	Cron (America/Sao_Paulo)	O que faz
`emails`	a cada 15 min (:00, :15, :30, :45)	`*/15 * * * *`	Reenvia a fila de e-mail (`Notificacao` EMAIL PENDENTE vencida; até 5 tentativas / 48 h)
`limpeza_status`	de hora em hora (:00)	`0 * * * *`	PENDENTE a menos de 1 h do início → CANCELADO (FSM) + e-mail à cliente ("não conseguimos confirmar a tempo"), vaga para a lista de espera. Não toca nos sem desfecho (ver `sem_desfecho`)
`pacote_expirar`	00:30	`30 0 * * *`	Compras ATIVO com `data_expiracao` < hoje → EXPIRADO (`sp_expirar_pacotes`)
`pacote_expirando`	07:00	`0 7 * * *`	E-mail a quem tem pacote vencendo em 7 ou 1 dia com saldo (1×/dia)
`lembrete_diario`	08:00 e 18:00	`0 8,18 * * *`	Lembrete D-1 dos atendimentos **AGENDADO** de amanhã: WhatsApp (template `confirmacao_d1`) a quem tem LEMBRETE_AGENDAMENTO/WHATSAPP vigente, telefone e o canal configurado; senão **e-mail** (fila, com os mesmos links de confirmar/cancelar). A rodada das 18:00 pega quem marcou depois das 8h; idempotente por atendimento (WhatsApp que falhou é tentado de novo)
`sem_desfecho`	08:00	`0 8 * * *`	Push a cada profissional com "N atendimentos aguardam desfecho" (AGENDADO/CONFIRMADO que já terminaram, sem Realizado/Faltou/Cancelado); o sistema nunca marca falta sozinho
`aniversario`	09:00	`0 9 * * *`	E-mail de felicitação (sem desconto) com consentimento de marketing; 29/02 comemora em

			28/02 (1×/dia)
nps_24h	10:00	`0 10 * * *`	WhatsApp de NPS para REALIZADO entre 24 h e 7 dias, **nunca retorno**, no máximo 1 pesquisa por cliente a cada 90 dias (`NPS_INTERVALO_DIAS`), com consentimento; até 3 tentativas (1×/dia)
detrator_alerta	10:30	`30 10 * * *`	E-mail de NPS ≤ 6 para `email_admin` > `ADMIN_EMAIL` > `CLINIC_EMAIL`
lgpd_purgar	dom 03:00	`0 3 * * 0`	Pseudonimização por retenção (confirmada cliente a cliente; saúde na janela de 20 anos e pacote a menos de 5 anos do fim retêm), fichas de pedidos não realizados, lista de espera vencida (`sp_aplicar_retencao_lgpd`)
housekeeping	04:00	`0 4 * * *`	Sessões, OTP > 24h, texto de notificação > 12 meses, auditoria e trilha `historico_alteracao` > 5 anos (a do prontuário sai com a saúde), link público vencido/revogado > 90 dias, **registro de saúde 20 anos depois do último evento** (etapa `saude`) — `sp_purgar_dados_expirados` —, axes > 90 dias (etapas independentes; falha de uma → 500 no fim)
feriados	dia 1, 04:30	`30 4 1 * *`	Feriados nacionais do ano atual e do seguinte (`services/feriados.carregar`)

O que um job muda nas tabelas com trilha sai em `historico_alteracao` com origem `cron:<job>` (HTTP) ou `comando:<job>` (serviço Cron e `rodar_job`).

O Brasil está sem horário de verão desde 2019: BRT = UTC−3 o ano todo. O disparo de promoção não é job: sai pelo painel (Promoções > Disparar), em lotes, uma vez (`REGRAS-DE-NEGOCIO.md` §19).

10. API (OpenAPI 3.1)

O sistema não expõe uma API REST pública: as telas são renderizadas no servidor. As rotas de máquina — chamadas pelo JavaScript do wizard, do portal da cliente e do painel, pelos provedores (Meta e Zenvia), pelo agendador dos jobs e pelos calendários externos — estão no contrato aranha_estetica/api/openapi.yaml (OpenAPI 3.1.0, 27 caminhos), navegável no Swagger UI do painel (/painel/api/, só ADMIN). Um teste reprova rota JSON sem documentação.

10.1 Contrato das respostas

Caso	Corpo JSON	Uso
Sucesso	{"ok": true, ...dados}	o front segue com os dados
Erro	{"ok": false, "codigo", "mensagem", ...extra}	o front decide pelo código (estável) e mostra a mensagem
Fora do envelope	formato do provedor / FullCalendar / text/calendar	webhooks, saúde, eventos do calendário, feed ICS

10.2 Autenticação

Esquema	Onde	Descrição
sessaoEquipe	cookie: sessionid	Sessão da equipe (papel ADMIN) após `/admin-login/` e, quando exigido, o desafio 2FA (TOTP).
sessaoProfissional	cookie: sessionid	Sessão de usuária com papel PROFISSIONAL (portal "Minha agenda").
sessaoCliente	cookie: sessionid	Sessão do portal da cliente aberta pelo código SMS (1 h, presa ao telefone do cadastro).
csrf	header: X-CSRFToken	Token CSRF do Django, lido do formulário da própria página (o cookie `csrftoken` é HttpOnly).
cronToken	header: X-Cron-Token	Segredo `CRON_TOKEN`. Nunca na URL (vazaria em log).
assinaturaMeta	header: X-Hub-Signature-256	`sha256=<HMAC-SHA256 do corpo com WHATSAPP_APP_SECRET>`. Sem o segredo configurado, toda requisição é recusada.
segredoZenvia	header: X-Zenvia-Signature	Segredo estático `ZENVIA_WEBHOOK_SECRET` (o corpo não é assinado).
tokenFeed	query: token	Token do link de calendário gerado no portal da profissional ou no painel (revogável).

10.3 Operações

Grupo	Operação	Acesso	Descrição
Agendamento online	GET /ajax/dias-disponiveis/	Pública	Dias do mês com expediente para o procedimento
Agendamento online	GET /ajax/horarios-disponiveis/	Pública	Horários livres de um dia, agrupados por horário
Agendamento online	POST /agendamento/otp/solicitar/	Pública	Envia o código de 6 dígitos por SMS
Agendamento online	POST /agendamento/otp/verificar/	Pública	Confere o código e devolve o pré-preenchimento do cadastro
Portal da cliente	POST /meus-agendamentos/otp/enviar/	Pública	Envia o código de login do portal
Portal da cliente	POST /meus-agendamentos/otp/verificar/	Pública	Confere o código e abre a sessão do portal
Portal da cliente	GET /meus-agendamentos/{pk}/horarios/	Cliente (sessão SMS)	Horários livres para reagendar um atendimento

Portal da cliente	POST /ajax/cancelar-agendamento/	Cliente (sessão SMS)	Cancela um agendamento pela cliente
Portal da cliente	POST /meus-agendamentos/meus-dados/	Cliente (sessão SMS)	"Baixar meus dados" pela sessão do portal
LGPD	POST /lgpd/meus-dados/	Pública	"Meus dados" do site: pede o código e baixa o export
Painel da equipe	POST /painel/atualizar-status/	Equipe (ADMIN)	Muda o status de um atendimento (select das listas)
Painel da equipe	GET /painel/calendario/eventos/	Equipe (ADMIN)	Eventos do calendário (formato FullCalendar)
Painel da equipe	POST /painel/calendario/mover/	Equipe (ADMIN)	Reagenda pelo arrastar e soltar do calendário
Painel da equipe	POST /painel/anotacao/{atendimento_id}/salvar/	Equipe (ADMIN) ou Profissional	Adiciona anotação clínica a um atendimento
Web Push	GET /webpush/public-key/	Pública	Chave pública VAPID
Web Push	POST /webpush/subscribe/	Equipe (ADMIN)	Registra (ou reativa) a assinatura do navegador
Web Push	POST /webpush/unsubscribe/	Equipe (ADMIN)	Remove a assinatura do navegador
Agenda externa	GET /agenda/{slug}/feed.ics	Token do link	Agenda da profissional como calendário (RFC 5545)
Integrações	GET /api/whatsapp/webhook/	Pública	Verificação do webhook pela Meta
Integrações	POST /api/whatsapp/webhook/	Assinatura HMAC da Meta	Mensagens recebidas no WhatsApp (Meta Cloud API)
Integrações	POST /api/zenvia/webhook/	Segredo da Zenvia	Status de entrega do SMS (Zenvia)
Jobs	POST /cron/run/{job_name}/	X-Cron-Token	Executa um job periódico
Saúde	GET /health/	Pública	Readiness — banco, cache e bundle do front
Saúde	GET /healthz/	Pública	Liveness — processo vivo (healthcheck do Railway)
PWA	GET /manifest.json	Pública	Manifest do site (app da clínica)
PWA	GET /painel/manifest.json	Pública	Manifest do app "Painel" (escopo /painel/)
PWA	GET /profissional/manifest.json	Pública	Manifest do app "Minha agenda" (escopo /profissional/)
Documentação	GET /painel/api/openapi.yaml	Equipe (ADMIN)	Esta especificação (OpenAPI 3.1, YAML)

11. Rotas

Namespace `aranha` (`aranha_estetica/urls/`) + `clinica/urls.py`. Rotas removidas nesta versão: `/account/*` (django-two-factor), `/api/schema/redoc/`, `/servicos/produtos/`, `/ajax/verificar-telefone/`, `/ajax/buscar-procedimentos/`, `/ajax/buscar-horarios/`, `/painel/cancelar-agendamento/`, `/reagendar/<token>/` (link legado; a cliente reagenda só pelo portal), `/lgpd/aceitar-cookies/` (o aviso de cookies não grava nada no servidor) e `/painel/integrations/google/*` (Google Agenda).

11.1 Site público

Rota	Função
`/`, `/quem-somos/`, `/equipe/`, `/especialidades/`, `/galeria/`, `/depoimentos/`, `/promocoes/`	Páginas institucionais; números do site só com dado real (`services/prova_social`); com uma profissional ativa, `/equipe/` vira "Sobre a profissional" (foto ou monograma, especialidade, bio e os tratamentos que ela realiza, do catálogo); depoimentos só com opt-in + aprovação; promoções com o mesmo preço do agendamento
`/duvidas/`	Perguntas frequentes por categoria (`conteudo/faq.py`, respostas geradas das constantes que o sistema aplica; JSON-LD `FAQPage`); a home mostra só as 5 em destaque e leva para cá. No menu, no rodapé, no sitemap e no cache do SW
`/servicos/faciais/`, `/servicos/corporais/`, `/servicos/detalhe/<slug>/`	Catálogo (cartões dos procedimentos ativos por categoria, com o "Agendar" já no procedimento; "Cuidados em destaque" saem do catálogo — até 4, só ativos com descrição, um por família)
`/contato/`	Formulário de contato (`views/publico/contato`, `forms/publico.ContatoForm`): vai ao e-mail de contato da Identidade visual (`CLINIC_EMAIL`; sem ele, o `DEFAULT_FROM_EMAIL` se não for "noreply"), com o e-mail da pessoa no reply-to; aceite da política obrigatório e **sem caixa de marketing**; captcha (Turnstile, quando configurado), honeypot, 5/min e 20/dia por IP e teto de 50 mensagens/dia no site todo (acima dele, ou com o e-mail fora do ar, a página mantém o texto e oferece o WhatsApp). Mapa = ilustração local + link "Abrir no Google Maps" (nada do Google carrega na página)
`/termos-de-uso/`, `/politica-de-privacidade/`	Textos legais; a política é o texto da **VersaoTermo LGPD vigente** (filtro `texto_termo`, sanitizado), com versão e vigência, emoldurado pela identificação da clínica e o encarregado (Identidade visual); sem versão ativa, o texto padrão da v1.0
`/lista-espera/` (+ `/sucesso/`)	Pedido de aviso de vaga **sem criar cadastro** (`forms/publico.ListaEsperaForm` + `services/lista_espera.inscrever`): `?procedimento=&profissional=&data=` preenche (o "Avise-me se vagar" do wizard); captcha, honeypot e 20/dia por IP; sucesso igual em todos os casos
`/sitemap.xml`, `/robots.txt`, `/manifest.json`, `/sw.js`, `/favicon.ico`	SEO/PWA

11.2 Agendamento e auto-atendimento da cliente

Rota	Função
`/agendamento/` · `/agendar/<slug>/` · `/embed/agendar/`	Wizard (geral, direto por profissional, widget para iframe — `EMBED_FRAME_ANCESTORS`)
`/ajax/dias-disponiveis/` · `/ajax/horarios-disponiveis/`	Dias do mês e horários livres (SlotService)
`POST /agendamento/otp/solicitar/` · `POST /agendamento/otp/verificar/`	OTP do celular (Turnstile quando configurado)

`POST /agendamento/confirmar/` · `/agendamento/sucesso/`	Grava o atendimento (PENDENTE)
`/meus-agendamentos/` (+ `otp/enviar/`, `otp/verificar/`, `POST sair/`)	Área da cliente (login por celular ou e-mail; código sempre ao celular; a sessão também abre no fim do wizard). Sair só por POST com CSRF
`POST /meus-agendamentos/preferencias/` · `POST saude/revogar/` · `POST pedido/` · `POST meus-dados/`	Minhas preferências (3 pares, origem PORTAL; 20/h por IP) · revogar o consentimento de saúde (10/h) · pedido de correção ou exclusão (`tipo` + descrição; trilha + e-mail à clínica; 5/h) · baixar meus dados **sem a ficha de saúde** (exige código de até 10 min; 10/h)
`/meus-agendamentos/<id>/reagendar/` (+ `<id>/horarios/`)	Reagendar pelo portal: id + sessão do OTP (só agendamento do telefone da sessão; até 24h antes) — o único caminho da cliente
`POST /ajax/cancelar-agendamento/`	Cancelar pelo portal: `{"atendimento_id"}` + sessão do OTP (`agenda.cancelar_pela_cliente`); resposta `{ok, mensagem, tardio}` ou `{ok: false, codigo}` (`dados_invalidos`, `sem_atendimento`, `nao_encontrado` 404, `passado`, `nao_cancelavel`) — com menos de 24 h de horário aprovado é cancelamento tardio e conta falta
`/confirmar/<token>/`	Confirmar presença / cancelar a partir do lembrete D-1 (WhatsApp ou e-mail); cancelar com menos de 24 h é cancelamento tardio (a página avisa antes)
`/termo/<token>/`	Aceitar termo(s) do atendimento (vale até o fim do atendimento)
`/nps/<token>/` (+ `obrigado/`)	NPS (link de 7 dias) + opt-in de depoimento; a página de obrigado oferece a pesquisa pós-atendimento, se houver
`/anamnese/<token>/` · `/pesquisa/<token>/` (+ `/anamnese/obrigado/`, `/pesquisa/obrigado/`)	Ficha/pesquisa por link (60 dias; ficha de saúde exige consentimento art. 11). Produtores: "Link da ficha" na agenda do painel e convite da pesquisa no fim do NPS (D18)
`/lgpd/meus-dados/` · `/lgpd/unsubscribe/<token>/`	"Meus dados": telefone → código (propósito DSAR, captcha, 10/h por IP e 5/h por telefone, latência mínima) → JSON **sem a ficha de saúde**; descadastro
`/agenda/<slug>/feed.ics?token=`	Feed iCal do profissional (`views/infra/feed_ics`; link gerado no painel ou no portal do profissional; gerar = revogar o anterior; a URL só aparece na geração)

11.3 Equipe

Rota	Função
`/admin-login/` · `/admin-logout/` (logout só via POST; GET mostra confirmação) · `/admin-login/recuperar/...`	Login da equipe e reset de senha
`/painel/seguranca/2fa/` (+ `verificar/`, `challenge/`)	Cadastro e desafio TOTP
`/profissional/` · `/profissional/atendimento/<pk>/{realizado,anotar,aprovar,rejeitar}/` · `POST /profissional/calendario/novo-link/` · `/profissional/comissoes/`	Portal do profissional (+ gerar o próprio link do feed ICS; "Minhas comissões" do mês, só leitura)
`/painel/`, `/painel/overview/`	Dashboard
`/painel/agendamentos/` (+ `novo/`, `bulk/`, `<pk>/aprovar/`, `<pk>/rejeitar/`, `<pk>/termo-link/`, `<pk>/ficha-link/`, `<pk>/valor/`, `<pk>/desfazer-realizacao/`), `/painel/atualizar-status/`	Agenda em lista (marcadores em lote, "Agendar retorno" quando devido), agendamento interno (`novo/`: uma tela, POST com `acao` buscar/horarios/agendar → `agenda.agendar_pela_recepcao`; 60/min), aprovação um a um e em lote (item a item), links do termo e da ficha, valor cobrado, desfazer realização indevida. As escritas usam `@escrita_painel` (POST + 30/min por usuário)
`POST /painel/atualizar-status/`	Seletor de status (JSON `{atendimento_id, status, sem_termo?}`, contrato `utils/respostas`): `200 {ok, status_anterior, status_novo}`; `400 dados_invalidos`/`atendimento_invalido`/`status_i`

	nvalido, transição recusada (transicao_invalida, status_nao_suportado...); 404 nao_encontrado; **409 termo_pendente** (com termos; o front pede a confirmação e reenvia com sem_termo: true, que fica na trilha); **409 desfecho_antes_do_inicio** (Realizado/Faltou antes do início); 500 erro_interno. PENDENTE vai para o AgendamentoService (aprovar/recusar + e-mail)
/painel/retornos/, /painel/agendamentos/<pk>/retorno/	Retornos a marcar (só horários livres dentro da janela)
/painel/calendario/ (+ eventos/, mover/)	Calendário FullCalendar com arrastar-e-soltar. eventos/ = lista crua do FullCalendar (range inválido → []). POST mover/ ({id, start, end} ISO, agenda.mover): 200 {ok, nova_data}; 400 dados_invalidos/fim_antes_do_inicio e status que não se move (transicao_invalida); 404 nao_encontrado; 409 recusa de horário com o motivo (conflito, bloqueio, folga, feriado, expediente — ex.: horario_ocupado)
/painel/clientes/, /painel/clientes/<pk>/	Clientes e ficha do cliente (pacotes, alertas, histórico)
/painel/prontuario/ (+ <cliente_id>/, <cliente_id>/salvar/), /painel/anotacao/<id>/salvar/	Prontuário com histórico de versões e anotações
/painel/profissionais/ (+ <pk>/ics/novo-token/, <prof_id>/excecoes/...), /painel/cadastrar-profissional/, /painel/editar-profissional/<pk>/	Profissionais, grade semanal, exceções, gerar link do ICS (a lista mostra só "ativo desde"; a URL aparece uma vez, na geração). Desligamento — profissional desativada, login dela desativado ou vínculo do login trocado — revoga o feed ICS (auditado); reativar exige gerar um link novo
/painel/procedimentos/..., /painel/pacotes/... (+ vender/, compras/<pk>/cancelar/, compras/<pk>/prorrogar/), /painel/promocoes/... (+ <pk>/disparar/)	Catálogo, pacotes (venda, cancelamento com reembolso, prorrogação da validade), promoções (disparo em lotes; reenvio só confirmado)
/painel/bloqueios/..., /painel/lista-espera/ (+ <pk>/notificar/)	Bloqueios de agenda e lista de espera (inscrições anônimas com nome e celular de contato; "Avisar" manda e-mail quando há destino e registra o aviso mesmo sem e-mail, auditado)
/painel/financeiro/?mes=AAAA-MM, /painel/exportar-relatorio/?mes=AAAA-MM	Financeiro do mês (competência; padrão o mês corrente) e o Excel gerencial do mesmo mês (exportação auditada com o mês)
/painel/comissoes/ (+ <pk>/pagar/, repasse/, repasse/<pk>/, regras/, regras/criar/, regras/<pk>/ativo/, regras/<pk>/excluir/)	Comissões: baixa de um crédito, repasse em lote ("pagar pendentes até") e o recibo para imprimir (sem nome de cliente), regras de comissão (usada só desativa)
/painel/nps/ (+ <pk>/publicacao/)	NPS e moderação de depoimentos
/painel/anamneses/..., /painel/termos/ (+ criar/, compliance/)	Formulários e termos
/painel/notificacoes/, /painel/auditoria/, /painel/usuarios/..., /painel/configuracoes/ (+ criar/, <pk>/editar/, <pk>/excluir/ = **restaurar padrão**), /painel/branding/, /painel/email-preview/[<nome>/]	Sistema: notificações, auditoria, usuários (ninguém muda o próprio papel; o último ADMIN ativo não é rebaixado nem desativado), **Regras operacionais** (só o registro tipado), **Identidade visual** (marca e contatos). As escritas de usuários, profissionais, procedimentos e regras usam @escrita_painel (POST + 30/min por usuário)
/webpush/{public-key,subscribe,unsubscribe}/	Web Push da equipe
/painel/api/ (+ openapi.yaml)	**Documentação da API**: Swagger UI (swagger-ui-dist do npm, tema claro/escuro do painel) do api/openapi.yaml; "Try it out" só em GET (as escritas mudam dados reais), sem validador externo. Só ADMIN (e 2FA); o YAML também baixa daqui
/django-admin-sv/	Django admin (só ADMIN com sessão 2FA verificada)

11.4 Webhooks e infraestrutura

Rota	Função
`GET/POST /api/whatsapp/webhook/`	Meta: handshake `hub.challenge` (`WHATSAPP_VERIFY_TOKEN`, tempo constante) e eventos assinados (`X-Hub-Signature-256` com `WHATSAPP_APP_SECRET`; sem o segredo recusa tudo); captura a nota de NPS 0–10 e o pedido de parar ("PARAR", "SAIR", "STOP", "cancelar mensagens", "descadastrar" — a mensagem inteira)
`POST /api/zenvia/webhook/`	Status de entrega de SMS: `ZENVIA_ALLOWED_IPS` (IP/CIDR) e/ou segredo compartilhado no header `X-Zenvia-Signature` (`ZENVIA_WEBHOOK_SECRET`, o valor configurado na assinatura do webhook na Zenvia; o corpo não é assinado), comparado em tempo constante; prod sem nenhum dos dois responde 503
`POST /cron/run/<job>/`	Jobs periódicos (§10), header `X-Cron-Token`, 30 chamadas/h por IP
`/healthz/`	Liveness (processo vivo + manifest do Vite); healthcheck do Railway
`/health/`	Readiness: banco + cache + manifest (503 se algo falha)

12. Testes e qualidade

- `python manage.py test aranha_estetica` — suíte Django (SQLite `.cache/db_test.sqlite3`, ~2.020 testes fora a tag `migracoes`, em 125 arquivos, alguns em pastas por área — `tests/conta/`, `infra/`, `jobs/`, `painel/`, `publico/`, `servicos/`, `utils/` — roda **tudo**, inclusive as data migrations). `manage.py test` sempre carrega `clinica/settings/test.py` (única detecção de teste, em `settings/__init__.py`; é o runner único — pytest/pytest-django/conftest saíram, e `test_deploy_config.RunnerUnicoTests` reprova a volta): hash de senha MD5, SMS em modo log, avisos `aranha.W001`–`W016` silenciados (testados à parte em `test_deploy_config.py` e `tests/infra/test_deploy.py`), django-vite em `dev_mode` (não precisa do build; `TESTE_VITE_BUILD=1` usa o bundle).
- **Atalho do dia a dia**: `python manage.py test aranha_estetica --exclude-tag migracoes` (~40 s) pula `tests/test_migrations_dados.py` (tag `migracoes`: desce e sobe a cadeia de migrations dezenas de vezes); `--tag migracoes` roda só ele. `--keepdb` reaproveita o banco de teste em arquivo (um módulo: ~15 s → ~2 s); `--parallel` no Windows/macOS exige `TESTE_SQLITE_MEMORIA=1`. Antes do merge, rode tudo (o CI roda as duas partes em jobs paralelos).
- **Navegador** (Playwright + Chromium; local com `npm run build` + `TESTE_VITE_BUILD=1`, no CI com `EXIGE_NAVEGADOR=1` — faltou navegador ou bundle = reprova, não pula): `tests/test_wizard_browser.py` cobre as regras que só existem no `wizard.js`; `tests/painel/test_status_navegador.py`, o seletor de status, a barra do lote e o FullCalendar do bundle; `tests/test_crawl_visual.py` (varredura visual) abre **toda** rota pública, da equipe e da profissional de `tests/test_rotas.ROTAS` no papel certo, em 1366px (tema claro) e 390px (tema escuro), e reprova em erro HTTP (da página ou de recurso do site), erro de JS/console (inclui violação de CSP), rolagem lateral no celular, sintaxe de template vazando no texto e violação serious/critical do axe-core (WCAG 2.0/2.1 A e AA; `axe-core` é devDependency, `npm ci`). Exceção de terceiro só pela lista `PERMITIDOS` do teste, com o motivo (hoje vazia); `VARREDURA_ROTAS=<trecho>` filtra as rotas para depurar uma tela.
- **Contrato da API** (`tests/test_openapi.py`): o `api/openapi.yaml` é OpenAPI 3.1 válido (`openapi-spec-validator`), cada caminho resolve para o `x-url-name` declarado, toda view que responde JSON/ICS/export LGPD (detectada pelo código da view e dos helpers do módulo) está documentada, o enum de jobs é o `JOB_MAP` e só a equipe abre `/painel/api/`.
- **Cobertura**: modo paralelo (um arquivo `.cache/coverage.*` por execução): `coverage run manage.py test ... && coverage combine && coverage report`; no CI o job `cobertura` junta SQLite + Postgres (o código só-Postgres — procedures, paridades, LGPD — conta como testado) e aplica a catraca `fail_under` do `pyproject.toml` (86,9 %, uma casa; só sobe) no total.
- **Lint**: `ruff check .` com lista fixa (F, E4/E7/E9, **DTZ** — `date.today()`/`datetime.now()` sem fuso erram o dia entre 21h e 24h; use `utils/datas` — e **TID251**: `os.environ`/`os.getenv` só em `clinica/settings`, nos pontos de entrada e nas exceções listadas no `pyproject.toml`); `pre-commit install` liga gitleaks (segredo no que está sendo commitado), ruff, `lint-imports` e `makemigrations --check` antes do commit.
- **Postgres local** (`docker-compose.yml`, PG 18 com ICU): `docker compose up -d db` e `TEST_DATABASE_URL=postgres://shivazen:shivazen@localhost:5432/shivazen PG_EXIGE_ICU=1 python manage.py test aranha_estetica`. No SQLite os testes do esquecimento, da retenção, do housekeeping e da expiração de pacotes são **pulados** (procedures sem fallback Python — §9.4); o CI roda tudo no Postgres.
- `TEST_DATABASE_URL=postgres://... python manage.py test aranha_estetica` — mesma suíte no Postgres; `tests/test_pg_ddl.py` confere os objetos só-Postgres (EXCLUDE, CHECK regex, triggers, funções, views, procedures, extensões, collation — com `PG_EXIGE_ICU=1` a collation tem de ser ICU), `tests/test_nomes_padrao.py` lê o `pg_catalog` e reprova objeto fora do padrão SQL, FK sem índice, `ON DELETE` divergente do model, tabela sem `COMMENT`, tabela com `updated_at` sem `tg_<t>_updated_at`,

índice `_like` (migration nova que recria FK pelo `AlterField` precisa terminar com `PadronizarNomes()`), `tests/test_pg_historico.py` prova a trilha (autor/IP do request, contexto limpo entre requests na mesma conexão, cron, `rodar_job`, redação da senha, append-only) e `tests/test_pg_paridade.py` compara cada `fn_*`/`vw_*` com o Python (mudou a regra no Python → atualize a função numa migration nova) e confere o efeito de cada `sp_*`.

- Guardas de regressão relevantes: `test_csp.py` (CSP x templates; nenhum script ou CSS de outro host além do Turnstile; libs do painel com versão exata no `package.json`), `test_front_pipeline.py` (Vite/cotton/webpush), `test_componentes.py` (os componentes cotton: nome acessível, classe mesclada, abas, menu, diálogo, KPI, tabela), `test_a11y_front.py` (tokens, tema escuro sem cor fixa, site sem `style=`/`<style>`/hex), `test_site_tags.py` (`{% foto %}` e variantes em dia), `tests/servicos/test_prova_social.py` (mínimos, arredondamento e cache dos números do site), `test_deploy_config.py` (checks, migrate atômico, runner único, pins do CI), `tests/infra/test_deploy.py` (perfis prod/staging, E001–E004, `.env.example` × código, Dockerfile × `railway.json`), `tests/infra/test_logs.py` (redação de token/PII no log), `tests/jobs/` (agenda dos jobs, 409 do 2º disparo, lembretes, NPS), `test_slots_disponibilidade.py` (caracterização do SlotService).

CI (`.github/workflows/ci.yml`, push em `main` e `dev` ou manual; sem gatilho de PR — o PR `dev` → `main` mostra os checks do push do mesmo commit). Commit novo na `dev` cancela a execução anterior; na `main` toda execução termina. `permissions: contents: read` e `timeout-minutes` em todos os jobs; actions fixadas por SHA e imagens (`postgres:18`, `python:3.12-slim`, `node:22-slim`, também no `Dockerfile` e no `docker-compose.yml`) por digest — o Dependabot (`.github/dependabot.yml`: pip, npm, actions, docker, compose; PRs semanais na `dev`) atualiza o pin; `tests/test_deploy_config.CiFixadoTests` reprova pin solto:

Job	Passos
testes	`check`, `makemigrations --check`, `ruff check` (erros, fuso e `TID251`), `lint-imports` (camadas e áreas de views), `mypy` (services), suíte Django (SQLite, `--exclude-tag migracoes`) sob `coverage` (o arquivo vai para o job `cobertura`), `check --deploy --fail-level ERROR` com settings de prod e de staging
migracoes-sqlite	`tests/test_migrations_dados.py` (`--tag migracoes --parallel auto`) no SQLite
postgres	Postgres 18: `migrate_atomico` do zero, `bootstrap_admin`, suíte Django no Postgres sem a tag `migracoes` sob `coverage` (bloqueante; inclui conformidade, trilha e paridade)
cobertura	Depois de `testes` e `postgres`: `coverage combine` dos dois e `coverage report` com a catraca (`fail_under`) no total
postgres-migracoes	`tests/test_migrations_dados.py` no Postgres 18 (reverso/ida de cada migration, inclusive triggers/funções/views/procedures), `--parallel auto` (um `test_ci_N` por worker)
front	`npm ci`, `npm run build`, confere o manifest
navegador	`npm ci` + `npm run build` + Playwright/Chromium, com o bundle (`TESTE_VITE_BUILD=1`, `EXIGE_NAVEGADOR=1`): `tests/test_wizard_browser.py` (wizard), `tests/painel/test_status_navegador.py` (seletor de status, lote e FullCalendar do bundle) e `tests/test_crawl_visual.py` (varredura visual de toda rota pública/equipe/profissional em 1366px claro e 390px escuro: HTTP, JS/console/CSP, rolagem lateral, template vazando e axe-core serious/critical)
docker	`docker build` (buildx, camadas no cache do Actions) e o smoke do 1º deploy: Postgres 18 exigindo SSL, o `preDeployCommand` e o `startCommand` **lidos do railway.json**, `/healthz/` com o Host do healthcheck do Railway, `/health/` e gunicorn fora do root
auditoria-deps	`pip-audit` no `requirements.lock`, `npm audit --package-lock-only --audit-level=high` e gitleaks no histórico do push (allowlist dos dados sintéticos em `.gitleaks.toml`) — bloqueante

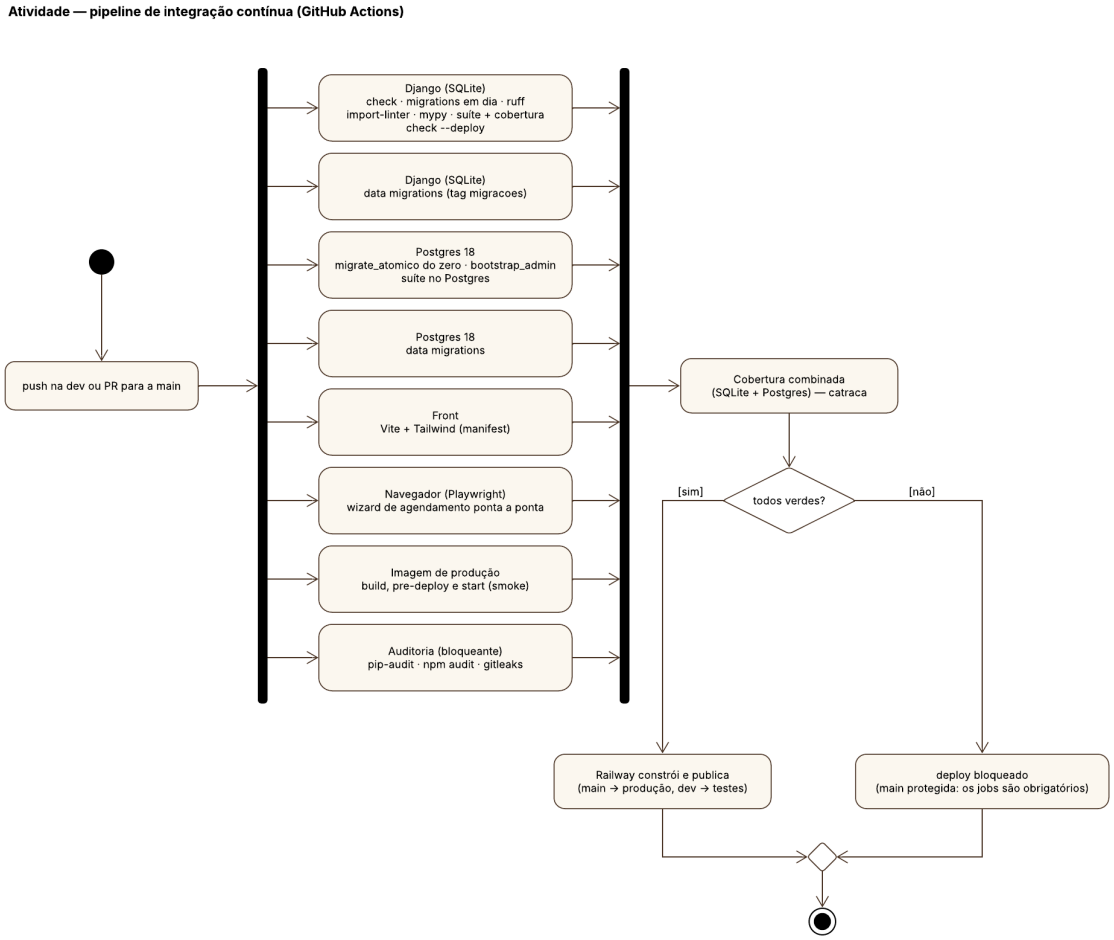


Figura 5 — Atividade — pipeline de integração contínua.

13. Implantação e configuração

- **Build** (`Dockerfile`, builder `DOCKERFILE` no `railway.json`; o Procfile/Nixpacks foi removido):

  1. stage `front` (`node:22-slim`): `npm ci` + `npm run build`; falha se não houver `static/dist/manifest.json`;
  2. stage `builder` (`python:3.12-slim`): wheels de `requirements.txt` com `-c requirements.lock`;
  3. stage `runtime`: só `libpq5` (sem `curl`), usuário de runtime `app` sem shell, código e estáticos copiados como root (só leitura para o processo), `DJANGO_ENV=prod`, smoke do QR SVG do 2FA (a imagem não tem Pillow), `collectstatic` (WhiteNoise `CompressedManifest`) + `check --tag staticfiles --fail-level WARNING` no build, `HEALTHCHECK` em `/healthz/` pelo Python da imagem. O `.dockerignore` deixa fora testes, `scripts/`, `docs/`, bancos, dumps e `.env`.

- **Pre-deploy** (`railway.json`): `python manage.py migrate_atomico --noinput && python manage.py bootstrap_admin`. Se uma migration falha, a transação inteira é desfeita e o deploy anterior continua no ar com o banco intacto.
- **Start**: `gunicorn clinica.wsgi` com `WEB_CONCURRENCY=1` worker × `WEB_THREADS=4`, timeout 120 s, access log com tokens redigidos. O `startCommand` do `railway.json` sobrescreve o `CMD` do Dockerfile (mudar os dois juntos). 1 réplica; restart ON_FAILURE (5×).
- **Healthcheck**: `/healthz/` (timeout 120 s) — 503 se o bundle do Vite faltar.
- **Produção**: `DEBUG` sempre falso, HTTPS + HSTS, e-mail *dummy* sem `EMAIL_BACKEND`, SMS falha fechado sem Zenvia, logs JSON redigidos. Sem worker de fila: jobs pelo serviço Cron (§13.2).
- O deploy de produção sai do branch `main` (configuração do serviço no Railway). O trabalho acontece na `dev`; a `main` é protegida no GitHub (os 6 jobs bloqueantes do CI obrigatórios, inclusive para admin; sem force-push) e recebe a `dev` por PR ou fast-forward.

13.1 Ambiente dev publicado (testes, sai da branch dev)

Primeiro deploy: um ambiente de testes na internet, igual à produção, mas sem risco de mandar mensagem a cliente real. Perfil `DJANGO_ENV=staging` (`clinica/settings/staging.py` = `prod.py` + travas): faixa "Ambiente de testes" no site e no painel, `robots.txt` fechado e `X-Robots-Tag: noindex` em toda resposta, código do SMS só no log (`sms_dev_log`, com a prévia), e-mail e WhatsApp só para `DESTINATARIOS_TESTE` (o `BackendAmbienteTeste` filtra todo e-mail, inclusive o de recuperação de senha) e `seed --demo` liberado. `DJANGO_ENV=dev` no Railway derruba o boot.

Passo a passo (projeto do Railway reativado):

1. **Environment** `dev` no projeto (Settings › Environments) com o serviço web ligado ao repositório, **branch dev** (produção segue da `main`). O `railway.json` já traz o pre-deploy deste environment: `migrate_atomico`, `bootstrap_admin` e `seed --demo` (idempotente).
2. **Postgres 18** no environment: imagem `ghcr.io/railwayapp-templates/postgres-ssl:18` (nunca `:latest`; o check `aranha.W015` avisa outro major).
3. **Variáveis** do serviço web:

   - `DJANGO_ENV=staging`, `DATABASE_URL=${{Postgres.DATABASE_URL}}` (sem ela o check `aranha.E002` barra o pre-deploy — antes caía em SQLite em silêncio);
   - `DJANGO_SECRET_KEY` e `CHAVES_CIFRAGEM` **próprias** do dev (`python manage.py gerar_chave_cifragem`; nunca as de produção);

- `SITE_URL=https://<app>.up.railway.app`, `CSRF_TRUSTED_ORIGINS` igual (o domínio do Railway entra sozinho em `ALLOWED_HOSTS`);
- `ADMIN_EMAIL`, `ADMIN_PASSWORD` (a dona define; trocar a senha no 1º acesso), `CRON_TOKEN`;
- `DESTINATARIOS_TESTE=<e-mail da dona>,<celular da dona>` (só eles recebem mensagem);
- e-mail opcional: `EMAIL_BACKEND=anymail.backends.resend.EmailBackend` + `RESEND_API_KEY` com `DEFAULT_FROM_EMAIL=onboarding@resend.dev` (sem domínio verificado o Resend só entrega na caixa do dono da conta);
- `SENTRY_DSN` opcional (o *environment* sai com o nome do ambiente no Railway, `dev`);
- `DB_APPLICATION_NAME=shivazen-dev`.

4. **Deploy**: push na `dev` → CI verde → o Railway constrói a imagem (o `watchPatterns` ignora commit só de documentação), roda o pre-deploy e troca o container quando `/healthz/` responde. O `seed --demo` só existe no pre-deploy do environment `dev` (`railway.json` › `environments`); o de produção nunca semeia demonstração.
5. **Conferir**: `/healthz/` 200; a faixa amarela no topo; login da equipe (`/admin-login/`) com o 2FA; agendar pelo site com o celular da lista e pegar o código no log (`sms_dev_log` → `preview`); `python manage.py check --deploy --database default` no shell do Railway.
6. **Cron** (opcional no dev): o serviço Cron do §13.2 no environment `dev`, com as variáveis do web deste environment (`DESTINATARIOS_TESTE` inclusive).

13.2 Serviço Cron (jobs)

Um segundo serviço no mesmo projeto, ligado ao mesmo repositório e branch, com o config-as-code **railway.cron.json** (Settings › Config-as-code › caminho do arquivo): mesma imagem do `Dockerfile`, *Cron Schedule* `*/15 * * * *`, comando `python manage.py rodar_jobs_devidos`, 1 réplica, sem restart (cada execução termina sozinha). A agenda do Railway é em UTC, mas o comando compara a `JOB_AGENDA` no fuso da clínica: de 15 em 15 min ele roda o que venceu na janela (§10).

- **Variáveis**: as mesmas do serviço web (o job roda o código direto, não pelo HTTP) — `DATABASE_URL=${{Postgres.DATABASE_URL}}`, `DJANGO_SECRET_KEY`, `CHAVES_CIFRAGEM`, e-mail, WhatsApp, `SITE_URL` (links das mensagens), `CRON_MONITOR_URL`; `DB_APPLICATION_NAME=shivazen-cron`. Não precisa de `CRON_TOKEN` nem de domínio público.
- **Sem pre-deploy**: as migrations são só do serviço web (o `railway.cron.json` não tem `preDeployCommand`).
- **Falha**: exit 1 se algum job falhou (o Railway mostra a execução vermelha); cada job avisa o monitor (`CRON_MONITOR_URL`) com o nome dele.
- **Alternativa sem o serviço**: um agendador externo chamando o cron HTTP (§14 d).

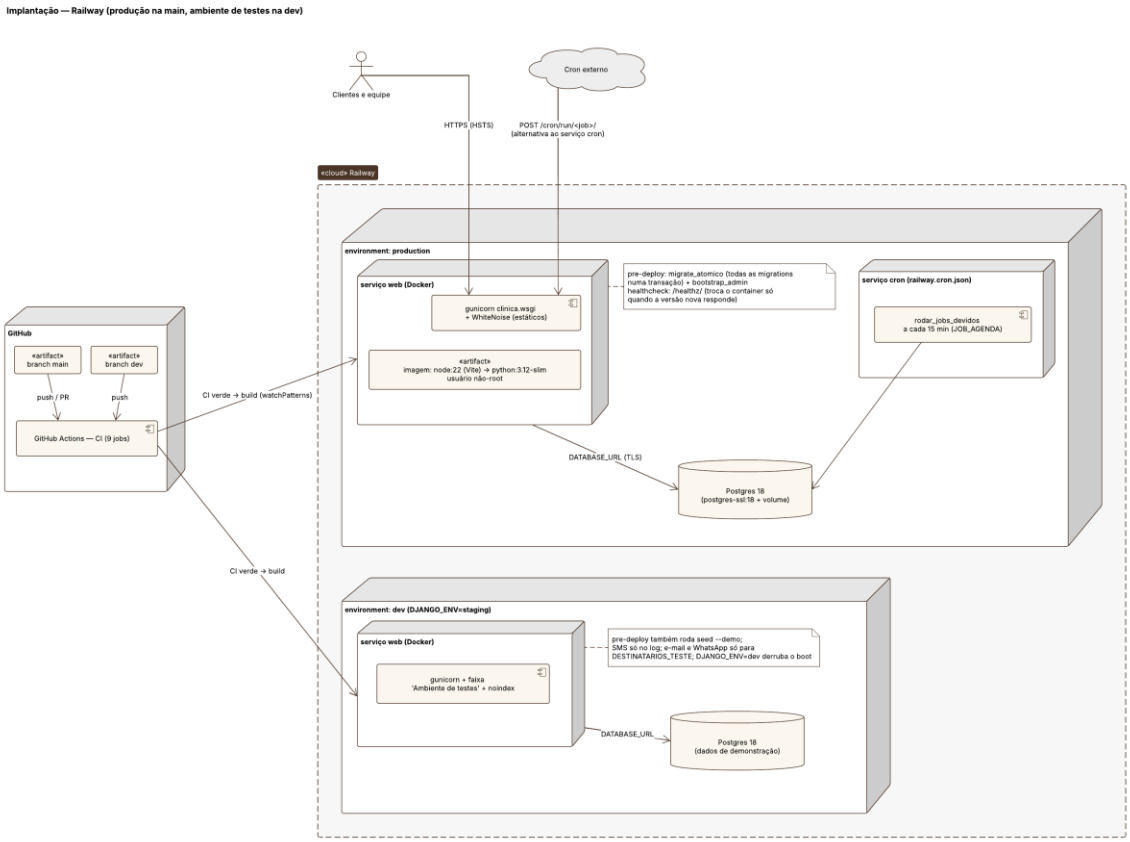


Figura 6 — Implantação — Railway.

13.3 Variáveis de ambiente

Lista completa e comentada em `.env.example` — conferida contra o código por `tests/infra/test_deploy.EnvExampleTests` (toda variável lida está lá e nenhuma sobra). Só os settings (`clinica/settings`) leem a env. Em produção:

13.3.1 Obrigatórias / pré-requisito do go-live

Variável	Propósito
`DJANGO_SECRET_KEY`	Chave do Django (sessões, reset; HMAC do OTP se não houver `OTP_HMAC_KEY`). Sem ela o boot falha fora de DEBUG; publicada, 50+ caracteres aleatórios, nunca valor de exemplo nem `django-insecure-` (E003)
`CHAVES_CIFRAGEM`	Chave(s) Fernet da cifragem em repouso do dado de saúde (`manage.py gerar_chave_cifragem`). **Sem ela o deploy para** (check `aranha.E001` no pre-deploy). Várias, por vírgula: a 1ª cifra, todas decifram. **Cópia offline com a dona**: sem a chave o prontuário não abre
`DJANGO_ENV=prod`	Carrega `settings/prod.py` (a imagem já define; vazio no Railway = `prod`; `staging` = ambiente de testes, §13.1; qualquer valor fora de `dev`/`prod`/`staging`/`test` derruba o boot, e `dev` no Railway também; `manage.py test` sempre usa `settings/test.py`)
`ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS`	Domínio próprio (o `RAILWAY_PUBLIC_DOMAIN` e o host do healthcheck entram sozinhos; em prod só origem `https://` vale no CSRF)
`SITE_URL`	URL pública https sem barra final — links de e-

	mail/WhatsApp/termos/JSON-LD (W001)
`DATABASE_URL`	**Não vem sozinha**: variável de referência `DATABASE_URL=${{Postgres.DATABASE_URL}}` no serviço web e no serviço Cron. Sem ela o check `aranha.E002` barra o pre-deploy (antes caía em SQLite em silêncio)
`ADMIN_EMAIL`, `ADMIN_PASSWORD`, `ADMIN_NOME`	**Pré-requisito do 1º deploy**: a `0042` desativa as contas demo; o `bootstrap_admin` cria o ADMIN real. Apagar `ADMIN_PASSWORD` depois. `ADMIN_EMAIL` também é o destino do alerta de detrator (sem `email_admin` nas Configurações) e o contato do claim VAPID (sem `WEBPUSH_VAPID_CLAIMS_EMAIL`)
`CRON_TOKEN`	Autentica o cron HTTP `/cron/run/<job>/` (W005); 32+ caracteres (E003). O serviço Cron do Railway não precisa dele (roda o comando direto)
`ZENVIA_API_TOKEN`, `ZENVIA_FROM`	SMS/OTP. **Sem SMS não há agendamento online** (CTA vira WhatsApp) (W002)
`EMAIL_BACKEND` + chave do provedor + `DEFAULT_FROM_EMAIL`	Sem ele nada é entregue (W003). **O Railway Hobby bloqueia SMTP**: use o `django-anymail` (já no requirements), ex.: `EMAIL_BACKEND=anymail.backends.resend.EmailBackend` + `RESEND_API_KEY` (ou Brevo/SendGrid/Mailgun/Postmark — §14 i). `DEFAULT_FROM_EMAIL` = remetente de um **domínio verificado** no provedor; sem a chave ou com o remetente de exemplo, W011 e nada sai. SMTP (`EMAIL_HOST*`) só no plano Pro
`WHATSAPP_NUMERO` **(ou tela Branding)**	Botões de WhatsApp e fallback do agendamento (W004)
`CLINIC_EMAIL` **(ou tela Branding)**	Canal do titular LGPD e fallback do alerta de detrator (W007)
`CLINIC_RAZAO_SOCIAL`, `CLINIC_CNPJ`, `ENCARREGADO_NOME`, `ENCARREGADO_EMAIL` **(ou tela Branding)**	Controlador e encarregado (art. 41) na política de privacidade; vazios somem do texto

13.3.2 Recomendadas

Variável	Propósito
`WHATSAPP_TOKEN`, `WHATSAPP_PHONE_ID`, `WHATSAPP_VERIFY_TOKEN` **(32+ caracteres, E003)**, `WHATSAPP_APP_SECRET`, `WHATSAPP_TEMPLATE_D1`/`_NPS`/`_LISTA_ESPERA`	Lembrete D-1, NPS, aviso de vaga e o webhook (nota do NPS, "PARAR"). Sem o canal o lembrete D-1 vai por e-mail
`TURNSTILE_SITE_KEY`, `TURNSTILE_SECRET_KEY`	Captcha no pedido de OTP; com SMS real sem captcha poucos IPs esgotam a cota global (W009)
`SENTRY_DSN` (+ `SENTRY_TRACES_SAMPLE_RATE`, `SENTRY_ENVIRONMENT`, `SENTRY_RELEASE`)	Monitoramento de erros. Ambiente padrão = nome do ambiente no Railway (`RAILWAY_ENVIRONMENT_NAME`: `production`, `dev`) e só depois `DJANGO_ENV` (que a imagem fixa em `prod`); release padrão = `RAILWAY_GIT_COMMIT_SHA`
`WEBPUSH_VAPID_PUBLIC_KEY`, `WEBPUSH_VAPID_PRIVATE_KEY`, `WEBPUSH_VAPID_CLAIMS_EMAIL`	Push ao profissional
`OTP_HMAC_KEY`	Chave própria do HMAC dos códigos OTP (vazia = `DJANGO_SECRET_KEY`): rotacionar uma não mexe na outra
`ADMIN_2FA_OBRIGATORIO`	Sem a env: ligado em prod. `false` = válvula de emergência (W008)
`PROFISSIONAL_2FA_OBRIGATORIO`	Sem a env: ligado em prod (o PROFISSIONAL lê prontuário e alertas de saúde). `false` = válvula de emergência (W012)
`EMBED_FRAME_ANCESTORS`	Origens que podem embutir `/embed/agendar/` (vazio = qualquer `https:`)
`REDIS_URL`	Opcional: cache/sessão compartilhados (obrigatório se `WEB_CONCURRENCY` > 1)
`CRON_MONITOR_URL`	Monitor dos jobs (healthchecks.io: `https://hc-ping.com/<ping key>`): cada job avisa `<url>/<job>` ou `<url>/<job>/fail`

13.3.3 Opcionais / ajuste fino

`CLINIC_NAME`, `CLINIC_SUBTITLE`, `CLINIC_PHONE`, `CLINIC_ADDRESS`, `CLINIC_HOURS`, `INSTAGRAM_URL`, `THEME_COLOR` (a tela Branding vence a env) · `USE_HTTPS` · `USE_X_FORWARDED_HOST` (prod sempre liga) · `CLIENT_IP_HEADER` (prod: `HTTP_X_REAL_IP`) · `WEB_CONCURRENCY` (1), `WEB_THREADS` (4), `WEB_TIMEOUT` (120) · `SESSION_COOKIE_AGE` (28800) · `ADMIN_PASSWORD_RESET` · `SMS_MAX_POR_HORA` (3), `SMS_MAX_POR_IP_HORA` (10), `SMS_MAX_GLOBAL_HORA` (60), `ZENVIA_API_URL`, `ZENVIA_WEBHOOK_SECRET` (segredo do header `X-Zenvia-Signature`), `ZENVIA_ALLOWED_IPS` (IP/CIDR) · `OTP_TTL_SEGUNDOS` (600), `OTP_MAX_TENTATIVAS` (5; fora de 1..10 é o erro E004 — o banco limita a 10), `OTP_REENVIO_MINIMO_SEG` (60) · `WHATSAPP_API_URL`, `WHATSAPP_API_VERSION`, `WHATSAPP_PHONE_NUMBER_ID` (alias) · `EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_USE_TLS`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD` (SMTP, só no plano Pro) · `AXES_FAILURE_LIMIT` (5), `AXES_COOLOFF_TIME_HOURS` (1), `PASSWORD_RESET_TIMEOUT_SECONDS` (3600) · `RETENCAO_OTP_HORAS`, `RETENCAO_NOTIFICACAO_DIAS`, `RETENCAO_LOG_AUDITORIA_DIAS` (também a trilha `historico_alteracao`; os três estão publicados na política — mudar sem versão nova do termo gera o W016), `RETENCAO_LINK_ACESSO_DIAS` (90), `RETENCAO_AXES_LOG_DIAS` (os prazos LGPD — cliente inativa, carência, saúde, ficha, pacote — **não** são env: ficam fixos no settings, §9.4) · `DJANGO_LOG_LEVEL` · `EMAIL_TIMEOUT` (10; vale também para as chamadas HTTP do anymail) · `MAILGUN_SENDER_DOMAIN` (Mailgun; padrão = domínio do remetente) · `DB_STATEMENT_TIMEOUT_MS` (30000; teto por statement na conexão Postgres, abaixo do `WEB_TIMEOUT`; `0` desliga; o `migrate_atomico` roda sem teto), `DB_APPLICATION_NAME` (`shivazen`; nome da conexão no `pg_stat_activity` e no log — ex.: `shivazen-cron` no serviço do cron, `shivazen-staging`).

Só dev/testes: `DEBUG`, `SMS_DEV_LOG_ONLY` (em prod gera W002), `DB_ENGINE`/`DB_*`, `TEST_DATABASE_URL`, `PG_EXIGE_ICU`, `TESTE_SQLITE_MEMORIA`, `TESTE_VITE_BUILD`, `EXIGE_NAVEGADOR`, `VARREDURA_ROTAS` (filtro opcional da varredura visual: só as rotas cujo nome contém o texto). Só no ambiente de testes publicado (`DJANGO_ENV=staging`, §13.1): `DESTINATARIOS_TESTE` (e-mails/celulares separados por vírgula — os únicos que recebem mensagem) (`DESENVOLVIMENTO.md`).

Legada — remover do Railway: `STATIC_ROOT` (era `/tmp`; `settings/prod.py` ignora e usa `/app/staticfiles`, preenchido no build). O código não lê mais `GOOGLE_OAUTH_*` (Google Agenda removido), `AWS_*` (sem upload), `TWO_FACTOR_REMEMBER_COOKIE_AGE`, `ZENVIA_API_KEY` (use `ZENVIA_API_TOKEN` + `ZENVIA_FROM`), `SECRET_KEY` (use `DJANGO_SECRET_KEY`) nem `LGPD_RETENCAO_CLIENTE_DIAS` (o prazo é o publicado na política: `RETENCAO_CLIENTE_INATIVO_DIAS` fixo no settings).

Fixos no settings, sem env (mudar = código + versão nova do termo, quando for prazo LGPD): os `RETENCAO_*` LGPD do §9.4 e `OTP_LATENCIA_MINIMA_SEGUNDOS` (0,8 s; 0 em `settings/test.py`).

14. Convenções de código

- Regra de negócio em `services/`/`utils/`, não em view; mudança de status só pela FSM.
- Camadas `views > services > models` (para baixo pode, para cima não) e nada importa `views` fora dela; `utils` não importa `services`; as áreas de views (`publico`, `painel`, `profissional`, `conta`, `infra`) não se importam — helper de view compartilhado vai para `views/comum.py`, regra para `services/`: `lint-imports` no CI (`pyproject.toml [tool.importlinter]`; exceção = inversão consciente listada lá).
- Configuração por ambiente só em `clinica/settings` (lida e validada no boot); o código lê `settings.X` na hora da chamada (`override_settings` funciona nos testes). O ruff `TID251` barra `os.environ`/`os.getenv` fora dali (exceções no `pyproject.toml`); variável nova entra também no `.env.example` (`EnvExampleTests`).
- Resposta JSON de view = `utils/respostas` (`json_ok`/`json_erro`/`json_recusa`); telefone = `utils/telefone` (normalizar, validar, E.164, exibição, máscara), fonte única da forma canônica.
- Entrada de formulário do painel = Django Form (`forms/`), não `request.POST.get` campo a campo; erro vira `messages.error` por `forms.painel.avisar_erros`.
- Recusa de regra = `excecoes.RegraDeNegocio` com mensagem pronta para a tela (nunca `None`/`False`/exceção privada da view); horário de atendimento só por `services/agenda`.
- Toda escrita multi-tabela em `transaction.atomic`; I/O externo (e-mail, WhatsApp, push) só no `on_commit`, best-effort.
- Datas "hoje/amanhã" sempre no fuso da clínica (`utils/datas.hoje`), nunca `date.today()`; data/hora de formulário por `utils/datas.ler_data`/`ler_hora`. O ruff (`DTZ`) barra no CI e no pre-commit. Filtro "do dia" em `DateTimeField` = `filter(**dias_locais('campo', dia))` (intervalo que usa o índice), nunca `campo__date=`.
- Lista com dado por linha (alerta de saúde, contagem...): carregue em lote (`alertas_em_lote`, `select_related`/`annotate`), nunca uma consulta por linha — `tests/test_consultas_por_tela.py`.
- Ação sensível → `utils/auditoria.registrar_log(usuario, acao, tabela, pk, detalhes)` — registrador único; autor e IP vêm do `Ator` da borda (`ator.py`: middleware `web`, cron `cron:<job>`), `usuario`/`request` explícitos só quando o Ator não basta (sem nome de cliente no texto — use pk em `detalhes`).
- Templates: CSP estrita (sem `onclick`, `<script>`/`<style>` com `nonce`), lógica Alpine em `Alpine.data()` no `app.js` (nas telas da equipe, comportamento de componente no `painel.js`, vanilla), tokens de cor do `tokens.css` (nunca cor crua), sem `style="..."` (classe do `componentes.css`/`site.css` ou utilitário). Tela da equipe usa os componentes cotton do design system antes de classe solta. Lib de terceiro entra pelo `package.json` e sai no bundle do Vite — nunca por CDN (`test_csp` reprova).
- Copy do site em tom de estética/bem-estar (cliente, atendimento, avaliação — nunca "paciente/consulta médica"). Texto que também é regra (política de privacidade, FAQ) mora em `conteudo/` e sai das mesmas constantes/settings que o sistema aplica — nunca número escrito à mão no template.
- E-mail (`templates/email/`): layout em tabela; faixa com gradiente leva `bgcolor` + `background-color` de reserva (o Outlook ignora gradiente) e botão vai num `<td bgcolor>` com `mso-padding-alt` (o Outlook ignora padding em `<a>`); texto do botão escuro (`#23200f`). `test_email_texto.EmailNoOutlookTests` cobra.
- Títulos com só a 1ª maiúscula ("Lista de espera", "Perguntas frequentes"); nome de tratamento do catálogo e sigla ficam como são. Dinheiro sempre `{{ valor|brl }}` (R$ 1.350,00; filtro builtin, `utils/formatos.formatar_brl`), nunca `R$ {{ x|floatformat }}`; telefone exibido sempre `{{ tel|telefone }}` (o canônico cru fica só em `href`/`value`).
- Migrations: nunca editar migration já aplicada em produção; DDL só-Postgres via `RunPython` com checagem de vendor; teste no Postgres (CI) antes do merge.

- Commits: Conventional Commits (`feat(scope)`, `fix(scope)`, `docs(scope)`...).
- Docs: este arquivo + requisitos (`docs/tcc/_dados_requisitos.json`, lido por `python docs/tcc/gerar_documentacao.py`) no mesmo PR da mudança.

15. Pendências técnicas e de operação

Itens em aberto (ARCHITECTURE.md §6.2), verificados no código. O que é de operação depende da dona da clínica (revisão jurídica, contas e chaves dos provedores); o que é de código é de baixa prioridade e não bloqueia o go-live.

- **Revisão jurídica da Política de Privacidade** (`aranha_estetica/conteudo/lgpd.py`: texto, `TRATAMENTOS`, `OPERADORES`, `PRAZOS_RETENCAO`) pela dona, de preferência com um advogado, **antes do primeiro deploy de produção** — a `0045` grava esse texto no banco novo e, depois do 1º aceite, mudar exige versão nova (runbook §14 a; `LGPD.md`)
- Backup configurado antes do 1º cliente real (o go-live cria o banco novo; ensaio 0026→0079 só se o banco antigo for reaproveitado — runbook §14 a.1)
- Env no Railway (`DATABASE_URL` por referência, ADMIN_*, SITE_URL, ZENVIA_*, EMAIL_BACKEND + provedor, WHATSAPP_*, TURNSTILE_*, SENTRY_DSN, CLINIC_EMAIL; CRON_TOKEN só com o cron HTTP) e remover `STATIC_ROOT` e as legadas (`GOOGLE_OAUTH_*`, `AWS_*`)
- Provedor de e-mail: `django-anymail` já instalado — escolher o provedor (ex.: Resend) e definir `EMAIL_BACKEND` + chave + `DEFAULT_FROM_EMAIL` de domínio verificado (check W011)
- Serviço Cron dos jobs (`railway.cron.json` + `rodar_jobs_devidos`, runbook §13.2) e o monitor (`CRON_MONITOR_URL`)
- Pós-deploy: 2FA do ADMIN, apagar `ADMIN_PASSWORD`, `axes_reset`, Branding real (razão social, CNPJ e encarregado aparecem na moldura da política), conferir que o termo LGPD v1.0 ativo é o texto revisado, mesclar duplicatas logadas pela 0034, `VALIDATE CONSTRAINT` dos CHECKs `NOT VALID` da 0043
- Hardening opcional do 2FA: settings de teste com `ADMIN_2FA_OBRIGATORIO=False` e tirar a dependência do marcador `usuario_id` no `Enforce2FAMiddleware`
- Baixa: `painel/termo_link.html` oferece `mailto:` para o e-mail do cadastro (não verificado)
- Tirar o jsDelivr da lista de operadores da política (`conteudo/lgpd.OPERADORES`, cobrado em `tests/test_politica_dsar.py`) e da tabela de `LGPD.md` §3: desde a D50 nenhuma página carrega nada dele. Junto com a revisão jurídica da política (item da operação, acima)
- Registry de regras (spec 5.1) e sessão única de cliente verificado (5.3) — ver `specs/regras-negocio-registry.md`
- Custo: código de acesso pelo WhatsApp (template de **autenticação**, R$ 0,035) com SMS só de reserva, e SMS de reserva num provedor pré-pago sem mensalidade (`utils/sms`); teto diário dos e-mails de marketing na fila (campanha em lotes, dentro do plano gratuito). Juntos levam o cenário Rotina de ≈ R$ 271 para ≈ R$ 82/mês (`CUSTOS-OPERACIONAIS.md` §6)
- Decidido não fazer (por ora): UNIQUE de regra de comissão ativa (quebraria o desempate por "mais recente"); consentimento de WhatsApp para marketing. (O trigger de `updated_at` entrou na 0056; nomes em inglês na 0079.)

16. Custo operacional

16.1 Premissas

- **Câmbio**: US$ 1 = R$ 5,10. A PTAX de 16/10/2026 foi de R$ 5,08, arredondada para cima.
- **IOF de 3,5%** sobre compra internacional no cartão. O custo efetivo fica em **R$ 5,28 por dólar** e é o valor usado em toda conversão deste documento.
- **Hospedagem**: plano Hobby do Railway, com um ambiente de produção (web + PostgreSQL 18) e um serviço Cron a cada 15 min.
- **Ambiente de testes**: o ambiente publicado da branch `dev` fica **desligado** fora dos ensaios (ver "Custos que não são mensais").
- **Volume**: estimado por atendimento realizado no mês, a partir das regras de comunicação do sistema (`REGRAS-DE-NEGOCIO.md` §23 e §24):

Mensagem por atendimento	Estimativa	Por quê
SMS (código OTP)	1,0	um código por pedido online e por acesso à área da cliente sem sessão; reenvio limitado
WhatsApp de utilidade	1,25	lembrete D-1 de quem autorizou (~0,8) + NPS no máximo a cada 90 dias por cliente (~0,4) + aviso de vaga da lista de espera (~0,05)
E-mail transacional	1,6	confirmação, termo, lembrete D-1 de quem não autorizou WhatsApp, reset de senha e convites da equipe
E-mail de marketing	por campanha	promoção disparada pelo painel só para quem consentiu + felicitação de aniversário

- Os preços são de **lista**. Os valores de infraestrutura são **estimativas de consumo**, não medições: com o trial vencido, os serviços estão parados e o Railway não tem métricas recentes. O banco de demonstração ocupa 0,12 GB.

16.2 Itens de custo

Item	Provedor	Como cobra	Preço de lista	Situação no sistema
Hospedagem web + banco + cron	Railway (Hobby)	US$ 5/mês com US$ 5 de uso incluídos; o excedente é cobrado por segundo	vCPU ≈ US$ 20/mês, RAM ≈ US$ 10/GB-mês, volume US$ 0,15/GB-mês, saída US$ 0,05/GB	1 réplica, gunicorn com 1 worker e 4 threads; cron roda e desliga
Domínio `.com.br`	Registro.br	anual	R$ 40/ano	DNS do próprio Registro.br, sem custo
SMS (OTP)	Zenvia	pacote mensal	R$ 100 (até 1.000 SMS, R$ 0,10 cada); R$ 250 (até 2.942, R$ 0,085)	só OTP; cotas de 3/h por telefone, 10/h por IP e 60/h no total
WhatsApp	Meta (Cloud API direta)	por mensagem entregue	utilidade R$ 0,035; marketing R$ 0,3217; resposta de atendimento: 1.000/mês grátis	só templates de **utilidade** (lembrete, NPS, vaga) e só com consentimento; marketing não sai por WhatsApp
E-mail	Resend (ou Brevo, Mailgun, SendGrid, Postmark pelo anymail)	plano mensal	Resend Free: 3.000/mês, até 100/dia; Pro: US$ 20 (50 mil/mês, sem limite diário). Brevo Free: 300/dia	o Hobby do Railway bloqueia SMTP; o e-mail sai por API HTTP
Backup externo	Cloudflare R2	por GB-mês	10 GB-mês grátis; depois US$ 0,015/GB	dump cifrado (age) diário; 30 diários + 12 mensais cabem no gratuito

Erros	Sentry (Developer)	plano	grátis: 5 mil erros/mês, 1 usuário	filtros de dado pessoal em `utils/pii`
Monitor dos jobs	healthchecks.io (Hobbyist)	plano	grátis: 20 checks	`CRON_MONITOR_URL`
Antirrobô	Cloudflare Turnstile	—	grátis	formulários públicos e pedido de SMS
Notificação da equipe	Web Push (VAPID)	—	grátis (serviço do navegador)	novo agendamento, cancelamento, sem desfecho
Agenda da profissional	feed ICS	—	grátis	sem API do Google Agenda
Mapa	link do Google Maps	—	grátis	sem mapa embutido, que exigiria chave paga
Fontes e bibliotecas	hospedadas no próprio site	—	grátis	sem Google Fonts nem CDN

16.3 Estimativa da hospedagem (Railway)

Serviço	Início (150 atend./mês)	Rotina (400)	Agenda cheia (800)
Web (RAM / CPU média)	0,30 GB / 0,03 vCPU → US$ 3,60	0,35 GB / 0,05 vCPU → US$ 4,50	2 workers: 0,60 GB / 0,10 vCPU → US$ 8,00
PostgreSQL (RAM / CPU / disco)	0,20 GB / 0,01 / 1 GB → US$ 2,35	0,25 GB / 0,02 / 2 GB → US$ 3,20	0,40 GB / 0,05 / 3 GB → US$ 5,45
Cron (96 execuções/dia, ~20 s cada)	US$ 0,35	US$ 0,40	US$ 0,50
Saída de dados	US$ 0,10	US$ 0,25	US$ 0,50
Fatura (mín. US$ 5)	**≈ US$ 6,50 → R$ 34**	**≈ US$ 8,50 → R$ 45**	**≈ US$ 14,50 → R$ 77**

O consumo fica baixo por decisões do próprio sistema:

- Páginas renderizadas no servidor, sem SPA.
- Um processo só, sem Redis nem Celery (os jobs rodam pelo serviço Cron, que liga, executa e desliga).
- `statement_timeout` de 30 s.
- Testes que reprovam consultas N+1 no painel.

Para ver o consumo real depois do go-live, use o painel **Usage** do Railway. Com ele, refaça esta tabela no primeiro mês.

16.4 Custo mensal por cenário

Item	Início (150)	Rotina (400)	Agenda cheia (800)
Railway	R$ 34,00	R$ 45,00	R$ 77,00
Domínio (R$ 40 ÷ 12)	R$ 3,33	R$ 3,33	R$ 3,33
SMS Zenvia (150 / 400 / 800 SMS)	R$ 100,00	R$ 100,00	R$ 100,00
WhatsApp utilidade (190 / 500 / 1.000)	R$ 6,65	R$ 17,50	R$ 35,00
E-mail (ver "E-mail: qual plano")	R$ 0,00	R$ 105,60	R$ 105,60
Backup, Sentry, healthchecks, Turnstile, push	R$ 0,00	R$ 0,00	R$ 0,00
Total por mês	**≈ R$ 144**	**≈ R$ 271**	**≈ R$ 321**
Sem plano pago de e-mail (campanha de até 300 envios/dia no Brevo Free)	≈ R$ 144	≈ R$ 166	≈ R$ 215
Por atendimento	R$ 0,96	R$ 0,68 (R$ 0,41)	R$ 0,40 (R$ 0,27)
Por ano	≈ R$ 1.730	≈ R$ 3.260	≈ R$ 3.850

Leitura:

- **O SMS é o maior custo fixo.** O pacote mínimo de R$ 100 vale até 1.000 códigos e, nos três cenários, ficaria subutilizado.
- **O e-mail pago só é preciso quando há campanha.** O Resend Free (100/dia) cobre os transacionais; uma promoção para uma base de 400 a 1.500 clientes que consentiram estoura o limite diário.
- **O custo cai por atendimento à medida que a agenda enche**, porque quase tudo é fixo.
- **No ticket da clínica, o sistema custa menos de 1% de um atendimento.** O catálogo vai de R$ 60 (drenagem facial) a R$ 2.300; num atendimento de R$ 150, R$ 0,68 equivale a 0,45%. Evitar **uma** falta de R$ 300 por mês (o lembrete D-1 existe para isso) já paga a operação do cenário Rotina.

16.5 E-mail: qual plano

Situação	Plano	Custo
Só transacionais (até ~100/dia)	Resend Free	R$ 0
Campanhas para até ~300 clientes por dia	Brevo Free (300/dia)	R$ 0 — confirmar se o plano gratuito põe marca do Brevo no e-mail
Campanhas maiores ou mais de uma por semana	Resend Pro (50 mil/mês, sem limite diário)	US$ 20 → R$ 105,60

A troca de provedor é só de configuração: `EMAIL_BACKEND=anymail.backends.<provedor>.EmailBackend` e a chave do provedor na env (`PROJECT.md` §14 i). A fila de e-mail reenvia o que falhou a cada 15 min.

16.6 Alternativas para gastar menos

Comparadas no cenário Rotina (400 atendimentos/mês, ≈ R$ 271):

Alternativa	Economia/mês	O que exige	Decisão
Código de acesso pelo WhatsApp (template de autenticação, R$ 0,035) no lugar do SMS; SMS só para quem não tem WhatsApp	≈ R$ 84 (de R$ 100 para ~R$ 16)	Código: um canal novo no OTP (o envio de template já existe em `utils/whatsapp`) e um template de **autenticação** aprovado na Meta, com botão "copiar código"	**Recomendada**: a maior economia, e o WhatsApp é o canal que a cliente já usa
SMS pré-pago sem mensalidade (SMSDev, Mobizon, Comtele, SMSGo: ~R$ 0,06 a R$ 0,07 por SMS) no lugar do pacote mínimo da Zenvia	≈ R$ 74 (400 × R$ 0,065 = R$ 26)	Código: outro provedor em `utils/sms` (o envio da Zenvia está isolado nesse módulo); conferir se os créditos expiram	**Recomendada** como reserva da alternativa anterior
Campanha de e-mail em lotes diários (até ~80 por dia, dentro do Resend Free)	≈ R$ 106 (sem o Resend Pro)	Código pequeno: teto diário de e-mails de marketing na fila. A promoção leva alguns dias para chegar a toda a base	**Recomendada**. Sem código: Brevo Free (300/dia)
Ambiente de testes dormindo (Serverless do Railway: o serviço para após 10 min sem tráfego e acorda na próxima visita) e sem serviço Cron no `dev`	≈ R$ 26 a R$ 32 (o ambiente ligado o mês todo custaria ≈ US$ 5 a 6)	Só configuração	**Aplicada** na infraestrutura do Railway
Amazon SES (US$ 0,10 por mil e-mails)	≈ R$ 106	Conta na AWS, outra dependência no anymail; os planos do SES mudaram em 2026	Descartada: o lote diário dá a mesma economia sem nova conta
VPS própria (ex.: Hetzner CX23, ~€ 5,49/mês) com Docker	~R$ 10	A clínica passa a cuidar de sistema operacional, certificado, backup e monitoramento	Descartada: economia pequena para muito mais operação

Nuvem gratuita (Oracle Always Free) ou banco grátis externo (Neon)	≈ R$ 15 a R$ 45	Sem garantia de serviço; banco fora da rede do app (latência, "acordar" lento)	Descartada: risco alto para dado de saúde
Lembrete D-1 só por e-mail	≈ R$ 14	Só configuração	Descartada: o WhatsApp reduz falta, e uma falta evitada vale mais que o mês inteiro

Cenário otimizado (as três recomendadas + ambiente de testes dormindo):

Item	Início (150)	Rotina (400)	Agenda cheia (800)
Railway (produção)	R$ 34,00	R$ 45,00	R$ 77,00
Domínio	R$ 3,33	R$ 3,33	R$ 3,33
Código pelo WhatsApp (R$ 0,035)	R$ 5,25	R$ 14,00	R$ 28,00
SMS de reserva (10%, R$ 0,065)	R$ 1,00	R$ 2,60	R$ 5,20
WhatsApp utilidade	R$ 6,65	R$ 17,50	R$ 35,00
E-mail (Resend Free, campanha em lotes)	R$ 0,00	R$ 0,00	R$ 0,00
Total por mês	**≈ R$ 50** (−65%)	**≈ R$ 82** (−70%)	**≈ R$ 149** (−54%)
Por atendimento	R$ 0,33	R$ 0,21	R$ 0,19

As duas mudanças de código (código pelo WhatsApp e lote diário de e-mail) ficam como próximos passos em `ARCHITECTURE.md` (pendências).

16.7 Custos que não são mensais

- **Na implantação (uma vez):**
  - Domínio no 1º ano: R$ 40.
  - Número de celular dedicado ao WhatsApp da API (chip pré-pago). Esse número não pode estar em uso no aplicativo WhatsApp comum.
  - Verificação do negócio na Meta: sem custo, mas exige CNPJ.
  - Contas no Railway, Sentry, Cloudflare e healthchecks: sem custo. O Railway pede cartão.
  - **Sem** a taxa de ativação de R$ 649 da Zenvia. Ela só existe para WhatsApp/RCS pela Zenvia, e o sistema fala com a Meta direto.
- **Ambiente de testes publicado** (branch `dev`): ligado o mês todo, custaria quase o mesmo que a produção (outro web + outro Postgres, ≈ US$ 5 a 6/mês a mais). Por isso o web do `dev` está no modo Serverless (dorme sem tráfego) e o `dev` não tem serviço Cron (`PROJECT.md` §13.1).
- **Plano Pro do Railway** (US$ 20/mês com US$ 20 de uso, ≈ R$ 106): só se a clínica precisar de mais de uma pessoa na conta ou de suporte do Railway. O consumo estimado cabe no crédito incluído.

16.8 Travas de custo no sistema

Cada canal pago tem um teto no código. Abuso ou erro não vira fatura:

Risco	Trava	Onde
Robô pedindo SMS em massa (*SMS pumping*)	Turnstile + 3 SMS/h por telefone, 10/h por IP e **60/h no site todo**: o pior caso fica em ~R$ 6/h e para no teto	`utils/sms`, env `SMS_MAX_*`
Reenvio de código em sequência	60 s entre reenvios; código vale 10 min	`OTP_REENVIO_MINIMO_SEG`
Spam pelo formulário	5/min e 20/dia por IP; teto de 50/dia no site	`views/publico/contato`

de contato		
NPS repetido	no máximo 1 pesquisa a cada 90 dias por cliente, nunca em retorno	`constants.NPS_INTERVALO_DIAS`
Marketing caro por WhatsApp	promoção só por e-mail; WhatsApp só em template de utilidade, ~9× mais barato que o de marketing	`services/comunicacao`, `services/marketing`
Lembrete para quem não aceitou WhatsApp	vai por e-mail (custo zero)	`services/lembretes`
Envio sem provedor configurado	falha fechada: nada sai, nada é cobrado nem marcado como enviado	`utils/sms`, `utils/whatsapp`
Avisos à equipe	Web Push gratuito em vez de SMS ou WhatsApp	`services/push`

16.9 Esforço de operação (tempo, não dinheiro)

Frequência	Tarefa	Tempo
Semanal	Olhar o Sentry e o healthchecks (job atrasado, erro novo)	~15 min
Mensal	Testar a restauração do backup (`scripts/backup/testar_restauracao.sh`)	~30 min
Mensal	Conferir as faturas (Railway, Meta, Zenvia, e-mail) e o saldo do pacote de SMS	~15 min
Anual	Renovar o domínio e revisar este documento com os preços do ano	~1 h
Quando mudar o texto	Reaprovar o template do WhatsApp na Meta	~1 dia de espera

16.10 Fontes (consultadas em 30/10/2026)

- Railway, planos e tarifas: https://railway.com/pricing
- Meta, modelo de cobrança do WhatsApp Business Platform: https://developers.facebook.com/docs/whatsapp/pricing
- Tarifas do Brasil, em vigor desde 01/10/2026 (tabela da Meta reproduzida por terceiro): https://zynfo.ai/whatsapp-pricing/brazil
- Zenvia, pacotes de SMS: https://zenvia.com/zoe/
- Resend, planos: https://resend.com/pricing
- Brevo, plano gratuito de 300 e-mails/dia (resenha de terceiro): https://www.sender.net/reviews/brevo/pricing/free-plan
- Registro.br, valor de R$ 40/ano: https://www.nuvemshop.com.br/blog/registro-br
- Sentry, plano Developer gratuito: https://costbench.com/software/developer-tools/sentry/free-plan/
- healthchecks.io, planos: https://healthchecks.io/pricing/
- Cloudflare R2, cota gratuita: https://developers.cloudflare.com/r2/pricing
- Railway Serverless: https://docs.railway.com/reference/app-sleeping
- SMS sem mensalidade no Brasil (comparativo de provedores): https://x-apps.com.br/sms-verificacao-otp-brasil-provedores-precos/ , https://smsgo.com.br/ , https://www.smsdev.com.br/ , https://comtele.com.br/disparo-sms/
- WhatsApp, tarifa de autenticação no Brasil igual à de utilidade (US$ 0,0068): https://www.authgear.com/post/whatsapp-api-pricing
- Amazon SES: https://www.usecarly.com/blog/amazon-ses-pricing/
- Hetzner, reajustes de 2026 (CX23): https://privatedevops.com/news/hetzner-june-2026-cloud-price-increase-what-to-do
- Câmbio (PTAX de 16/10/2026): https://www.simianx.ai/pt/stories/us-dollar-exchange-rate-by-year-every-year-1971-2026-table
- Cotação oficial: https://www.bcb.gov.br/estabilidadefinanceira/historicocotacoes

Algumas tarifas vieram de páginas de terceiros. Antes de contratar, confira o valor na página do próprio provedor.

17. Identidade visual, design e marketing

17.1 Posicionamento

A Jaqueline Aranha Estética é uma clínica de **estética e bem-estar** em São José do Rio Preto/SP. O catálogo vai da massagem relaxante e da drenagem a injetáveis (toxina botulínica, preenchimentos, bioestimulador).

A identidade precisa resolver uma tensão:

- **Confiança técnica**, porque são procedimentos no rosto e no corpo.
- **Acolhimento e prazer**, porque é cuidado pessoal, não tratamento de doença.

A direção visual escolhida, **"Clínico-premium"** (decisão D4), fica no meio do caminho:

- **Premium:** o dourado e a tipografia serifada passam valor e cuidado.
- **Sóbrio:** os neutros claros e o texto de alto contraste passam limpeza e seriedade.
- **Nunca hospitalar:** nada de branco-gelo com azul ou verde cirúrgico, nada de cruz nem de linguagem de consultório.

O site é um **funil para o agendamento**. Cada página leva a "Agendar avaliação" ou ao WhatsApp da clínica.

17.2 Paleta e psicologia das cores

A cor é o primeiro sinal de personalidade de uma marca. Matiz, saturação e luminosidade mudam a personalidade percebida (sinceridade, entusiasmo, competência, sofisticação, robustez), como mostram Labrecque e Milne (2012). Os significados das cores são culturais e dependem do contexto (Heller, 2013): por isso a paleta foi escolhida pelo **conjunto** e pelo uso, não por uma cor isolada.

Papel	Cor	Onde aparece	Por quê
Marca	Dourado `#C9A84C`	botões principais, ícones, fios decorativos, cor do tema do PWA	Ver "Por que dourado", logo abaixo.
Dourado para texto	`#7A5F1F` (claro) / `#C9A84C` (escuro)	links, títulos em destaque, anel de foco	O `#C9A84C` como texto no fundo claro dá só 2,2:1. O tom escurecido dá 5,8:1 (AA) e continua lido como "dourado".
Texto sobre o dourado	`#23200F` (café quase preto)	rótulo dos botões principais	7,2:1 sobre o dourado. Mais quente que o preto puro e sem o brilho do branco sobre o amarelo.
Fundo	`#FAFAF8` (off-white quente)	página	O branco puro (`#FFFFFF`) com luz fria é a cor do hospital. O off-white levemente quente lembra papel, linho e toalha de spa.
Superfície	`#FFFFFF`	cartões	O cartão se destaca do fundo sem precisar de borda forte.
Areia	`#F1EFE8`	realces, faixas alternadas	Respiro entre seções, no registro "natural/terra".
Texto	Grafite `#1B1F23`	corpo	15,9:1 no fundo. Menos duro que o preto absoluto em leitura longa.
Texto suave	Taupe `#6B6258`	legendas, metadados	Cinza **quente** (puxado ao marrom) em vez de cinza azulado: conversa com o dourado. 5,7:1 (AA).
Menu da equipe	Café `#1F1A14`	barra lateral do painel e do portal	Substituiu o **azul-marinho** do template comprado. O azul passa "corporativo/sistema" e não tem relação com a marca. O café com destaque dourado (`#E3C674`, 7,9:1) faz o painel parecer da mesma casa que o site.

Rodapé	#16181C	rodapé do site	Fecha a página com peso visual e faz o dourado do logo brilhar.
Ação do WhatsApp	Verde #1D7A4F	botão flutuante e CTAs de WhatsApp	O verde é reconhecido como "WhatsApp/seguir". O verde oficial #25D366 com texto branco dá 2:1 (ilegível); o tom fechado dá 5,3:1.
Erro	Vermelho tijolo #A32D2D	mensagens de erro	Sério sem ser alarmante. Nada de vermelho de emergência.

17.2.1 Por que dourado

- **Valor e cuidado especial.** O ouro é associado a valor, celebração e luxo (Heller, 2013). Ele passa a ideia de "momento de cuidado consigo" que a estética vende, e é a linguagem visual de cosméticos e spas de categoria superior.
- **Ouro velho, não amarelo.** O tom é **dessaturado e terroso**, não o amarelo-limão nem o metalizado brilhante. Em excesso, o dourado vira ostentação. Por isso ele é **acento** (botão, ícone, fio) e nunca fundo de grandes áreas.
- **Calor sem infantilizar.** O dourado é quente como o rosa e o pêssego, mais comuns em estética, mas não infantiliza a marca nem restringe o público.
- **Diferenciação.** Clínicas de estética da região tendem ao rosa, ao lilás ou ao azul-clínico. O dourado com café se destaca sem fugir do universo da beleza.
- **Decisão do dono.** O dono da clínica confirmou o #C9A84C como cor final da marca.

17.2.2 Cores que foram descartadas

Opção	Por que não
Azul + branco frio	Lido como hospital, laboratório ou plano de saúde: o contrário de "bem-estar".
Verde-água / menta	Mesmo problema (centro cirúrgico, farmácia). Fica só como verde de ação do WhatsApp.
Rosa / lilás dominante	Clichê do segmento, pouca diferenciação e tom infantil para procedimentos injetáveis.
Preto + dourado brilhante	Luxo "noturno" e distante (joalheria, balada). Afasta quem busca acolhimento.
Azul-marinho no painel	Herança do template comprado, sem relação com a marca. Trocado pelo café (tabela da paleta).

17.2.3 Cores de status

A agenda usa cores de status por convenção, para leitura rápida pela equipe:

Status	Cor
Pendente	âmbar
Agendado	cinza
Confirmado	azul
Realizado	verde
Cancelado	vermelho
Faltou	laranja
Reagendado	roxo

- **Sempre com o texto.** O status nunca depende só da cor (WCAG 1.4.1): o selo mostra o nome, para quem tem daltonismo.
- **Fundo claro + texto escuro** do mesmo matiz, com AA nos dois temas.

17.2.4 Tema escuro

- **Disponível nas duas pontas.** Site, painel e portal têm tema escuro: segue o sistema ou é escolhido no botão do cabeçalho, e a escolha fica num cookie.
- **Por quê:**
  - Conforto à noite: a cliente costuma agendar pelo celular fora do horário comercial.
  - Menos brilho para a equipe que passa o dia na agenda.
- **Como:**
  - Fundo grafite azulado `#14171A`, não preto puro.
  - Texto `#EDEFEC`.
  - O dourado da marca volta a ser cor de texto (7,9:1 sobre o fundo escuro).

17.3 Tipografia

Uso	Fonte	Por quê
Títulos	**Playfair Display** (serifada, 500/600/700)	Serifa de alto contraste entre traços finos e grossos, no estilo didone das revistas de moda e beleza. Passa elegância e o lado "editorial/premium" da marca.
Texto e interface	**Lato** (sem serifa humanista, 400/700)	Formas abertas e arredondadas: legível em tela pequena e mais calorosa que as sem serifa geométricas ou técnicas. Boa nos números da agenda e das tabelas.

- **Contraste entre as duas.** O par serifa + sem serifa separa o "tom de marca" dos títulos da "clareza de uso" do texto.
- **Hospedadas no próprio site** (decisão F9), sem Google Fonts:
  - nenhum IP de visitante vai para terceiro (LGPD);
  - nada a mais na política de segurança (CSP);
  - só o subconjunto latino e os pesos usados (página mais leve).
- **Escala das telas da equipe:**
  - texto base de 15 px;
  - rótulos de 13 px;
  - **nada abaixo de 12 px**;
  - espaçamento em múltiplos de 4 px;
  - cantos de 8 px (controle), 12 px (menu, aviso) e 16 px (cartão, janela).
- **Caixa de frase** em títulos e botões ("Agendar avaliação", "Perguntas frequentes"). É mais conversacional e mais fácil de ler que Tudo Em Maiúsculas.

17.4 Tom de voz e conteúdo

- **Estética, não hospital.** O template comprado veio com textos de consultório. Eles foram trocados:

Antes (template)	Depois
paciente	cliente
consulta	avaliação / horário / atendimento

prontuário (no site)	histórico de cuidados
"pronto atendimento", "intercorrência"	acompanhamento pós-procedimento

- **Caloroso e direto.** Frases curtas na segunda pessoa ("Escolha o melhor horário"). Erros dizem o que fazer ("Escolha a data e um dos horários livres"), sem códigos técnicos.
- **Ícones** de folha, flor e brilho, em traço fino, em vez de escudo, cadeado ou cruz médica.
- **O catálogo real** (serviços e preços informados pela clínica) substituiu o conteúdo genérico. O preço aparece no agendamento, por transparência.

17.5 Experiência e layout

- **Celular primeiro.** A cliente chega pelo Instagram, pelo WhatsApp ou pela busca no celular. Cada tela é desenhada primeiro em 375 px. O CI confere três larguras (celular, tablet e desktop), sem rolagem lateral.
- **Página inicial curta** (de 750 para 313 linhas na reforma):

4. Herói com a chamada.
5. Serviços.
6. Equipe com "Agendar com…".
7. Chamada final.
8. Dúvidas frequentes.

O botão "Agendar avaliação" aparece no topo e no fim da página.

- **Agendamento em etapas:** procedimento → data e horário → dados e código SMS → confirmação. Uma decisão por tela reduz a desistência, e cada etapa tem um botão "Continuar".
- **Botão flutuante do WhatsApp** em todas as páginas do site. É o canal de conversa preferido do público da clínica, e quem não quer agendar sozinho fala com uma pessoa.
- **Animação discreta** ao rolar (as seções surgem suavemente). Ela é desligada para quem pede menos movimento no sistema (`prefers-reduced-motion`).
- **Telas da equipe:**
  - um design system próprio: cartões, tabelas que viram lista no celular, agenda em grade;
  - o painel é um **PWA** (instalável no celular, com notificação de novo agendamento).

17.6 Acessibilidade como decisão de design

- **Meta: WCAG 2.1 nível AA** nos dois temas:
  - texto com 4,5:1;
  - borda de campo com 3:1 (`#8C857A`, 3,6:1);
  - foco do teclado sempre visível, no dourado escuro.
- **Teste em toda rota.** O CI roda o axe-core em todas as rotas do site e do painel (`tests/test_crawl_visual.py`) e reprova violações graves.
- **Formulários:** rótulo visível, ajuda e erro ligados ao campo, e mensagem anunciada ao leitor de tela.
- **Por que importa para o marketing:** acessibilidade amplia o público (pessoas idosas, baixa visão, daltonismo, uso com uma mão no celular) e melhora o SEO.

17.7 Recursos de marketing do sistema

Recurso	O que faz	Por quê
SEO local	título e descrição por página, Open Graph, dados estruturados schema.org (`BeautySalon`/`HealthAndBeautyBusiness`, `Service` com `Offer`, `FAQPage`, `BreadcrumbList`), sitemap e robots; páginas privadas com `noindex`	Aparecer na busca "estética em Rio Preto" e no compartilhamento pelo WhatsApp com imagem e texto certos.
Agendamento online 24 h	a cliente escolhe o horário livre e confirma com um código SMS	Converte na hora em que a vontade surge, sem esperar resposta.
Prova social honesta	nota média só com **10+ avaliações**; "clientes atendidas" só a partir de **50** (arredondado para baixo); anos de experiência só se a clínica preencher	Número real gera confiança. Número inventado é publicidade enganosa (CDC, art. 37).
Depoimentos	só nota ≥ 9 no NPS, com autorização da cliente e aprovação da clínica	Depoimento verdadeiro, com consentimento (LGPD).
NPS pós-atendimento	pesquisa de 0 a 10 por WhatsApp, no máximo a cada 90 dias; nota ≤ 6 gera alerta para a equipe	Mede a satisfação e permite recuperar a cliente insatisfeita antes de uma avaliação pública ruim.
Lembrete D-1	WhatsApp ou e-mail na véspera, com confirmar ou remarcar	Reduz falta (horário vazio é receita perdida) e facilita remarcar em vez de sumir.
Lista de espera	inscrição por procedimento; aviso quando abre vaga	Recupera o horário de quem cancelou.
Promoções	página pública e e-mail disparado pelo painel, só para quem consentiu	Ativa a base atual (mais barato que conquistar cliente nova), dentro da LGPD.
Aniversário	e-mail de felicitação com consentimento	Relacionamento e lembrança da marca na data certa.
Indicação com cashback	a indicadora ganha **R$ 50** de crédito no 1º atendimento pago da indicada	Boca a boca, o canal mais forte em estética, com recompensa rastreável.
Pacotes e retornos	sessões com validade; retorno sem custo dentro da janela do procedimento	Fidelização e resultado completo do tratamento.
Área da cliente	ver, remarcar e cancelar horários, preferências de comunicação e pedidos LGPD	Autonomia e transparência geram confiança.
Confiança e LGPD	só cookies essenciais (sessão, segurança e tema), sem pixel de anúncio nem rastreador de terceiros; política de privacidade clara; descadastro em um clique	Privacidade é argumento de venda numa clínica que guarda dado de saúde. A medição fica no painel (ver "Métricas para acompanhar"), com dados próprios.

Matriz de canais pelo custo. O canal de cada mensagem foi escolhido pelo custo (`CUSTOS-OPERACIONAIS.md`):

- **Marketing vai por e-mail.** É gratuito ou barato. O template de marketing do WhatsApp custa R$ 0,32 por mensagem.
- **WhatsApp é só para utilidade** (lembrete, NPS, vaga), a R$ 0,035.
- **SMS é só para o código de acesso.**

17.8 Métricas para acompanhar

O painel mostra no dia a dia os números que dizem se o design e o marketing funcionam:

- atendimentos do dia e da semana e ocupação da agenda;
- receita do mês e ticket médio;
- clientes ativas nos últimos 90 dias;
- faltas no mês;
- resposta aos lembretes (confirmados, cancelados, sem resposta);

- NPS, com promotores, neutros e detratores;
- valor total por cliente (LTV), na ficha de cada uma.

A recomendação é comparar antes e depois de cada campanha ou mudança no site.

17.9 Referências

- HELLER, Eva. *A psicologia das cores: como as cores afetam a emoção e a razão*. São Paulo: Gustavo Gili, 2013.
- LABRECQUE, Lauren I.; MILNE, George R. Exciting red and competent blue: the importance of color in marketing. *Journal of the Academy of Marketing Science*, v. 40, n. 5, p. 711–727, 2012.
- W3C. *Web Content Accessibility Guidelines (WCAG) 2.1*. 2018. https://www.w3.org/TR/WCAG21/
- BRASIL. Lei nº 8.078, de 11 de setembro de 1990 (Código de Defesa do Consumidor), art. 37.
- BRASIL. Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais).